University of Udine

Department of Mathematics, Computer Science and Physics

Social Computing - 2023/2024

Crowd_Frame:
TECHNICAL DOCUMENTATION

Supervisor:
Prof. Stefano Mizzaro

Author:
Michael Soprano

Year 2023

Contents

List of Figures vi

List of Tables viii

List Of Equations ix

List Of Listings xii

List Of Algorithms xiii

1. Introduction 1

1.1 The Crowdsourcing Activity Workflow 1
1.2 Supporting Crowdsourcing-Based Approaches 2
1.3 Crowsourcing Platforms 4
1.3.1 Amazon Mechanical Turk 4
1.3.2 Toloka 7
1.3.3 Prolific 12
1.3.4 Discussion 13
1.4 Aims 16
1.5 System Design 16
1.5.1 General Architecture 17
1.5.2 Generator 17
1.5.2.1 Use Cases 17
1.5.2.2 Architecture 17
1.5.2.3 Case Study 19
1.5.3 Skeleton 22
1.5.3.1 Use Cases 22
1.5.3.2 Architecture 23
1.5.3.3 Wrapper 23
1.5.3.4 Data Format 25

1.5.3.5 Cost Estimation . . . 26
1.5.4 Search Engine . . . 33
1.5.4.1 Use Cases . . . 33
1.5.4.2 Architecture . . . 34
1.5.4.3 Microsoft Search API . . . 35
1.5.4.4 Entrez Programming Utilities . . . 37
1.5.4.5 fakeJSON . . . 40
1.5.4.6 Cost Estimation . . . 41
1.5.5 Logger . . . 43
1.5.5.1 Architecture . . . 43
1.5.5.2 Event Handling . . . 44
1.5.5.3 Performance Evaluation . . . 46
1.5.5.4 Pilot Experiment . . . 49
1.5.5.5 Cost Estimation . . . 52
1.5.5.6 Log Events . . . 54
1.6 System Usage . . . 59
1.6.1 Getting Started . . . 59
1.6.2 Environment Variables . . . 62
1.6.3 Build Output . . . 64
1.6.3.1 `build/task/` . . . 65
1.6.3.2 `build/environments/` . . . 65
1.6.3.3 `build/config/` . . . 65
1.6.3.4 `build/skeleton/` . . . 65
1.6.3.5 `build/deploy/` . . . 66
1.6.3.6 `build/mturk/` . . . 66
1.6.3.7 `build/toloka/` . . . 66
1.6.4 Task Configuration . . . 66
1.6.5 HITs Allocation . . . 67
1.6.5.1 Manual Approach . . . 67
1.6.5.2 Automatic Approach . . . 68
1.6.6 Quality Checks . . . 71
1.6.7 Local Development . . . 74
1.7 Task Performing . . . 76
1.7.1 Getting Started . . . 76
1.7.2 Manual Recruitment . . . 76
1.7.3 Amazon Mechanical Turk . . . 77
1.7.4 Toloka . . . 77
1.7.5 Prolific . . . 78
1.8 Task Results . . . 79
1.8.1 Getting Started . . . 79
1.8.2 `result/Task/` . . . 80
1.8.3 `result/Data/` . . . 80
1.8.4 `result/Resources/` . . . 80
1.8.5 `result/Crawling/` . . . 81
1.8.6 `result/Dataframe/` . . . 81

1.9 Conclusions 87
1.10 Future Work 88

Bibliography 91

Analytical Index 99

List of Figures

1.1 Type selection for a task on Amazon Mechanical Turk. 5
1.2 Parameters configuration for a task on Amazon Mechanical Turk. 6
1.3 Task type selection for a project on Toloka. 8
1.4 Interface design for a project on Toloka using its template builder. 9
1.5 Input and output data specification for a project on Toloka. 9
1.6 Worker attributes configuration for a pool of a project on Toloka. 12
1.7 General parameters set up for a study on Prolific. 13
1.8 Data collection set up for a study on Prolific. 14
1.9 Audience configuration for a study on Prolific. 15
1.10 Study cost configuration for a study on Prolific. 15
1.11 General architecture of Crowd_Frame. 18
1.12 Use case diagram for a requester that designs and deploy a task using Crowd_-Frame. 18
1.13 Worker interface of a misinformation assessment task deployed by Roitero et al. [58] using Crowd_Frame. 21
1.14 Use case diagram for a worker that performs a task deployed using Crowd_-Frame. 22
1.15 Crowd_Frame Skeleton execution flow. 23
1.16 Wrapper interface for to a task configured Crowd_Frame that relies on Amazon Mechanical Turk for recruiting workers. 24
1.17 Use case diagram for a worker that performs a task deployed using Crowd_-Frame. 33
1.18 Interface of the Search Engine component of Crowd_Frame. 34
1.19 Interface of the Search Engine component of Crowd_Frame. 35
1.20 Infrastructure of the Logger component of Crowd_Frame. 45
1.21 Requests sent to evaluate the performances of the infrastructure during each round of the test. 47
1.22 Requests managed per second by the dedicated server solution during each round of the test. 48
1.23 CPU usage of the dedicated server solution during each round of the test. . . . 48
1.24 Memory usage of the dedicated server solution during each round of the test. . 48
1.25 Event type distribution of the pilot test deployed to gather real data using the Logger component. 49

1.26 Worker behavior while evaluating a HIT's element reconstructed using the log data produced by the Logger component. 50
1.27 Worker behavior while answering a questionnaire reconstructed using the log data produced by the Logger component. 51
1.28 Deploy diagram of the infrastructure used to allow Crowd_Frame and the solver communicating. 71
1.29 Sample configuration for the solver used to automatically allocate elements to be evaluated in a set of HITs. 72

List of Tables

1.1 Summary of each configuration step of the Generator component. 20
1.2 Sample cost estimation parameters for the usage of the data table by the Skeleton component of Crowd_Frame. 30
1.3 Headers provided while sending requests to the Bing Web Search API. 36
1.4 Headers captured while receiving responses from the Bing Web Search API. . . 36
1.5 Query parameters provided when sending queries to the Bing Web Search API. 37
1.6 General description the of the nine E-utilities provided by the Entrez system. . . 38
1.7 Query parameters provided when sending requests to the ESearch utility of the Entrez system. 39
1.8 Payload parameters used to customize the behavior of the responses generated by the fakeJSON service. 41
1.9 Environment variables used to customize Crowd_Frame. 62
1.9 Environment variables used to customize (cont.) Crowd_Frame. 63
1.9 Environment variables used to customize (cont.) Crowd_Frame. 64
1.10 Folder structure of the output of a Crowd_Frame build. 64
1.11 Configuration files of a task deployed using Crowd_Frame. 65
1.12 Source files of a task deployed using Crowd_Frame. 66
1.13 Content of the build folder to deploy a task on Toloka. 67
1.14 Structure of the results folder of Crowd_Frame. 79
1.15 DataFrame produced when downloading the final results of a task. 84

List of Equations

1.1 Sample estimation of the storage cost for the usage of S3 by the Skeleton component of Crowd_Frame. 28
1.2 Sample estimation of the storage cost for the usage of S3 by the Skeleton component of Crowd_Frame. 28
1.3 Sample estimation of the data transfer cost for the usage of S3 by the Skeleton component of Crowd_Frame. 28
1.4 Cost estimation for the usage of Amazon S3 for the Skeleton component. 28
1.5 Sample estimation of the WRUs cost for the usage of the data table of the Skeleton component. 30
1.6 Sample estimation of the RRUs cost for the usage of the data table of the Skeleton component. 30
1.7 Sample estimation of the data storage cost for the usage of the data table by the Skeleton component of Crowd_Frame. 31
1.8 Sample overall estimation of the cost for the usage of the data table by the Skeleton component of Crowd_Frame. 31
1.9 Sample estimation of the WRUs cost for the usage of the access control list table of the Skeleton component. 31
1.10 Sample estimation of the RRUs cost for the usage of the access control list table of the Skeleton component. 32
1.11 Sample estimation of the data storage cost for the usage of the access control list table by the Skeleton component of Crowd_Frame. 32
1.12 Sample overall estimation of the cost for the usage of the data table by the Skeleton component of Crowd_Frame. 32
1.13 Sample overall estimation of the cost for the usage of the data table by the Skeleton component of Crowd_Frame. 33
1.14 Sample estimation of the cost for the usage of the Bing Web Search API. 43
1.15 Sample estimation of the API Gateway cost for the usage of the Logger component. 52
1.16 Sample estimation of Simple Queue cost for the usage of the Logger component. 52
1.17 Sample estimation of the Lambda cost for the usage of the Logger component. . 53
1.18 Sample estimation of the DynamoDB cost for the usage of the Logger component. 54
1.19 Sample estimation of the overall cost for the usage of the Logger component. . . 54

List of Listings

1.1 Interface of a sample task on Amazon Mechanical Turk built using Crowd Elements. 7
1.2 Image classification JSON configuration for the interface of a project on Toloka built using the template builder. 10
1.3 Input data specification for a project on Toloka. 11
1.4 Output data specification for a project on Toloka. 11
1.5 Input data initialization for two HITs within a pool of a project on Toloka. . . 11
1.6 Format for a DynamoDB table that contains the access control list of a task deployed using Crowd_Frame. 25
1.7 Format for a DynamoDB table that contains the data produced by the worker during a task deployed using Crowd_Frame. 26
1.8 JSON data stored for the element of a HIT evaluated by a worker during a task deployed using Crowd_Frame. 27
1.9 Sample request that contains headers and query parameters sent to the Bing Web Search API. 37
1.10 Sample initial request sent to the ESearch utility of the Entrez system. 40
1.11 Sample initial request sent to the ESummary utility of the Entrez system. . . . 40
1.12 Sample request sent to the fakeJSON service using cURL. 41
1.13 Payload of the request sent to the fakeJSON service to generate fake search results. 42
1.14 Payload of each log request sent by the Logger component of Crowd_Frame. . 46
1.15 Input data specification for a project on Toloka. 61
1.16 Sample `credentials.json` file to store IAM user access key. 61
1.17 Subset of environment variables required by Crowd_Frame. 61
1.18 Python packages required to initialize Crowd_Frame. 62
1.19 Valid set of one HIT for a task designed and deployed using Crowd_Frame. . 68
1.20 Fragment of elements to evaluated in the task published by Roitero et al. [57]. 72
1.21 Fragment of the allocation built automatically using the solver integrated with Crowd_Frame. 73
1.22 Valid set of one HIT with two elements, where one is used within a custom quality check. 74

1.23 Default implementation of the static method that performs custom quality checks in Crowd_Frame. 75
1.24 Sample development environment of Crowd_Frame. 76
1.25 Snapshot of a worker who participates in a task with a single batch deployed using Crowd_Frame. 80
1.26 Subset of the information obtained by perform the reverse lookup of the IP address of a worker. 82
1.27 Subset of the information obtained by analyzing the user agent string of a worker. 83
1.28 Metadata produced by the download script while trying to crawl a webpage retrieved by the search engine of Crowd_Frame. 85
1.29 Example of the `workers_acl` dataframe produced by Crowd_Frame. 86
1.30 Example of the `workers_answers` dataframe produced by Crowd_Frame. . . 87

List of Algorithms

1.1 Procedure to allocate a dataset into HITs using the format required 69
1.2 Procedure to sample elements without duplicates for a single HIT 70

Chapter 1

Introduction

This work is based on the article published at the fifteenth ACM International Conference on Web Search and Data Mining [17]. It describes in detail Crowd_Frame, a software system implemented to support the whole crowdsourcing activity workflow. Initially, Section 1.1 and Section 1.2 describe the relevant related work. Section 1.3 describes the design and deployment workflow of a crowdsourcing task on three different platforms. Then, Section 1.4 motivates the overall aim that the software system aims to satisfy. Section 1.5 analyzes in detail each one of the four components. Section 1.6 describes how to configure and deploy a crowdsourcing task. Section 1.7 describes how to recruit a crowd of workers to perform the task. Section 1.8 describes the structure of the final results, while Section 1.9 discusses the current state of the implementation. Finally, Section 1.10 sketches possible development directions.

1.1 The Crowdsourcing Activity Workflow

In recent years, crowdsourcing has become a popular method for collecting human work on a large scale. Typically, there are platforms that host the tasks to be performed. These tasks are then allocated to crowd workers in a first-come, first-served approach. Several crowdsourcing platforms such as Amazon Mechanical Turk, thus, emerged to support the ever-increasing need for crowd-powered data gathered using outsourced tasks. These platforms aim to help human task requesters to outsource their tasks to a diverse, distributed, and large workforce able to perform them. Indeed, although an increasing amount of published studies show the usage of Amazon Mechanical Turk [34], alternatives are also used [53]. Popular alternatives include Prolific, a platform dedicated to the scientific community where the crowd workers are explicitly recruited for participation in research tasks [51], or Toloka, a platform mainly focused on data labelling tasks.

A task requester is an individual who wants to deploy a crowdsourcing task on a chosen platform. The workflow involves several phases. Initially, the requester sets some parameters such as the number of crowd workers required, the time allowed for each worker to perform the task, and so on. Then, they designs the task layout. Usually, a markup lan-

guage is provided to help to build the user interface. The requester must write the logic to handle the task using client-side programming. Once the project is finalized the task can be deployed an arbitrary amount of times using a support file to vary the input data. Each instance of a task assigned to a worker is usually called HIT (Human Intelligence Task). Paolacci et al. [52] define a set of HITs as a batch. When a worker completes the HITs assigned, the requester can approve and thus pay the worker or reject the final submission. This workflow shows several difficulties across all platforms: the requester must have advanced programming skills; the user interface is built by mixing the presentation and business logic; the input data passing mechanism is often cumbersome, and the responsibility to store the data produced by each worker lies on the requester for non-trivial experiments.

Moreover, requesters sometimes need to run studies that require a specific worker to perform new chucks of work over multiple days, weeks, or months: namely longitudinal studies (LS). Longitudinal studies aim at observing changes that may occur with respect to a chosen subject over a given or extended period of time. Running longitudinal studies on crowdsourcing platforms has become popular, as also shown by Litman et al. [44], who present a tool for longitudinal study functions on top of Amazon Mechanical Turk, mainly because of the convenience and easy means that crowdsourcing platforms provide to access potential crowd workers. Despite the growing popularity of turning to the crowd as opposed to carrying out lab studies [26], little is currently understood about how workers perceive longitudinal studies.

1.2 Supporting Crowdsourcing-Based Approaches

Individuals and organizations who need to gather data of some kind using crowdsourcing-based approaches may rely on different crowdsourcing platforms. These platforms help task requesters access the global human workforce available. Amazon Mechanical Turk is one of the most well-known platforms. Paolacci et al. [52] presented, in the past, demographic data about the Mechanical Turk worker population, reviewing the strengths of Mechanical Turk relative to other online and offline methods of recruiting subjects, and comparing the magnitude of effects obtained using Mechanical Turk and traditional worker pools. More recently, Mellis et al. [48] reviews different research conducted using Mechanical Turk, provides examples and discusses the limitations and best practices of the platform.

In the last years, researchers are arguing that the quality of data collected by recruiting workers from Mechanical Turk is declining. According to Peer et al. [53], already in 2017, Mechanical Turk workers were becoming less naive. In more detail, they define workers as "professional survey-takers". Related to this, Chmielewski et al. [15] conducted before, during and after the summer of 2018 an experiment related to psychological research, finding empirical evidence of a substantial decrease in data quality. Kennedy et al. [35] show the presence of many fraudulent responders that provide low-quality data. Chandler et al. [12] explain that a substantial number of participants misrepresent relevant characteristics to meet the eligibility criteria expected in the studies. Webb et al. [68] explain that the data collected for their research were valid for the 2.6% of humans recruited and claim that a call for caution is needed while using Mechanical Turk. However, other crowdsourcing platforms seem to be a viable alternative. Peer et al. [54] compare 5 crowdsourcing platforms

and panels by examining aspects of data quality for online behavioral research. They conduct two studies and only the platform [51] provides high data quality on all measures for both studies, while CloudResearch (formerly known as TurkPrime [44]) only for the second study. Litman et al. [42] analyze the claims of Peer et al. [54] and point out the presence of methodological decisions undisclosed in their work that limit the inference that can be drawn from the data collected. In more detail, they assert that Peer et al. [54] chose to turn off the recommended data quality filters while using the CloudResearch platform. They thus replicate the studies with CloudResearch using the recommended options, finding that the final data are of better quality. Litman et al. [42] point out in their work that Peer et al. [54] are members of Prolific . However, Litman et al. [42] clarify that they are part of the CloudResearch team. While other researchers suggest that using Prolific leads to some extent to results of better quality [64], Litman et al. [43] advocates that the platform should be selected by matching the study's goals and the platform's strengths and weaknesses.

Several tools that aid requesters during the whole crowdsourcing activity exist. Vukovic [66] proposes a taxonomy for the categorization of crowdsourcing platforms and evaluates a set of systems with respect to such taxonomy. Erickson [23] proposes a conceptual framework for the design of systems to support crowdsourcing and human computation. Liu [45] proposes a set of best practices to develop crowdsourcing systems designed to support the articulation work needed to facilitate spontaneous volunteer effort during emergencies. Clark et al. [16] develops a framework to allow governments to use crowdsourcing bases approaches to solve problems when interacting with their citizens. Brito et al. [9] propose a conceptual framework to guide the design of gamification-based approaches within crowdsourcing platforms. Li et al. [39] conceptualize a blockchain-based decentralized framework in which a requester can propose a task without relying on any third trusted institution. Ye et al. [69] introduce a crowdsourcing framework to support the annotation of medical data sets. Hamrouni et al. [29] propose a framework for spatial mobile crowdsourcing, where workers are required to be physically present at a particular location. The requesters solicit workers to provide photos of ongoing events for event reporting purposes. CloudResearch [44] is a research platform that integrates with Amazon Mechanical Turk that aims to improve the quality of the crowdsourcing data collection process. CrowdForge [36] is a general-purpose framework for accomplishing complex human computation tasks. It is based on the idea that complex work can be broken up into small and independent pieces while the system manages its coordination dependencies. iCrowd [24] is an adaptive crowdsourcing framework that estimates in real-time the accuracy of workers by evaluating their performances on completed tasks. It can be used before the task's launch to choose the best workers available. CrowdTruth 2.0 [22] is a method to aggregate crowdsourcing responses after the task using disagreement-aware metrics. It can be used to leverage workers' data processing after the task. CrowdEIM [59] is a tool based on mobile social media platforms which allow crowdsourcing information during emergencies.

Researchers proposed in the past approaches to model and predict user behaviour when interacting with web applications. Historically, these approaches relied heavily on Markov models, which have been widely popular for this type of task. Ching et al. [14] study the usage of such models to analyze categorical data sequences. The research community later focused on different challenges. Borges et al. [6] describe user web navigation sessions up to a given length. Shukla et al. [60] model the behaviour of users that chooses to switch the

browser used to surf the web. Manavoglu et al. [47] aims to generate probabilistic browsing behaviour for users on the web. Benevenuto et al. [5] analyze user behaviour on social networks. Li et al. [40] integrate user behaviour into contextual advertising. Awad et al. [3], Deshpande et al. [18], and Dongshan et al. [20] aim predicting web page accesses. Anderson et al. [2] take advantage of user behaviour to personalize websites. Ju et al. [33] and Ren et al. [55] use behavioural data to build systems that can detect and prevent access to malicious users. More recent approaches involve the usage of behavioural data to represent user interaction using embeddings. Tran et al. [65], for instance, propose a recommendation model that learns user and item attributes represented using embeddings in the context of a recommender system. Learning vector representations for texts have been studied by the research community in depth. For example, Le et al. [38] and Mikolov et al. [49] analyze the usage of sentences as a better way to learn word semantics. Embedding can be helpful also when considering information retrieval tasks such as query rewriting. Grbovic et al. [28] propose a query rewriting method based on a query embedding algorithm. Other approaches include content advertising, as the one by Grbovic et al. [27]. They propose a neural language-based algorithm specifically tailored for delivering effective product recommendations. The embedding representations learnt can be used to predict and understand user behaviour across different scenarios, as done by Chen et al. [13] and Han et al. [30]. Crowdsourcing-based approaches can provide a massive amount of behavioural data due to the availability of a large human workforce. Robinson et al. [56] find out that there are more than 250.000 workers around the whole world whose potential is largely untapped. Stewart et al. [63] explains that a task requester can reach more than 7.000 workers each quarter year. Difallah et al. [19] analyze the population dynamics and demographics of Amazon Mechanical Turk workers based on the results of a survey that they conducted over 28 months. They also discover that during the day the peak of active US workers is 90% around 11 PM UTC. Han et al. [31] perform a data-driven analysis of logs collected during a large-scale relevance judgment experiment to study the phenomenon of crowdsourcing task abandonment.

1.3 Crowsourcing Platforms

Understanding in more detail how the crowdsourcing platforms support the task design and deploy workflow, which may be challenging, is useful to grasp the improvements provided by the usage of Crowd_Frame. Three crowdsourcing platforms are described in the following, namely Amazon Mechanical Turk (Section 1.3.1), Toloka (Section 1.3.2) and Prolific (Section 1.3.3).

1.3.1 Amazon Mechanical Turk

The task design workflow for a requester who uses Amazon Mechanical involves three phases. Initially, the requester must choose the type of task to publish (Figure 1.1). The platform provides several templates ready to be customized. The requester can also choose using a blank template.

Then, the requester starts designing the overall task. Initially, they inputs name and de-

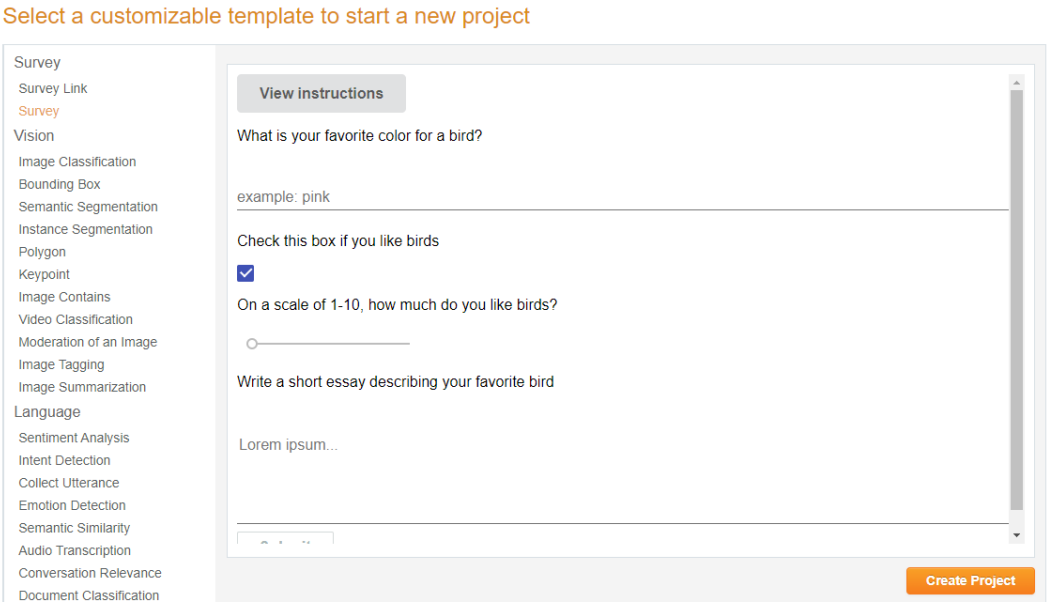

Figure 1.1: Type selection for a task on Amazon Mechanical Turk.

scription and sets five configuration parameters (Figure 1.2). The parameters are the number of workers to recruit, the time allowed to perform the task, the amount of the reward in USD$, the expiration date of the task, and the auto-approve threshold in days for the workers' submissions. The requester can also set various criteria to filter the workers to recruit. Amazon Mechanical Turk uses the word "Qualification Type" to indicate such criterion. The qualification types include the country of provenance of the worker, the age or household income, etc. It is also possible to define custom qualification types.[1] The requester designs the interface of the task after setting the parameters and criteria. They must use a set of custom markup tags defined as Crowd Elements.[2] Each task must contain the tag `<crowd-form>`, which is thus the most important element of the codebase. Listing 1.1 shows the code of a sample task designed using the Crowd Elements.

The requester can preview each HIT of the task when the design is finalized. The task can thus be published an arbitrary amount of times. Amazon Mechanical Turk uses the word "batch" to indicate a single set of workers recruited within a given crowdsourcing task. The requester can recruit multiple batches of workers, at the same time. Recruiting a batch of workers involves providing a special CSV file to set the input and output data. The platform considers each column of such a CSV file as data. Each row of the file is assigned to a single worker. The values of the row assigned are used to initialize each input or output. The requester must thus provide a file with n rows to recruit n workers. Each worker accepts the HIT initialized using such a mechanism and completes the task. The requester approves or denies the payment. The final data are provided by the platform through a second CSV

[1] `https://docs.aws.amazon.com/AWSMechTurk/latest/AWSMechanicalTurkRequester/WorkWithCustomQualType.html`

[2] `https://docs.aws.amazon.com/AWSMechTurk/latest/AWSMturkAPI/ApiReference_HTMLCustomElementsArticle.html`

Edit Project

1 Enter Properties | 2 Design Layout | 3 Preview and Finish

Project Name: Survey This name is not displayed to Workers.

Describe your survey to Workers

Title: Answer a survey about your opinions

Describe the survey to Workers. Be as specific as possible, e.g. "answer a survey about movies", instead of "short survey", so Workers know what to expect.

Description: Give us your opinion about our products

Give more detail about this survey. This gives Workers a bit more information before they decide to view your survey.

Keywords: survey

Provide keywords that will help Workers search for your tasks.

Setting up your survey

Reward per response: $

This is how much a Worker will be paid for completing your survey. Consider how long it will take a Worker to complete your survey.

Number of respondents: 3

How many unique Workers do you want to complete your survey?

Time allotted per Worker: 1 Hours

Maximum time a Worker has to complete the survey. Be generous so that Workers are not rushed.

Survey expires in: 7 Days

Maximum time your survey will be available to Workers on Mechanical Turk.

Auto-approve and pay Workers in: 3 Days

This is the amount of time you have to reject a Worker's assignment after they submit the assignment.

Figure 1.2: Parameters configuration for a task on Amazon Mechanical Turk.

file when each worker of the batch completed the HIT.

```
<script src="https://assets.crowd.aws/crowd-html-elements.js"></script>
<crowd-form answer-format="flatten-objects">
  <crowd-instructions link-text="View instructions" link-type="button">
    <short-summary>
      <p>Provide a brief instruction here</p>
    </short-summary>
    <detailed-instructions>
      <h3>Provide more detailed instructions here</h3>
      <p>Include additional information</p>
    </detailed-instructions>
    <positive-example>
      <p>Provide an example of a good answer here</p>
      <p>Explain why it's a good answer</p>
    </positive-example>
    <negative-example>
      <p>Provide an example of a bad answer here</p>
      <p>Explain why it's a bad answer</p>
    </negative-example>
  </crowd-instructions>
  <div>
    <p>What is your favorite color for a bird?</p>
    <crowd-input name="favoriteColor" placeholder="example: pink"
    ↪ required>
    </crowd-input>
  </div>
  <div>
    <p>Check this box if you like birds</p>
    <crowd-checkbox name="likeBirds" checked="true" required>
    </crowd-checkbox>
  </div>
  <div>
    <p>On a scale of 1-10, how much do you like birds?</p>
    <crowd-slider name="howMuch" min="1" max="10" step="1" required>
    </crowd-slider>
  </div>
</crowd-form>
```

Listing 1.1: Interface of a sample task on Amazon Mechanical Turk built using Crowd Elements.

1.3.2 Toloka

The task design workflow for a requester who uses Toloka involves three phases. Initially, the requester chooses the type of task to be performed (Figure 1.3) and inputs name

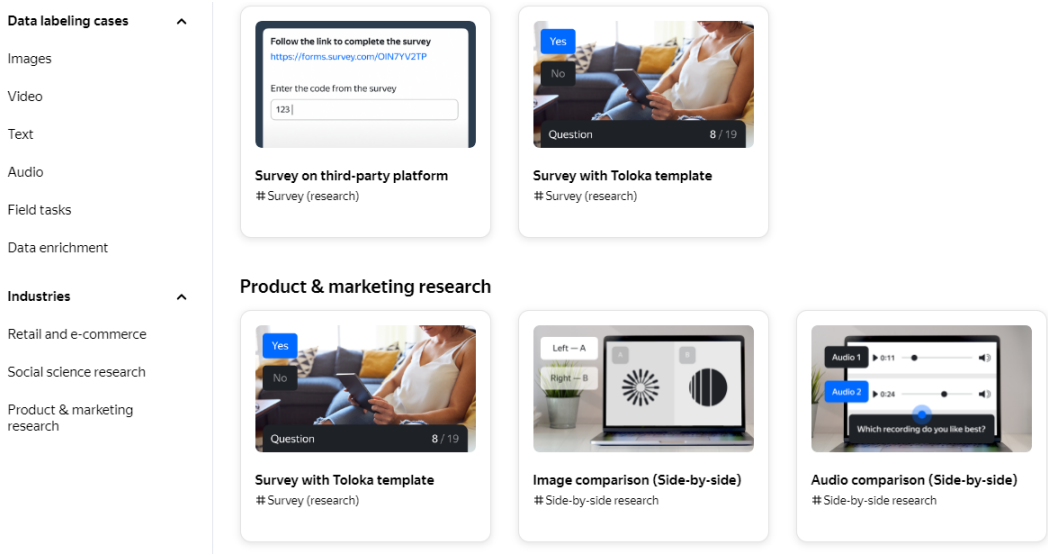

Figure 1.3: Task type selection for a project on Toloka .

and description, together with a private comment that can help, for instance, distinguishing between two projects having the same name. Toloka, indeed, uses the word "Project" to indicate a crowdsourcing task.

Then, the requester designs the task interface. They can either use standard HTML markup tags together with JavaScript code and CSS styling or Toloka 's template builder[3] (Figure 1.4). The template builder uses a set of predefined JSON objects used to initialize the overall interface. The input and output data specification[4] for the task must be declared in a separate section of the user interface (Figure 1.5). The data can be specified either manually using special JSON objects or using the user interface itself. The input and output data specified are then referenced while building the task interface. Listing 1.2 show the JSON specification of the sample image classification task interface shown in Figure 1.4. Listing 1.3 and Listing 1.4 show the JSON specification of the corresponding input and output data. The requester then writes the instructions that the workers will read when accepting a HIT. Toloka uses either the words "Toloker" or "User" to identify workers.

The requester can define multiple pools of workers to recruit for a given task when its design is finalized. Toloka uses the word "Pool" to indicate groups of workers who share a predefined set of attributes. In more detail, the requester specifies for each pool its name and description along with the set of attributes. These attributes may include the language spoken, the world region of provenance, the operating system used, and many others. Then, the requester specifies a speed/quality balance percentage. In other words, the percentage of top-rated workers who can access the pool. Requiring more quality workers means a slower pool completion time. The requester can also specify a set of quality control mechanisms and rules. They can also require overlap for each HIT published within the pool. Lastly, the requester sets the reward for completing each HIT. Figure 1.6 shows part of the

[3] `https://toloka.ai/docs/template-builder/index.html`
[4] `https://toloka.ai/docs/guide/concepts/incoming.html`

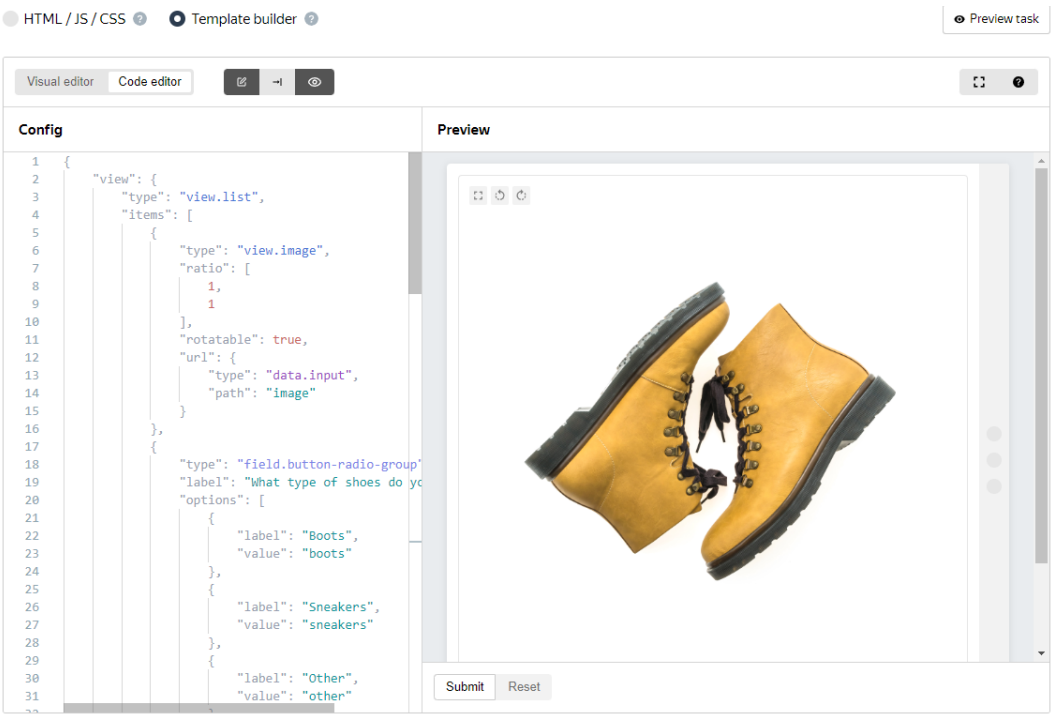

Figure 1.4: Interface design for a project on Toloka using its template builder.

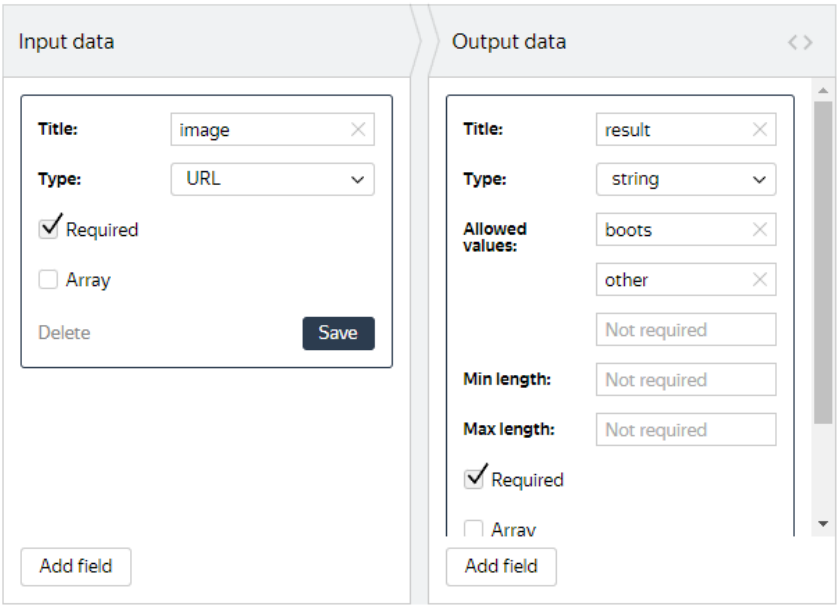

Figure 1.5: Input and output data specification for a project on Toloka.

Toloka's pool configuration interface. Each pool can be started, paused and stopped independently. Multiple pools can be active at the same time.

```
{
  "view": {
    "type": "view.list",
      "items": [
          {
            "type": "view.image",
            "ratio": [1,1],
            "rotatable": true,
            "url": { "type": "data.input", "path": "image" }
          },
          {
            "type": "field.button-radio-group",
            "label": "What type of shoes do you see?",
            "options": [
              { "label": "Boots", "value": "boots"},
              ...
            ],
            "data": { "type": "data.output", "path": "result"},
            "validation": { "type": "condition.required" }
          }
        ]
    },
  "plugins": [
     {
       "type": "plugin.toloka",
       "layout": { "kind": "scroll", "taskWidth": 600 }
     },
     {
       "1": {
         "type": "action.set",
         "data": { "type": "data.output", "path": "result" },
         "payload": "boots"
        },
        "2": {
            ...
        },
        "type": "plugin.hotkeys"
     }
  ], "vars": {}
}
```

Listing 1.2: Image classification JSON configuration for the interface of a project on Toloka built using the template builder.

The requester must provide values for the input and output data defined during the

```
{
  "image": {
    "type": "url",
    "required": true
  }
}
```

Listing 1.3: Input data specification for a project on Toloka.

```
{
  "result": {
    "type": "string",
    "allowed_values": [ "boots", "other" ],
    "required": true
  }
}
```

Listing 1.4: Output data specification for a project on Toloka.

design phase for at least one pool to start publishing the task designed on Toloka and thus recruiting workers. The mechanism is similar to the one of Amazon Mechanical Turk, described in Section 1.3.1. A special file containing the values for the input and output data must be provided. The file can comply with the XLSX, TSV or JSON formats. Each column (or attribute, if the file is in JSON format) is labelled with the prefixes `INPUT:` or `OUTPUT:` and then matched with the corresponding input or output data defined by the requester. Each row of the file is assigned to a worker and its values are used to initialize each data. In other words, the requester must provide a file with n rows to recruit n workers. Listing 1.5 show the content of a sample XLSX file used to recruit two workers within a pool for a task having a single input data named `image`. Each worker accepts the HIT assigned and completes the task. The requester can either approve or deny the reward for each HIT submitted. The final data are provided by Toloka through a second TSV file at the end of each pool of tasks.

```
INPUT:image
https://labs-images-testing.s3.yandex.net/presets/for%20tb%20and%20datas
↪ et/leather-boots.jpg
https://labs-images-testing.s3.yandex.net/presets/for%20tb%20and%20datas
↪ et/pair-trainers.jpg
```

Listing 1.5: Input data initialization for two HITs within a pool of a project on Toloka.

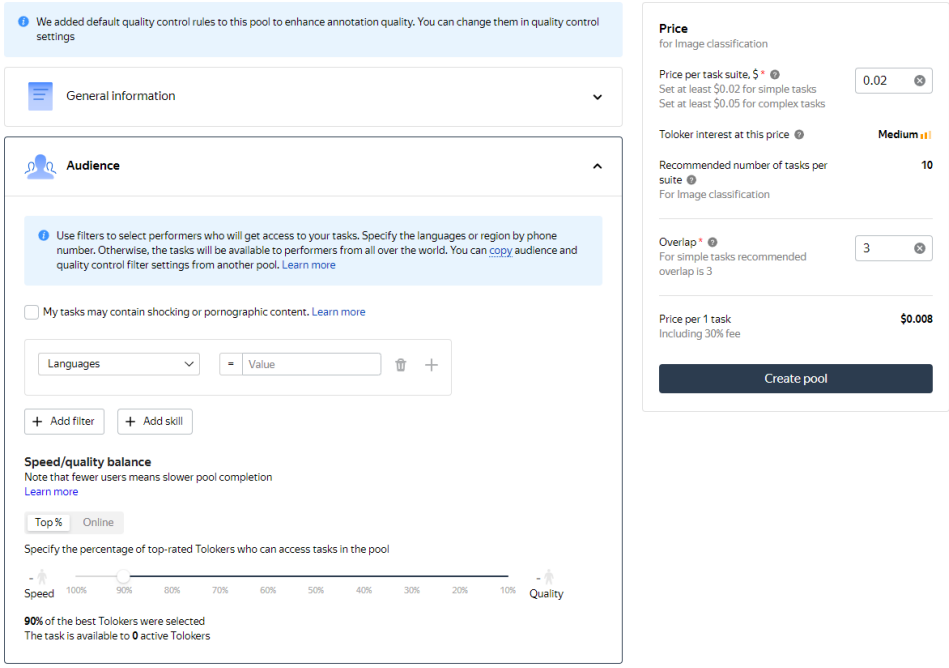

Figure 1.6: Worker attributes configuration for a pool of a project on Toloka.

1.3.3 Prolific

The task deployment workflow for a requester that uses the Prolific platform involves four phases. Initially, the requester sets five parameters concerning the task (Figure 1.7). Prolific uses the word "Study" to indicate a task, while the word "Participant" is used to indicate a worker. The preliminary parameters are the study name and description and the devices that crowd workers can use for participating in the task. The requester can also indicate whether the study needs to use the workers' microphone, camera, or audio and if the workers must download additional software to perform the task.

The second phase of the task deployment workflow on Prolific requires the requester to specify the data collection modality (Figure 1.8). The most evident difference when comparing Prolific with Amazon Mechanical Turk (Section 1.3.1) and Toloka (Section 1.3.2) is that Prolific does not offer any modality to design the task in-house. The task requester must rely on external tools to design and deploy the task and provide its URL to Prolific. There are two approaches to provide an anonymous identifier to the task deployed externally to detect each worker. The former consists of simply asking workers explicitly for their Prolific identifiers. The latter involves sending automatically the identifier of each worker by appending it to the external URL provided. The requester can send along also identifiers for the current task and session using the latter modality. Prolific uses the word "Session" to indicate the current batch of workers recruited. Furthermore, the requester can choose between two approaches to allow workers to confirm task completion from the software deployed externally. The former consists of embedding an URL in the software's user interface (e.g., in a button) to redirect the worker to Prolific. The latter consists of providing a

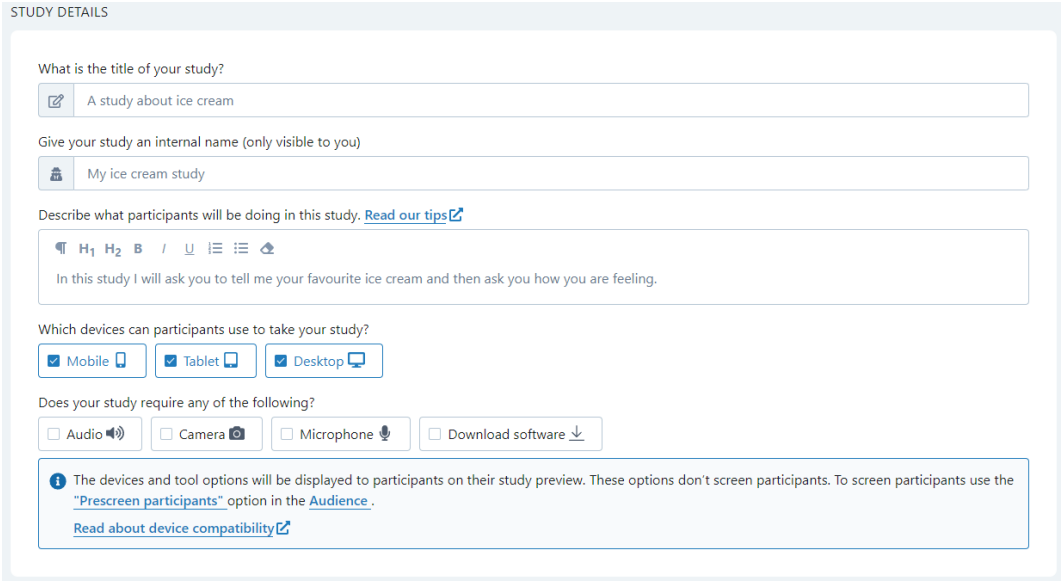

Figure 1.7: General parameters set up for a study on Prolific.

completion code that the worker copies and pastes manually on Prolific's interface.

The third phase involves configuring the audience required for the task. Prolific uses the word "audience"to indicate the workers to recruit for the current task. The requester can indicate how many workers must be recruited and from which country. The requester can choose between three sampling modalities to recruit the workers. Furthermore, various criteria can be used to further filter the audience of workers. The fourth phase of the task deployment workflow on Prolific requires the requester to specify the time needed to complete the task and the final reward in pounds. The platform thus computes the hourly pay rate and acknowledges the requester whether the amount is sufficient or not. The platform does not prevent the requester from publishing the task if the amount is insufficient. However, the platform could stop the task from continuing if the true median completion time exceeds the completion time estimated and suggest the requester provide additional payments. The requester can publish the task and wait for completion once its deploy is finalized. The status of each HIT can be monitored in real-time and the payment can be approved or denied.

1.3.4 Discussion

The design and deployment workflow of a crowdsourcing task is often cumbersome and counter-intuitive, as described in Section 1.1. Let us consider Amazon Mechanical Turk. The task interface code must be written in a single box and is a combination of a custom subset of HTML tags and CSS and JavaScript statements. The business and presentation logic are mixed. Hidden form fields with JSON objects are used as values to store data. They must build a file for each batch of the task. La Barbera et al. [37] deploy a misinformation assessment task directly on such a platform. They crowdsource judgments for 120 political-

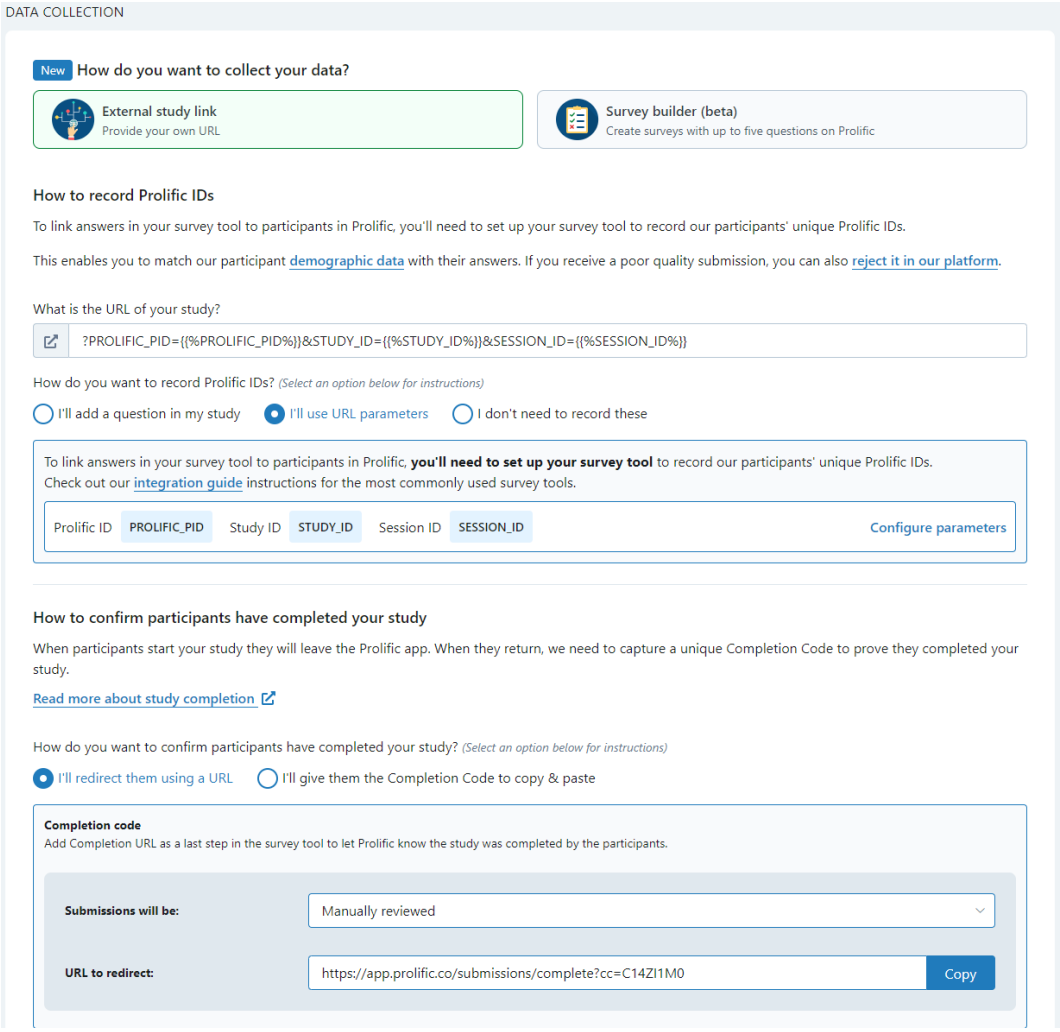

Figure 1.8: Data collection set up for a study on Prolific.

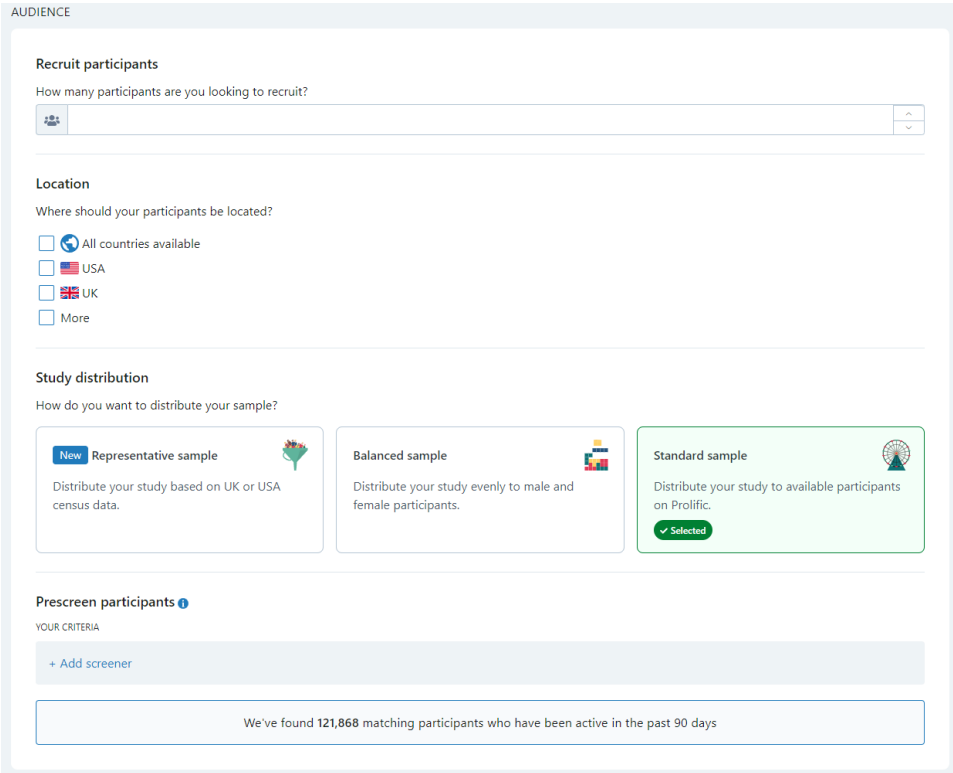

Figure 1.9: Audience configuration for a study on Prolific.

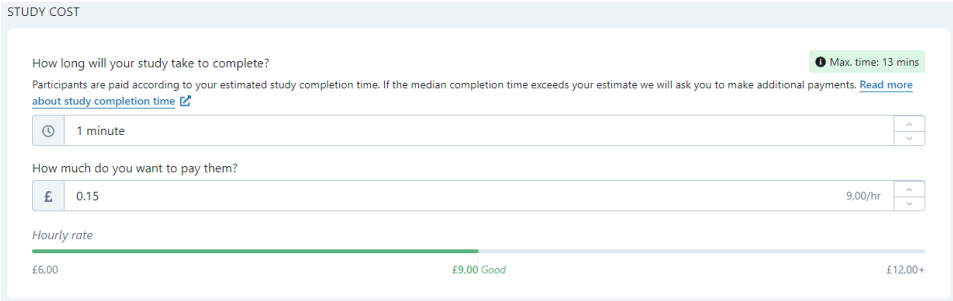

Figure 1.10: Study cost configuration for a study on Prolific.

related statements judged by 10 distinct crowd workers. They deploy 400 different HITs. The task is created using a paginated structure using the custom markup tags provided by Amazon Mechanical Turk. They had to implement custom Javascript code to show/hide elements of the user interface since there must be only a single `<crowd-form>`.

Toloka is affected by most of the difficulties that a requester faces when using Amazon Mechanical Turk. The platform further complicates the overall picture. The quality control rules and criteria used to initialize pools are non-trivial to set up. The allocation mechanisms of the HITs can be confusing because the platform groups them in overlapping task suites. On the other hand, the pool-based mechanism to recruit workers allows more fine-grained control over the workers to be recruited. Prolific is the platform that offers the most streamlined and easy workflow, but the absence of any way to deploy a task without relying on external tools can be a daunting challenge for many task requesters.

1.4 Aims

Several tools exist to help task requesters while employing crowdsourcing-based approaches (Section 1.2). However, none of these tools allows for coping with the difficulties of the workflow detailed in Section 1.3. A solution to these problems is to rely on crowdsourcing platforms only to recruit the workforce needed. The recruited workers reach the task deployed using external software in an external platform to perform it. Then, they return to the crowdsourcing platform of provenance to receive the reward. The task design and deployment processes can be handled by external software.

"Crowd_Frame: A Simple and Complete Framework to Deploy Complex Crowdsourcing Tasks Off-the-Shelf" [17] developed Crowd_Frame. It is a software system that allows to easily design and deploy diverse types of crowdsourcing tasks regardless of the chosen platform. The tasks can be composed of different sets of HITs. Each task is deployed in a customizable and controllable environment. The software is freely available and downloadable[5] by the research community and it has already been used by researchers and practitioners to deploy several tasks [7, 8, 10, 21, 57, 58, 61, 62].

1.5 System Design

Initially, Section 1.5.1 outlines the general architecture of Crowd_Frame. Then, Section 1.5.2 describes the Generator component, used by the requester to configure tasks. Section 1.5.2 describes the Skeleton component, with which the workers interact. Optionally, the skeleton may embed an instance of the Search Engine component described in Section 1.5.4, which is used by workers to retrieve some kind of evidence within the task. The Logger component, described in Section 1.5.5, can be used to capture workers' behavior during the whole task.

[5] `https://github.com/Miccighel/Crowd_Frame`

1.5.1 General Architecture

Crowd_Frame is a client-side application developed using Angular,[6] an open source framework for web development. Figure 1.11 shows its architecture. The software is composed of four main components namely Generator, Skeleton, Search Engine, and Logger. It relies on Amazon S3[7] to store tasks configuration and source files. It is an object storage service built to store and retrieve any type and amount of data. The software also uses Amazon DynamoDB[8] to store the data produced by each worker while performing the task. It is a fully managed NoSQL key-value and document database service. It is serverless and can scale autonomously according to the workload.

A requester uses the Generator to instantiate and configure tasks using a simple user interface. The configuration is uploaded to a private S3 bucket (i.e., a storage resource). Then, the requester can publish a set of HITs on the chosen crowdsourcing platform. Each crowd worker interacts with A wrapper used by the requester as a bridge between the platform and the workers themselves. The wrapper redirects the worker recruited to the application deployed on a public S3 bucket. The worker interacts with the Skeleton to perform the task. The Skeleton may embed a custom and configurable Search Engine. The Logger analyses the worker's behavior during the task. The data produced is stored on a DynamoDB table. Each component uses also external services to function.

1.5.2 Generator

The Generator component allows requesters to design and customize the configuration of a crowdsourcing task, as shown by Figure 1.11.

1.5.2.1 Use Cases

The diagram shown in Figure 1.12 provides a high-level description of the interaction between a requester and the Generator component of Crowd_Frame. The requester authenticates him/herself to unlock the usage of the component. Then, they designs or customizes the crowdsourcing task. The component provides an option to clone the configuration of a task previously deployed. The configuration phase involves seven different steps, described in detail in Section 1.5.2.2. The overall configuration is automatically generated and updated during each step. The requester finalizes the upload of the configuration when satisfied. They may need to review errors that invalidate the configuration to be able to upload it.

1.5.2.2 Architecture

The implementation of the Generator component is a form composed of eight separate steps. A requester configures a certain aspect of the task within each step. Table 1.1 provides a brief description of the role of each configuration step. The first step concerns questionnaires. It allows the creation of one or more questionnaires that workers will fill out before

[6] https://angular.io/
[7] https://aws.amazon.com/it/s3/
[8] https://aws.amazon.com/it/dynamodb/

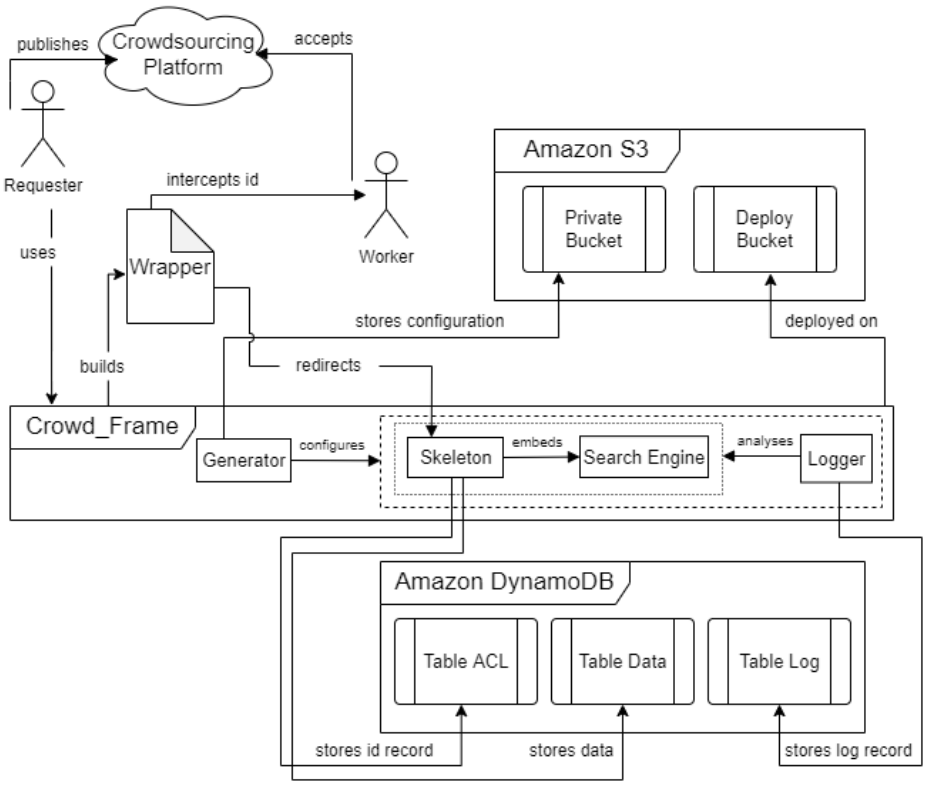

Figure 1.11: General architecture of Crowd_Frame.

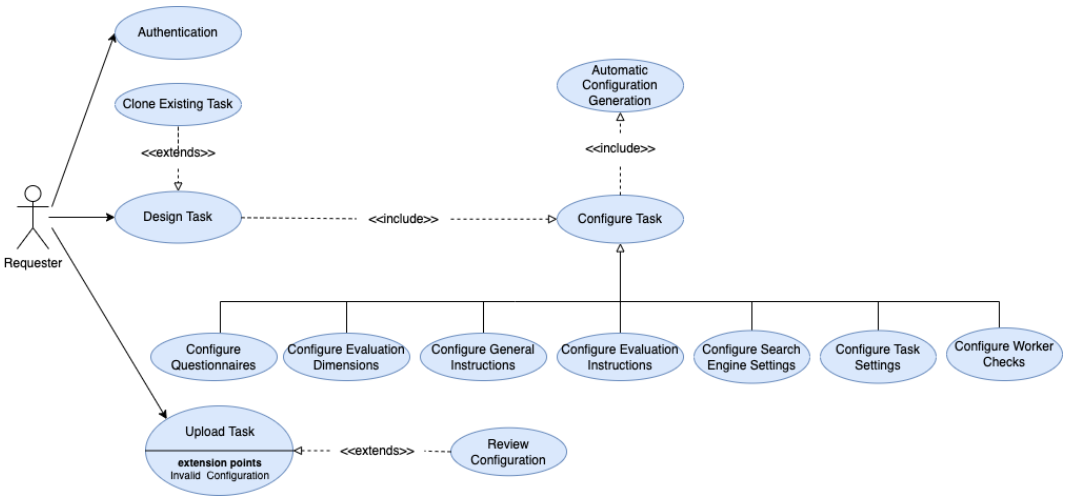

Figure 1.12: Use case diagram for a requester that designs and deploy a task using Crowd_-Frame.

or after task execution, depending on the requester's needs. Three different types of questionnaires are available. The `Standard` option allows implementing simple questionnaires where the worker responds to questions that may require either a predefined and limited number of alternative answers, a text-based answer or a numerical value. The `CRT` option allows implementing Cognitive Reflection Tests [25], often used to estimate workers' cognitive abilities. The `Likert` option allows implementing questionnaires where workers choose from a range of possible responses to a specific question [41]. It is possible to easily extend the Generator to add additional types of questionnaires.

The second step concerns the evaluation dimensions. It allows configuring, in other words, the aspects the worker evaluates for each element of the HIT assigned. A dimension may require expressing a judgment using some sort of scale. Every possible type of rating scale [1] is available in Crowd_Frame. A requester can thus configure categorical, interval, and ratio scales such as magnitude estimation [50]. Other features can be activated within a given evaluation dimension. For instance, the requester may require the worker to provide a textual justification or to search for an URL using the custom search engine integrated. The third step concerns task instructions. The requester writes the set of instructions using a rich textual editor. The worker is shown such instructions before the task. Similarly, the fourth step concerns evaluation instructions. The worker is shown with such instructions within the task's body while evaluating each element of the HIT assigned. Such instructions explain how they should assess each element along each dimension. The fifth step concerns the Search Engine. It allows choosing the search provider wanted and adding a list of domains to filter from search results.

The sixth step concerns general task settings. The requester can set the maximum amount of tries for each worker. This allows workers to revise their work if they do not pass the quality checks at the end of the task and helps in reducing task abandonment [31, 32]. The requester can also configure the minimum time that each worker must spend on each HIT element. Such a check is useful to prevent automated bots from completing a task. The requester can set a countdown to limit the time available for the worker to complete the assessment. Maddalena et al. [46] show that such a constraint leads to an improvement in worker quality. It also allows for optimizing the cost of a crowdsourcing task. The requester can enable an annotation interface to require the worker to annotate and label texts, such as for social media conversation annotation [70]. The requester may choose previous batches of workers to block for the current task. For example, this can be useful when performing a longitudinal study, where the same task is repeated multiple times with new workers only [58]. Lastly, the requester is required to upload the HITs for the task. The seventh step concerns additional checks on workers. The requester can manually specify a set of worker identifiers allowed or blocked for the current task. It allows finetuning the admitted workers without blocking an entire batch. The eighth and last step shows a summary of the configuration created. The requester can upload the configuration to the private S3 bucket if satisfied.

1.5.2.3 Case Study

Roitero et al. [58] use Crowd_Frame to deploy a misinformation assessment task to understand whether the crowd workers can identify and correctly classify online misinfor-

Table 1.1: Summary of each configuration step of the Generator component.

Step #	Name	Description
1	Questionnaires	Allows creating one or more questionnaires that workers will fill before or after task execution.
2	Evaluation Dimensions	Allows configuring what the worker will assess for each element of the HIT assigned.
2	Evaluation Dimensions	Allows configuring what the worker will assess for each element of the HIT assigned.
3	Task Instructions	Allows configuring the instructions shown to each worker before starting the task.
4	Evaluation Instructions	Allows configuring the instructions shown to each worker while assessing each element of the HIT assigned.
5	Search Engine	Allows configuring the search provider wanted. Furthermore, it is possible to add a list of domains to filter from search results.
6	Task Settings	Allows configuring the overall task settings, such as the maximum amount of tries for each worker, the usage of an annotation interface, and more.
7	Worker Checks	Allows configuring additional filters and checks on the workers recruited.
8	Summary	Allows reviewing and uploading the final configuration.

mation concerning the COVID-19 pandemic. Their task uses two questionnaires to collect workers' backgrounds and to estimate workers' cognitive abilities. Each worker recruited is shown a set of statements. A six-level assessment scale is used to provide truthfulness labels. Each worker must provide a URL using a custom search engine and a textual justification to support the rationale behind each assessment.

Figure 1.13 shows the resulting interface. A requester can easily replicate Roitero et al. [58] setup using Crowd_Frame. They use a single standard questionnaire and three CRT questionnaires. They set a single evaluation dimension which is the overall truthfulness. The dimension uses a six-level categorical scale and they implement quality control on the values provided. In more detail, they require evaluating a statement that is obviously true with a higher truthfulness value than a statement that is obviously false. They also filter out search results originating from three fact-checking-related websites. A sample configuration that can be used to replicate the task of Roitero et al. [58] is available on the repository of Crowd_Frame.[9]

[9] https://github.com/Miccighel/Crowd_Frame/tree/master/examples/misinformation_assessment

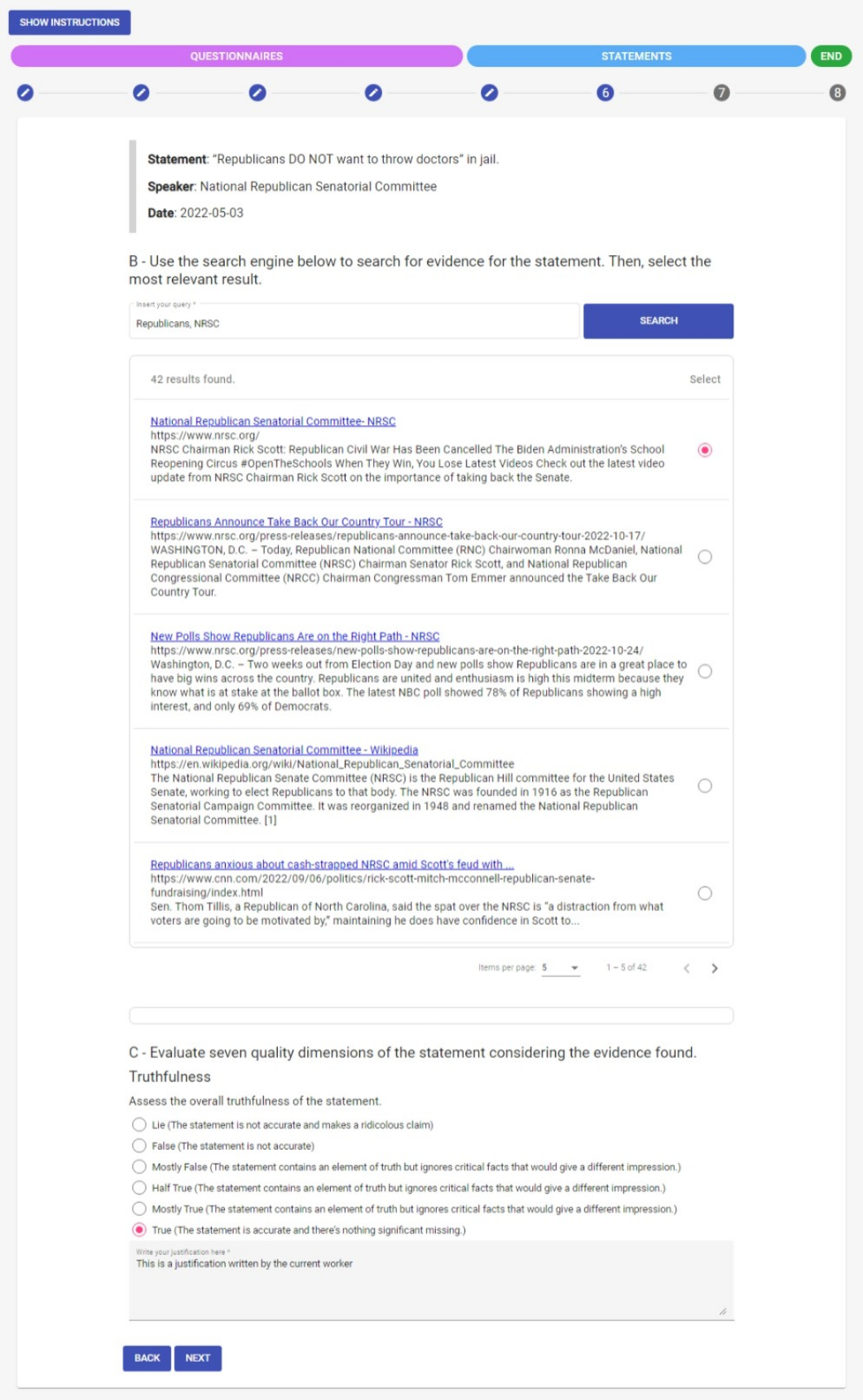

Figure 1.13: Worker interface of a misinformation assessment task deployed by Roitero et al. [58] using Crowd_Frame.

1.5.3 Skeleton

The Skeleton component allows workers to perform the task after being recruited, as shown by Figure 1.11.

1.5.3.1 Use Cases

The diagram shown in Figure 1.14 provides a high-level description of the interaction between a worker and the Skeleton component of Crowd_Frame. The worker accesses the task deployed after being recruited. They are initially forced to read the general instructions of the task before starting to work on the HIT assigned. Then, they finally start working. The task may require filling up one or more questionnaires, at the start or the end. Three types of questionnaires can be configured, as described in Section 1.5.2.2. The workers thus evaluate every dimension concerning every element of the HIT. A dimension may require providing a judgment using a rating scale, a URL using the search engine or writing a justification. Every type of rating scale is implemented as described, again, in Section 1.5.2.2.

Data is uploaded throughout the whole task. Quality checks are performed on user behavior during the performing of the HIT assigned or on the values provided for each dimension. The task may thus terminate allowing the worker to write a final comment to the requester. The worker can perform another try if the quality checks fail due to some reason.

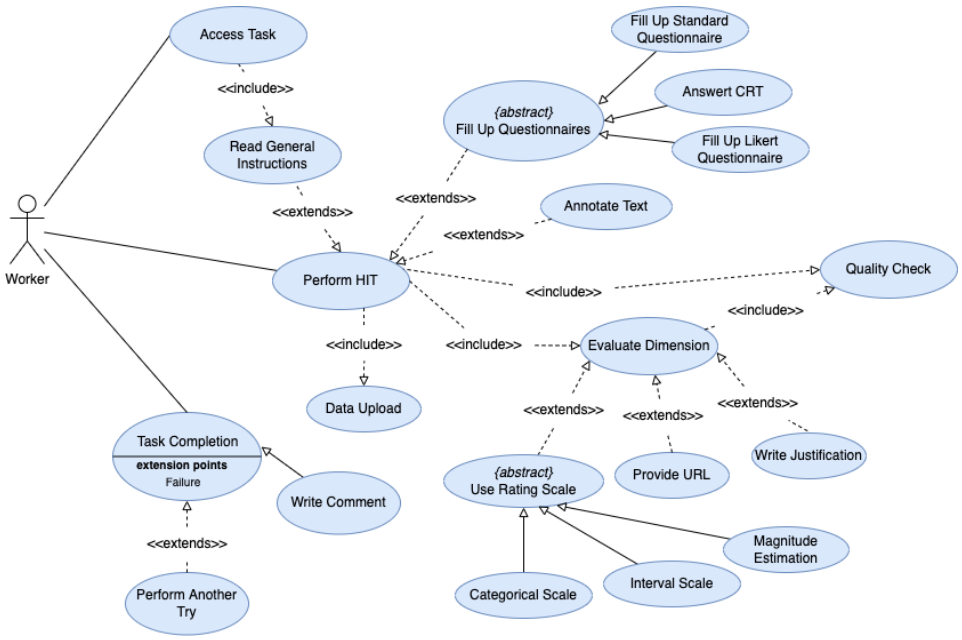

Figure 1.14: Use case diagram for a worker that performs a task deployed using Crowd_-Frame.

1.5.3.2 Architecture

Figure 1.15 details the interaction between a crowd worker and the Skeleton component. The worker reaches the application deployed on the public bucket through the wrapper. The identifier is sent as a URL parameter. When the worker reaches the current deployment the identifier is stored in an access control list (i.e., the DynamoDB table shown in Figure 1.11). This allows matching the worker with the data produced while performing the task. Furthermore, it allows tracking of how much time the worker still has to complete the work assigned. The requester can prevent the workers from accessing multiple times to the task deployed if needed.

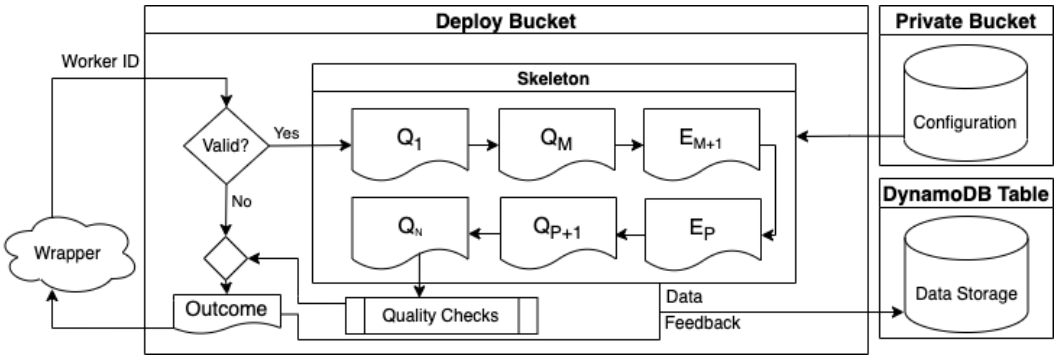

Figure 1.15: Crowd_Frame Skeleton execution flow.

The worker is initially shown the general instructions. The task unlocks after the initial check on the identifier. The Skeleton fetches the task configuration from the private bucket and instantiates the layout required. Then, the worker performs the task configured by the requester. The Skeleton creates a page for each of the $[0, M]$ questionnaires shown at the task's start. Then, the Skeleton initializes a page for each of the $[M + 1, P]$ elements to be evaluated. Finally, the Skeleton initializes a page for each of the $[P + 1, N]$ questionnaires shown at the task's end. The appearance of each page depends on the type of questionnaire of the dimension's parameters. The evaluation instructions are shown on every page. The worker can go back and forth on each element and perform the work required. The quality checks trigger after the last element or questionnaire. The outcome page is shown to the worker if the quality checks are satisfied. The worker may provide the final comment on the same page. Finally, the worker can reach the crowdsourcing platform to receive the reward. During the whole execution flow, the Skeleton stores the data produced in the DynamoDB table.

1.5.3.3 Wrapper

Figure 1.11 shows the presence of a wrapper between the workers recruited on a crowdsourcing platform and a task deployed using Crowd_Frame. The mechanism is used to identify a worker and assign a HIT to him/her. It relies on the possibility to provide input variables to the platform. The requester thus provides the crowdsourcing platform with a file containing input and output tokens. The input token is an alphanumeric string used to

match the worker with a certain HIT. The output token is used by the worker to confirm the successful completion of the HIT. The wrapper fetches the platform-specific identifier of the worker. Figure 1.16 shows the wrapper's interface used to redirect a worker recruited on Amazon Mechanical Turk. The wrapper's interface for a task deployed on Toloka is similar to the one shown by Figure 1.16. On the other hand, Prolific does not require this at all the wrapper, since the requester must provide a direct URL to the task deployed externally.

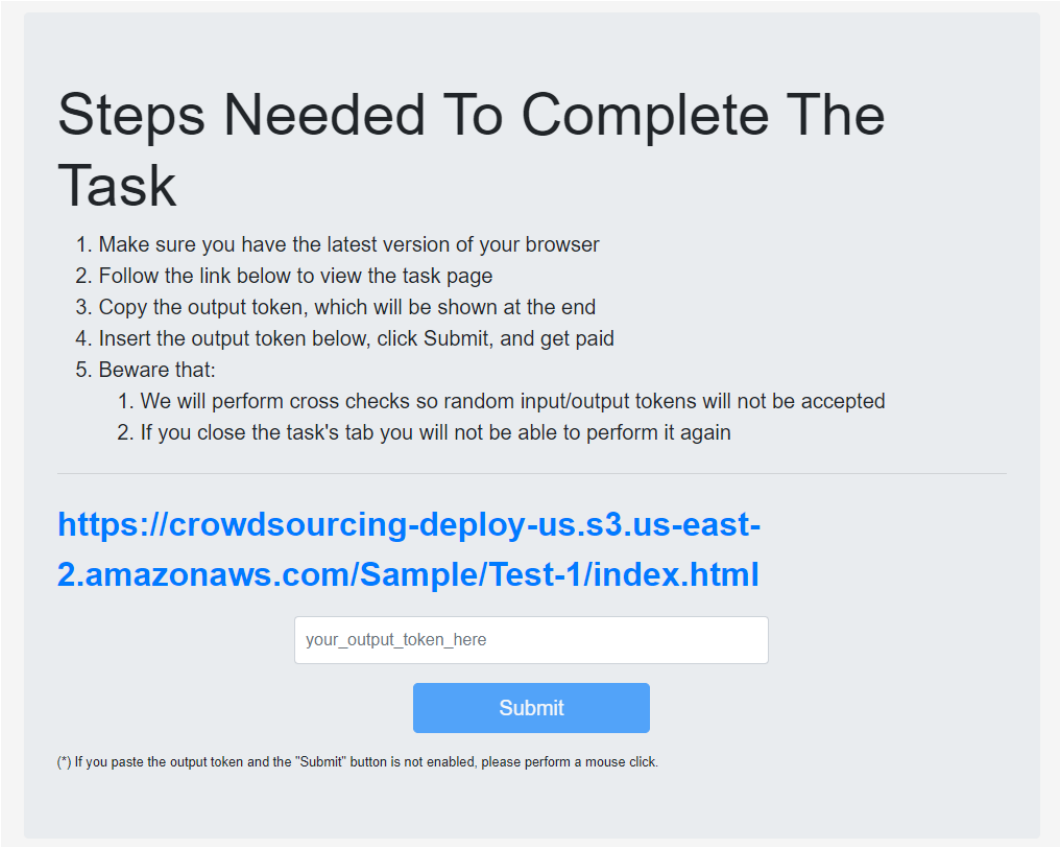

Figure 1.16: Wrapper interface for to a task configured Crowd_Frame that relies on Amazon Mechanical Turk for recruiting workers.

The worker recruited on the crowdsourcing platform clicks the link shown by the wrapper to reach the task and is redirected to the deployment on the public S3 bucket. They is then automatically and implicitly matched with an input token which is not already allocated to someone else. Crowd_Frame implements a custom allocation scheme that ensures consistency in HITs assignment. Then, they is shown an output token when completing the task. Such a token must be copied back into the wrapper, depending on the platform used. If the output token provided matches with the input token, the worker can be paid. Worker identifiers and input/output tokens are fundamental to ensure correct matching between workers and HITs.

1.5.3.4 Data Format

The data produced throughout the task is stored on DynamoDB tables, as shown in Figure 1.11. DynamoDB is a NoSQL database service, thus the records inserted in tables may contain structured data. The Skeleton component uses two different tables. One of these tables contains the access control list for the task deployed. In more detail, the access control list contains a single record for each worker. The attributes of the table vary depending on the platform of the provenance of the workers.

Listing 1.6 shows the CSV dump of a single record present in the access control list table of a task. The attributes include timestamps useful to understand when the worker participated, completed, or abandoned the task. There are also the identifiers and the tokens of the HIT assigned. It is possible to understand whether the worker should be paid or not and how many tries they performed, and so on.

```
1 "identifier","access_counter","batch_name","folder","generated","in_prog
  ↪ ress","ip_address","ip_source","paid","platform","status_code","task
  ↪ _name","time_arrival","time_completion","time_expiration","time_expi
  ↪ ration_nearest","time_expired","time_removal","token_input","token_o
  ↪ utput","try_current","try_left","unit_id","user_agent","user_agent_s
  ↪ ource"
2 "<<anonymized>>","1","Batch-9-Toloka","<<anonymized>>","true","false","<
  ↪ <anonymized>>","cf","true","toloka","200","<<anonymized>>","Fri, 17
  ↪ Mar 2023 09:25:36 GMT","Fri, 17 Mar 2023 09:30:04 GMT","Fri, 17 Mar
  ↪ 2023 10:25:36 GMT","Fri, 17 Mar 2023 10:25:54 GMT","false","PKPBSI
  ↪ NUKHS","BHPCIBOKBDL","1","10","unit_44","Mozilla/5.0 (Linux; Android
  ↪ 11; TECNO KG7h) AppleWebKit/537.36 (KHTML, like Gecko)
  ↪ Chrome/108.0.0.0 Mobile Safari/537.36","cf"
```

Listing 1.6: Format for a DynamoDB table that contains the access control list of a task deployed using Crowd_Frame.

The remaining table contains the data produced by the worker. It contains several records for each worker, depending on how the worker performs the task. The attributes of the table are always the same. The `sequence` attribute is a combination of worker and HIT identifiers, an index of the current element evaluated, and a sequence number. Such a combination is needed to identify each piece of data stored. The `access` attribute describes how the number of accesses to a HIT's element. The `action` attribute describes whether the worker is progressing through the HIT assigned or they is revising a previously evaluated element. The `element` attributes describe which kind of data is stored. The `index` element refers to the current element of the HIT. The `sequence_number` attribute is used to identify the sequence of data stored for the current worker. The `time` attribute is the data upload timestamp. The `try` attribute identifies the try performed by the worker. The `data` attribute contains the actual data produced during the task that needs to be stored. Listing 1.7 shows the CSV dump of a single record present in the data table of a task.

Each piece of data is a JSON object. A piece of data can consist of the values provided for a set of evaluation dimensions, the outcome of quality checks, the answers provided

```
"identifier","sequence","access","action","data","element","index","sequence_number","time","try"
"<<anonymized>>","<<anonymized>>-unit_118-1-10","1","Next",{...},"document","6","10","Tue, 23 Aug 2022 11:49
```

Listing 1.7: Format for a DynamoDB table that contains the data produced by the worker during a task deployed using Crowd_Frame.

for a questionnaire and so on. Listing 1.8 shows a sample JSON payload stored in the DynamoDB table for a single element of a HIT assigned to the worker. The worker evaluates nine different dimensions, and one of them requires providing a URL. The payload thus shows the presence of queries and responses retrieved by the search engine.

1.5.3.5 Cost Estimation

The architecture implemented using Amazon Web Services is pay-per-use, with some fluctuation depending on the request size, number and other factors. The AWS pricing calculator[10] allows making a rather precise cost prediction. The prices considered in the following refer to the AWS Region US-East-2 (Ohio). The most recent variant of the task deployed by Soprano et al. [62] and Draws et al. [21] is used to provide a sample estimation of the impact of the Skeleton component on the overall cost. The variant of the task deployed by Soprano et al. [62] and Draws et al. [21] considered involves 200 workers recruited from Prolific in a single batch.

The first service to consider is S3. The pricing[11] depends on various cost components, all determined based on the storage class chosen. Crowd_Frame uses the `Standard` class, usually recommended for general-purpose storage without particular requirements. The service applies a fee of \$0.023 for the first 50 TB/Month of data stored region-wide. Then, it applies a fee of \$0.005 for every 1000 HTTP requests of type PUT, COPY, POST and LIST, lowered to \$0.004 for all the remaining requests. Lastly, a fee is applied for bandwidth usage. Each AWS account receives free 100 GB of data transferred out of S3 to the rest of the internet. A fee of \$0.09 is applied beyond such a quantity for the first 10 TB/Month. The transfer of data from the internet to S3 is always free. The service applies additional fees for the usage of other features not required by Crowd_Frame. The amount of data stored in the S3 bucket is the size of the configuration and source code file for each batch of the task. A worker transfers a total of $7 + 3 = 10$ files each time they access the task. The access control list contains the number of accesses for each worker. The size of the configuration of the task deployed is roughly 0.98 MB. Its source code weighs roughly 5.38 MB. The service thus stores $(5.38 + 0.98) = 0.00636$ GB/Month. The workers access the task 214 times and thus send a total $214*(7+3) = 2140$ HTTP requests to perform it. The amount of data transferred given by is $214*0.00636 = 1.36$ GB/Month. Equation 1.1 shows the detailed computation of the storage cost component. Equation 1.2 details the cost of the data retrieval component in

[10] https://calculator.aws/#/
[11] https://aws.amazon.com/s3/pricing/

```
{
  "info": {
    "action": "Next",
    "access": 1,
    "try": 1,
    "index": 6,
    "sequence": 10,
    "element": "document"
  },
  "answers": { ... },
  "notes": [ ... ],
  "dimensions_selected": {
    "data": [ ... ],
    "amount": 9
  },
  "queries": {
    "data": [ ... ],
    "amount": 1
  },
  "timestamps_start": [ 1661254995.22 ],
  "timestamps_end": [ 1661255107.96 ],
  "timestamps_elapsed": 112.74000000953674,
  "countdowns_times_start": [],
  "countdowns_times_left": [],
  "countdowns_expired": [],
  "accesses": 1,
  "responses_retrieved": {
    "data": [ ... ],
    "amount": 15,
    "groups": 1
  },
  "responses_selected": {
    "data": [ ... ],
    "amount": 1
  }
}
```


Listing 1.8: JSON data stored for the element of a HIT evaluated by a worker during a task deployed using Crowd_Frame.

terms of HTTP requests. Equation 1.3 shows the detailed computation of the data transfer cost component. Finally, Equation 1.4 shows the overall cost estimation.

$$\begin{aligned}
\text{Data Storage} &= \$0.023 \text{ * Data Size (TB/Month)} \\
&= \$0.023 * (0.00636/1024) \\
&= \$0.023 * 0.00000621 = \$0.00000014 \quad (\$0 \iff \text{free tier}) \\
&\quad \text{storage class: standard} \\
&\quad \text{threshold: 50 TB/Month} \\
&\quad \text{free tier: 5 GB/Month}
\end{aligned} \tag{1.1}$$

$$\begin{aligned}
\text{Data Retrieval} &= \$0.0000004 * \text{Requests Amount} \\
&= \$0.0000004 * (\text{Worker Accesses} * (\text{Files Retrieved})) \\
&= \$0.0000004 * (214 * (7 + 3)) \\
&= \$0.0000004 * 2140 = \$0.000856 \quad (\$0 \iff \text{free tier}) \\
&\quad \text{storage class: standard} \\
&\quad \text{threshold: 1000 requests} \\
&\quad \text{free tier: 2000 requests}
\end{aligned} \tag{1.2}$$

$$\begin{aligned}
\text{Data Transfer} &= \$0.09 * \text{Data Transfer Size} \\
&= \$0.09 * \lceil \text{Worker Accesses} * \text{Data Size (GB/Month)} \rceil \\
&= \$0.09 * \lceil 214 * 0.00636 \rceil \\
&= \$0.09 * 1 = \$0.09 \quad (\$0 \iff \text{free tier}) \\
&\quad \text{storage class: standard} \\
&\quad \text{threshold: 10 TB/Month} \\
&\quad \text{free tier: 100 GB/Month}
\end{aligned} \tag{1.3}$$

$$\begin{aligned}
\text{S3} &= \text{Data Storage} + \text{Data Retrieval} + \text{Data Transfer} \\
&= \$0.00000014 + \$0.000856 + \$0.09 = \$0.09085614
\end{aligned} \tag{1.4}$$

The second service to consider is DynamoDB. The service offers two capacity modes[12] that come with specific billing options for processing read and write requests on the table. The on-demand capacity mode charges for the data reads and writes performed by an application on the tables. The provisioned capacity mode charges for the number of reads and writes that the application is expected to require. Crowd_Frame uses the on-demand mode since the amount of read and write requests depends on the number of workers recruited.

[12] `https://aws.amazon.com/dynamodb/pricing/`

A write request unit (WRU) is the billing unit of a set of API calls used to write data to tables. A standard write request unit can store items up to 1 KB. Additional write request units are used if the item is larger than 1 KB. A transactional write requires two units. A read request unit (RRU) is the billing unit of API calls used to read data from tables. A strongly consistent read request of up to 4 KB requires one read request unit. Additional read request units are used if the item is larger than 4 KB. An eventually consistent read request requires one-half request unit, while a transactional read requires four read request units. The type of read request chosen has an impact of consistency.[13] DynamoDB uses eventually consistent reads unless specified otherwise. Crowd_Frame sends read requests using the default parameters, thus relies on such type of reads. The response might not reflect the results of a recently completed write operation when the read request is eventually consistent. In such a case, a second read request is needed to retrieve the most up-to-date data. On the other hand, the usage of strongly consistent reads has some constraints and leads to using more throughput capacity, thus being more expensive. The fee applied for each write or request unit depends on the storage class chosen for the tables when the on-demand capacity modes are used. DynamoDB offers a Standard table class and a Standard-infrequent access class. Crowd_Frame relies on the former class. The on-demand capacity mode charges $1.25 for one million write request units and $0.25 for one million read request units. Data storage is another table class dependent cost component that must be considered. The first 25 GB/Month are free using the Standard class, and a fee of $0.25 is applied per GB/Month after that. There is not any free quota when using the Standard-infrequent table class. The fee is thus $0.10 per GB/Month. The service applies additional fees for the usage of other features not required by Crowd_Frame.

The variant of the task deployed by Soprano et al. [62] and Draws et al. [21] considered uses two of the three tables shown in Figure 1.11. The component uses only the access control list table and the data table. The log table is used by the Logger component. Its cost estimation process is described in Section 1.5.5.5. The data table has records of varying sizes. It depends on the type of data stored, as explained in Section 1.5.3.4. In the variant of the task considered there are at minimum 13 records for each try of each worker, depending on how they behave during the task. These records are broken down into (at least) 8 document records, 3 questionnaire records, a single record containing the task setup and worker attributes, and a single record addressing quality checks. An additional record may appear if the worker provides a final comment to the requester. Table 1.2 shows the parameters needed to estimate the cost for a single worker of the task considered. The data table for the whole 200 workers recruited for the task contains a total of 2183 document records, 787 questionnaire records, 262 general data records, 256 quality checks records and 41 comment records.

Equation 1.5 shows the detailed computation for the write request units cost component of the data table. Equation 1.6 shows the detailed computation for the read request units cost component of the data table. Equation 1.7 shows the computation for the storage cost of the data written. Equation 1.8 further summarizes the contribution of each cost component for the usage of the data table by Crowd_Frame for the sample task considered.

[13] https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

Table 1.2: Sample cost estimation parameters for the usage of the data table by the Skeleton component of Crowd_Frame.

Record Type	Average Amount	Average Size (KB)	Average WRUs	Average RRUs
`document`	11	182.85	183	23
`questionnaire`	4	14.48	15	2
`data`	1	38.86	39	5
`checks`	1	1.27	1	0.5
`comment`	1	0.04	1	0.5
Total	18	237.5	239	31

$$\begin{aligned}
\text{WRUs} &= \$1.25 * (\text{Workers Amount} * \text{Avg. Message Number}) * \\
&\quad \lceil \text{Avg. Payload Size (KB) / Unit Amount} \rceil \\
&= \$0.00000125 * (200 * 18) * \\
&\quad \lceil (182.85 + 14.48 + 38.86 + 1.27 + 0.04)/1 \rceil \\
&= \$0.00000125 * 3600 * (238 \text{ Avg. WRUs / Record}) \\
&= \$0.00000125 * 856800 \text{ Billable WRUs} = \$1.07 \\
&\quad \text{threshold: 1 million WRUs}
\end{aligned} \tag{1.5}$$

$$\begin{aligned}
\text{RRUs} &= \$0.25 * (0.5 * (\text{Workers Amount} * \text{Avg. Message Number}) * \\
&\quad \lceil \text{Avg. Payload Size (KB) / Unit Amount} \rceil) \\
&= \$0.00000025 * (0.5 * (200 * 18) * \\
&\quad \lceil 182.85 + 14.48 + 38.86 + 1.27 + 0.04)/4 \rceil) \\
&= \$0.00000025 * (0.5 * (3600 * (60 \text{ Avg. RRUs / Record})) \\
&= \$0.00000025 * 108000 \text{ Billable RRUs} = \$0.03 \\
&\quad \text{threshold: 1 million RRUs}
\end{aligned} \tag{1.6}$$

$$
\begin{aligned}
\text{Data Storage} &= \$0.25 * (\text{Message Number * Avg. Payload Size (GB)}) \\
&= \$0.25 * (200 * ((182.85 + 14.48 + 38.86 + 1.27 + 0.04)/1024/1024))) \\
&= \$0.25 * (200 * ((239/1024)/1024)) \\
&= \$0.25 * 0.00022793\ \text{GB/Month} = \$0.00005698 \quad (\$0 \iff \text{free tier}) \\
&\quad \text{threshold: 1 GB/Month} \\
&\quad \text{note: on-demand capacity mode, standard table class} \\
&\quad \text{free tier: 25 GB/Month}
\end{aligned} \tag{1.7}
$$

$$
\begin{aligned}
\text{Data Table} &= \text{WCUs} + \text{RCUs} + \text{Data Storage} \\
&= \$1.07 + \$0.03 + \$0.00005698 = \$1.10005698
\end{aligned} \tag{1.8}
$$

The access control list table has an average record size of roughly 800 B. Such an item consumes 0.5 RRUs when read with eventual consistency and 1 WRUs when being written. There is a total of 200 workers recruited and each worker has a single record. The total size of the table is roughly 160 KB, thus 160 WRUs are used. There may be a certain number of additional RRUs performed on the table. These RRUs take place when a worker does not complete the work assigned on time and the unit must be reallocated. In such a case, the table must be scanned to determine which unit to reallocate and update the affected workers' records. Such a scenario does not happen when considering the aforementioned variant. Otherwise, the RRUs computation depends on how many records were already present in the access control list when each additional worker was recruited. The impact of the access control list table is negligible due to its small number of records, request units and storage size.

$$
\begin{aligned}
\text{WRUs} &= \$1.25 * (\text{Workers Amount} * \text{Avg. Message Number}) * \\
&\quad \lceil \text{Avg. Payload Size (KB) / Unit Amount} \rceil \\
&= \$0.00000125 * (200 * 1) * \\
&\quad \lceil (800/1024)/1 \rceil \\
&= \$0.00000125 * 3600 * (1\ \text{Avg. WRUs / Record}) \\
&= \$0.00000125 * 3600\ \text{Billable WRUs} = \$0.045 \\
&\quad \text{threshold: 1 million WRUs}
\end{aligned} \tag{1.9}
$$

$$\begin{aligned}
\text{RRUs} &= \$0.25 * (0.5 * (\text{Workers Amount} * \text{Avg. Message Number}) * \\
&\quad \lceil \text{Avg. Payload Size (KB) / Unit Amount} \rceil) \\
&= \$0.00000025 * (0.5 * (200 * 1) * \\
&\quad \lceil (800/1024)/4 \rceil) \\
&= \$0.00000025 * (0.5 * (3600 * (1 \text{ Avg. RRUs / Record})) \\
&= \$0.00000025 * 1800 \text{ Billable RRUs} = \$0.00045 \\
&\quad \text{threshold: 1 million RRUs}
\end{aligned} \tag{1.10}$$

$$\begin{aligned}
\text{Data Storage} &= \$0.25 * (\text{Message Number * Avg. Payload Size (GB)}) \\
&= \$0.25 * (200 * (800/1024/1024/1024))) \\
&= \$0.25 * (200 * 0.00000075) \\
&= \$0.25 * 0.00014901 \text{ GB/Month} = \$0.00003725 \quad (\$0 \iff \text{free tier}) \\
&\quad \text{threshold: 1 GB/Month} \\
&\quad \text{note: on-demand capacity mode, standard table class} \\
&\quad \text{free tier: 25 GB/Month}
\end{aligned} \tag{1.11}$$

$$\begin{aligned}
\text{Table ACL} &= \text{WCUs} + \text{RCUs} + \text{Data Storage} \\
&= \$0.045 + \$0.00045 + \$0.00003725 = \$0.04548725
\end{aligned} \tag{1.12}$$

Equation 1.9 shows the detailed computation for the write request units cost component of the access control list table. Equation 1.10 shows the detailed computation for the read request units cost component of the access control list table. Equation 1.11 shows the computation for the storage cost of the records written. Equation 1.12 further summarizes the contribution of each cost component for the usage of the access control list table by Crowd_Frame for the sample task considered. Finally, Equation 1.13 summarizes the cost of the usage of the Skeleton component of Crowd_Frame for the task considered. It must be noted that data are only written to the data table during the task and read-only afterwards while downloading results. The cost of the RCUs component for the data table can be charged at a later time. Furthermore, the cost must be intended on a per-month basis only for the storage components of both S3 and DynamoDB.

$$
\begin{aligned}
\text{Skeleton Component} &= \text{S3} + \text{DynamoDB} \\
&= \text{S3} + \text{Table Data} + \text{Table ACL} \\
&= (\text{Storage} + \text{Data Retrieval} + \text{Data Transfer}) + \\
&\quad (\text{WRUs} + \text{RRUs} + \text{Data Storage}) + \\
&\quad (\text{WRUs} + \text{RRUs} + \text{Data Storage}) \\
&= (\$0.00000014 + \$0.000856 + \$0.09) + (\$1.07 + \$0.03 + \$0.00005698) + \\
&\quad (\$0.045 + \$0.00045 + \$0.00003725) \\
&= \$1.23640037
\end{aligned}
\tag{1.13}
$$

1.5.4 Search Engine

Crowd_Frame implements a component that allows integrating a customizable search engine within the task body, as shown by Figure 1.11.

1.5.4.1 Use Cases

The Search Engine component imitates the standard approach followed by the most popular search engines such as Google. Such an approach involves showing the search results found for a given query provided by a user.

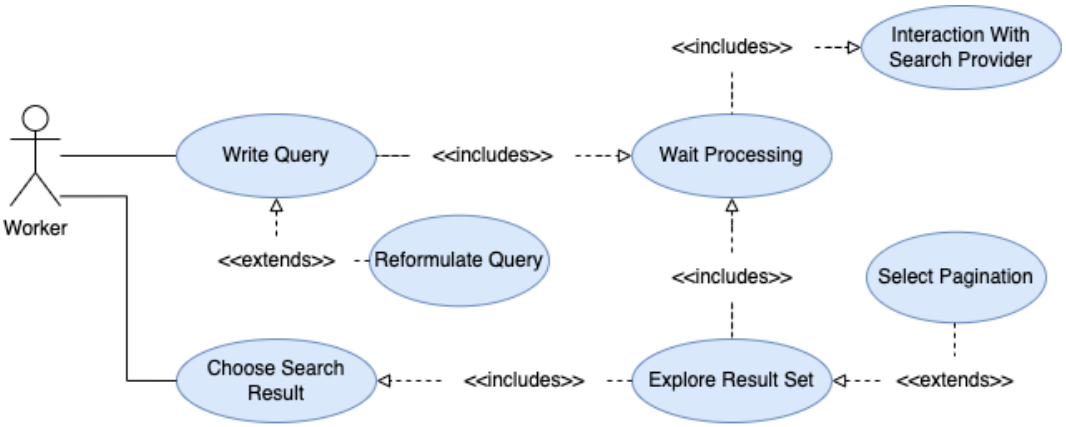

Figure 1.17: Use case diagram for a worker that performs a task deployed using Crowd_Frame.

The diagram shown in Figure 1.17 provides a high-level description of the interaction between a worker and the Search Engine component of Crowd_Frame. The worker writes a query in a simple text box shown by the user interface. The query is then sent to a search provider and the worker waits for its processing. The search provider processes the query and returns a result set. The results are shown below the query box in a tabular format.

Each search result is shown to the worker by providing its URL, page name and snippet. The result set is paginated by default. The worker can choose how many results to see for each page. The worker can reformulate the query and obtain a new result set at any time.

The approach followed by the component deviates from the standard one when exploring the result set. Integrating a custom search engine allows workers to provide a URL for certain evaluation dimensions. The underlying goal can be of various kinds. The requester, for instance, may want the worker to provide some kind of evidence, as done by Roitero et al. [57]. The user interface thus shows a button to the right of each search result shown. The worker explores each result and then finalizes the choice by clicking the corresponding button. Figure 1.17 shows a sample of the user interface of the Search Engine component. In more detail, the figure shows a result set made of 48 elements split into pages of 5 elements each retrieved for the query having text `Barack Obama`.

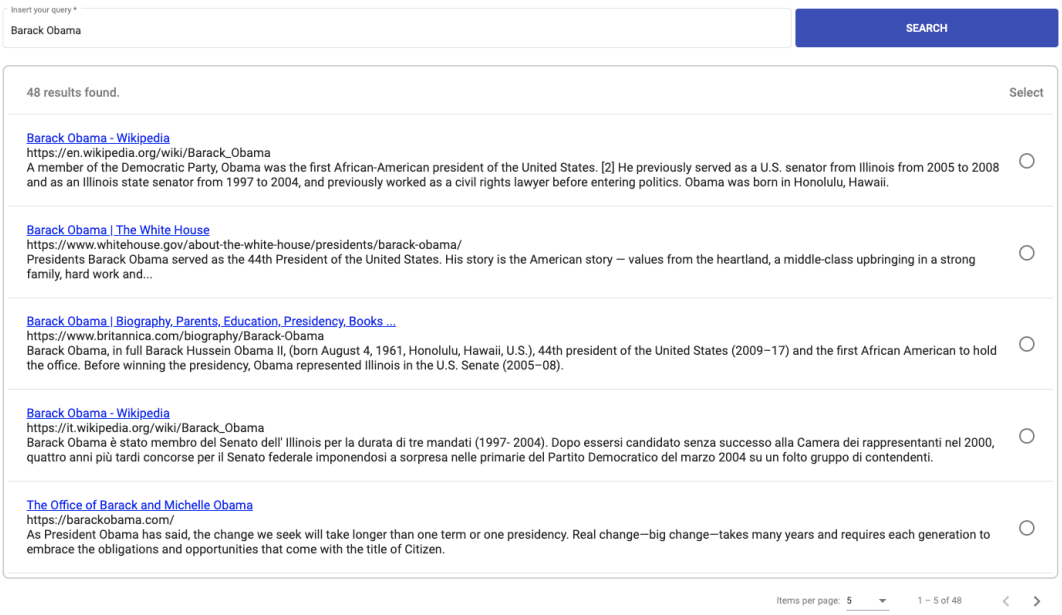

Figure 1.18: Interface of the Search Engine component of Crowd_Frame.

1.5.4.2 Architecture

Figure 1.15 details the interaction between the Search Engine component and the APIs of the search providers that a requester chooses while configuring a task. The query written by the worker is provided to a static method that encapsulates it in an HTTP message, along with the API Key of the search provider chosen. The component implements the API of three different search providers. The search provider processes the query and sends a second HTTP message to the component. The raw response is parsed using the corresponding model and then decoded using an interface that allows abstracting from the underlying data structure. The result set is composed of multiple base responses, depending on how many search results the provider retrieves. Such a set is thus provided to the user inter-

face that paginates and shows it to the worker. The worker is required to select one of the results provided by clicking the corresponding radio button. The Search Engine component implements the API of three search providers, in sub-modules named `BingWebSearch`, `PubmedSearch`, and `FakerWebSearch`.

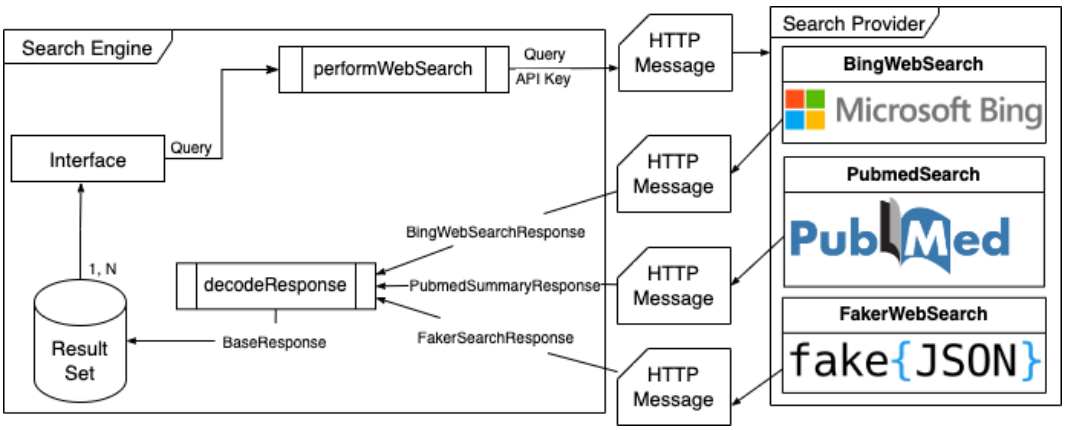

Figure 1.19: Interface of the Search Engine component of Crowd_Frame.

1.5.4.3 Microsoft Search API

The `BingWebSearch` provider implements the Bing Web Search API[14], part of the Microsoft Search API. It is mainly designed to be used as a result of a direct user query or search, or as a result of an action within a system that can be logically interpreted as a user's search request. Acceptable search or search-like scenario thus include a user that provides a query directly into a search box, a user that requests "more information" about a text or image using some kind of user interface control, and so on. The API returns and rank implicitly whatever content is relevant to a query. It is possible to filter and control to some extent the results retrieved. For instance, it is possible to include or exclude specific types of results, return pages discovered within the last week, and so on.

Crowd_Frame relies on the `v7.0` version of the Bing Web Search API. To use the API a developer must obtain a subscription key. Then, they can send HTTP GET messages to the API's endpoint[15] and thus retrieve results. The subscription must be added in a header of the HTTP request called `Ocp-Apim-Subscription-Key`. The GET parameter `q` is used to attach the user's query and must be URL-encoded. The component also provides four headers to improve the search experience for the worker. Table 1.3 describes such headers sent to the API. Furthermore, the component captures three headers returned in each response. The headers include, for instance, `X-MSEdge-ClientID`, which the component stores in a cookie after the first request, since it allows to recognize a user's device and should be attached to the subsequent one. Table 1.4 described such headers and why they are captured.

[14] `https://www.microsoft.com/en-us/bing/apis/bing-web-search-api`
[15] `https://api.bing.microsoft.com/v7.0/search`

There are various GET parameters[16] that can be used to customize the results retrieved. The component attaches two additional parameters to the query's text when sending a request to the API. Table 1.5 describes the GET parameters added while sending each query.

Table 1.3: Headers provided while sending requests to the Bing Web Search API.

Header	Value	Description
`User-Agent`	The user agent originating the request	Bing uses the user agent to provide mobile users with an optimized experience.
`X-MSEdge-ClientID`	Alphanumeric string	Bing uses the client id to provide users with consistent behavior across queries
`X-MSEdge-ClientIP`	The IPv4 or IPv6 address of the client device.	The IP address is used to discover the user's location. Bing uses the location information to determine safe search behavior.
`X-Search-Location`	A semicolon-delimited list of key/value pairs that describe the client's geographical location	Bing uses the location information to determine safe search behavior and to return relevant local content.

Table 1.4: Headers captured while receiving responses from the Bing Web Search API.

Header	Value	Description
`X-MSEdge-ClientID`	Alphanumeric string returned after the first request	Bing uses the client id to provide users with consistent behavior across queries
`BingAPIs-TraceId`	The ID of the log entry that contains the details of the request	Identifier of the request to provide to the support team in case of errors.
`BingAPIs-Market`	The form is `<languageCode>-<countryCode>`	The market used by Bing to process the query

Listing 1.9 shows a sample request sent to retrieve results for a query having text `microsoft devices` that contains the suggested headers and GET parameters. The request is reproduced using the cURL software. The response returned is a JSON object that contains various parameters and the array of web pages retrieved. Such an object is the one decoded by the component, as shown by Figure 1.15. The final result set for the query written by the worker and retrieved using the Bing Web Search API is then decoded using the static

[16] `https://learn.microsoft.com/en-us/bing/search-apis/bing-web-search/reference/query-parameters`

Table 1.5: Query parameters provided when sending queries to the Bing Web Search API.

Parameters	Description
`count`	The number of search results to return in a response.
`offset`	The number of search results to skip.
`mkt`	The market where the results come from. Typically, the country where the worker is making the query from.

method shown in Figure 1.19. The array of base responses is thus returned to the user interface of the Search Engine component of Crowd_Frame.

```
curl -H "Ocp-Apim-Subscription-Key: <yourkeygoeshere>" -H
↪  "X-MSEdge-ClientID: 00B4230B74496E7A13CC2C1475056FF4" -H
↪  "X-MSEdge-ClientIP: 11.22.33.44" -H "X-Search-Location:
↪  lat:55;long:-111;re:22" -A "Mozilla/5.0 (X11; Linux x86_64)
↪  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/29.0.1547.65
↪  Safari/537.36" https://api.bing.microsoft.com/v7.0/search?q=microsof
↪  t+devices&mkt=en-us&count=10&offset=0
```

Listing 1.9: Sample request that contains headers and query parameters sent to the Bing Web Search API.

1.5.4.4 Entrez Programming Utilities

The Entrez Programming Utilities[17] (E-utilities) is a set of nine server-side programs that provide a stable interface into the Entrez query and database system at the National Center for Biotechnology Information (NCBI). Such programs require the usage of fixed endpoints whose URL-based syntax translates a set of input parameters into the values necessary for the underlying software components to search for and retrieve the data requested. In other words, the E-utilities are the structured interface to the Entrez system, which includes 38 databases covering a variety of biomedical data. Table 1.6 provides a brief description of each E-utility. PubMed[18] is a free search engine that access primarily the MEDLINE[19] database which contains references to the biomedical literature. The United States National Library of Medicine (NLM) at the National Institutes of Health (NIH) maintain the database as part of the Entrez system.

The NCBI recommends using an API Key to access the E-utilities. Its usage helps to avoid overloading the underlying systems. Any IP address that sends more than 3 requests per second to the E-utilities without an API key receives an error message. On the other hand, IP addresses are allowed to send up to 10 requests per second when the key is pro-

[17] https://www.ncbi.nlm.nih.gov/books/NBK25501/
[18] https://pubmed.ncbi.nlm.nih.gov/
[19] https://www.nlm.nih.gov/medline/medline_overview.html

Table 1.6: General description the of the nine E-utilities provided by the Entrez system.

E-utility	Goal	Description
`EInfo`	Database statistics	Provides the number of records indexed in each field of a given database, date of the last update and available links towards other Entrez databases.
`ESearch`	Text searches	Responds to a text query with the list of matching UIDs (identifiers) in a given database for later use in other E-utilities, along with the term translations of the query.
`EPost`	UID uploads	Accepts a list of UIDs from a given database and responds with the web environment of the uploaded dataset and its query key.
`ESummary`	Document summary download	Responds to a list of UIDs in a given database with the corresponding document summaries.
`EFetch`	Data record download	Responds to a list of UIDs in a given database with the corresponding data records in a specified format.
`ELink`	Entrez Links	Responds to a list of UIDs in a given database with either a list of related UIDs (and relevancy scores) in the same database or a list of linked UIDs in another Entrez database and more.
`EGQuery`	Global Query	Responds to a text query by providing the number of records matching the query in each Entrez database.
`ESpell`	Spelling Suggestions	Retrieves spelling suggestions for a text query in a given database
`ECitMatch`	PubMed Batch citation search	Retrieves PubMed IDs (PMIDs) corresponding to a set of input citation strings.

vided. The API Key can be obtained from the NCBI account page.[20] Each request must be sent to the base endpoint `https://eutils.ncbi.nlm.nih.gov/entrez/eutils/`. The API key is added by appending its value to the `api_key` GET parameter. The E-utility wanted can be chosen by appending its lowercase name with the `.fcgi` suffix. The only exception is the ECitMatch utility, which requires the `.cgi` suffix. The usage of the ESummary utility, for instance, involves sending requests to the endpoint `https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi`. The `PubMedSearch` provider interacts with the ESummary and ESearch utilities of the Entrez system. The E-utilies must be combined to create an effective and useful data pipeline. The system provides the Entrez History server that simplifies transferring context and data between successive requests. The ESearch utility is the first block of the pipeline. The query provided by the worker is added as a parameter to the request. The request is sent to the PubMed search engine, which returns a list that contains the identifiers of items relevant to the query. There is a total of six GET parameters used to customize the results returned by the utility, shown in Table 1.7.

Table 1.7: Query parameters provided when sending requests to the ESearch utility of the Entrez system.

Parameters	**Usage**
`db`	Database from which retrieve results, which is `pubmed`.
`term`	Text query for which retrieve results. All special characters must be URL encoded. Spaces may be replaced by + character.
`usehistory`	ESearch will post the UIDs resulting from the search operation in the History server to be used in the subsequent call, when set to `y`.
`retmode`	Format of the returned output. The JSON format is used by the search provided.
`WebEnv`	Web environment returned by a previous ESearch, EPost or ELink request. ESearch appends the result set retrieved to the one contained in the pre-existing environment. The usage of `usehistory` is required.
`query_key`	Integer query key returned by a previous ESearch, EPost or ELink request. ESearch will find the intersection of the result set identified by the key and the one retrieved for the current `term`. The usage of `WebEnv` is required.

Listing 1.10 shows a sample initial request sent to the ESearch utility to retrieve a JSON set of identifiers for a query having text `vaccines`. The subsequent requests can use the `WebEnv` and `query_key` parameters if needed. The ESummary utility is the second and last block of the pipeline implemented by the search provider. The identifiers of each result item retrieved for the query written by the worker using the ESearch utility is used to fetch the details of the corresponding data records. The GET parameters used to customize the results returned are those shown in Table 1.7. The only difference is that the `term` parameter is replaced with the `id` one. Such a parameter requires either a single UID or a comma-delimited list of UIDs. The search providers all those retrieved by sending the query to the ESearch utility. For instance, let us hypothesize that one of such UIDs retrieved for

[20] `https://www.ncbi.nlm.nih.gov/account/`

the request shown in Listing 1.10 is 36511263. Listing 1.11 shows the request sent to the ESummary utility to fetch the details of the data record. The final result set for the query written by the worker and detailed using the pipeline implemented by the PubMedSearch provider is then decoded using the static method shown in Figure 1.19 and the array of base responses are returned to the user interface of the Search Engine component of Crowd_-Frame.

```
curl https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?api_key=your_api_key&db=pubmed&term=vaccines&usehistory=y&retmode=json
```

Listing 1.10: Sample initial request sent to the ESearch utility of the Entrez system.

```
curl https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi?api_key=your_api_key&db=pubmed&id=36511263&retmode=json
```

Listing 1.11: Sample initial request sent to the ESummary utility of the Entrez system.

1.5.4.5 fakeJSON

The fakeJSON[21] service provides a simple API for building mock data and helps the frontend development, end-to-end testing, data generation, etc. The underlying idea is that the developer sends a request directly to a single endpoint. In the request's body, they specify the format of the response desired. The service creates a JSON object populated with the data requested, initializing them with random values. The requests can be sent using any of the standard HTTP methods such as POST, PUT, DELETE, and PATCH. The service supports cross-origin resource sharing, thus allowing to receive asynchronous requests from anywhere. Every request must be sent to the endpoint https://app.fakejson.com/q. The FakerWebSearch provider interacts with the service to retrieve fake search result data, to allow testing the search engine of Crowd_Frame while designing a task. Every request sent to the service to generate fake data must comply with the same payload format. The payload must be a JSON object that includes a unique token, an optional set of parameters and the data field. The unique token can be obtained on the account page.[22]. The parameters allow customizing the behavior of the response generated depending on the needs. For instance, the parameters allow delaying the response by a fixed amount of seconds, or to add a set of custom headers. Table 1.8 shows the four parameters that can be included in the request's payload.

The data field of the request's payload is where the format of the responses to be returned is defined. It consists of fields, objects and arrays like any other valid JSON. It follows an attribute value pair format. The name of each key can be any string, however, the value must follow exactly the syntax of the corresponding data type defined by fakeJSON.

[21] https://fakejson.com/
[22] https://app.fakejson.com/member/token

Table 1.8: Payload parameters used to customize the behavior of the responses generated by the fakeJSON service.

Parameter	Usage
`code`	Specifying the response code number to fake.
`delay`	Delaying the response's sending by a fixed number of seconds.
`headers`	Adding additional custom headers in the response generated.
`consistent`	Disabling the service from returning the same fake dataset cached. The only possible value is `false`.

The `_repeat` attribute can be added to the object to specify how many iterations of an object the service should return.

Listing 1.13 shows a sample request sent to the service using the cURL software. Listing 1.13 shows the payload of the request sent by the search provided to generate fake data and allow the requester to test the search engine. The service is requested to generate a response with code 200. The response is made of 8 JSON objects. Each object has a `text` field that contains a long string. The `name` field contains a string of shorter length. The `url` field contains a random URL. The idea is imitating the results retrieved by a search engine, characterized by a page address, a page name and a text snippet that describes the page itself. In general, there are several formats[23] that can be used to mock data. The fake search result data generated by the fakeJSON service encapsulated into the `FakerWebSearch` provider is then decoded using the static method shown in Figure 1.19 and the array of base responses is returned to the user interface of the Search Engine component of Crowd_Frame.

```
curl --request POST --url https://app.fakejson.com/q --header
↪  'content-type: application/json' --data 'your_data_object'
```

Listing 1.12: Sample request sent to the fakeJSON service using cURL.

1.5.4.6 Cost Estimation

A platform that provides some kind of search API usually relies on a payment model that considers usually the number of queries issued and their sending rate on a per-second basis as parameters to estimate the overall cost. In light of this, a more adequate estimation of the usage cost of the Search Engine component of Crowd_Frame can be performed by relying on the data gathered for tasks deployed in the past, as described in Section 1.5.3.5 for the Skeleton component. The most recent variant of the task deployed by Soprano et al. [62] and Draws et al. [21] is thus used to estimate also such a usage cost.

The variant of the task considered publishes 200 HITs. The workers are required to provide a URL using the search engine for one of the evaluation dimensions. A total amount

[23] `https://fakejson.com/documentation#request_data`

```
{
  "token": "...",
  "parameters": {
    "code": 200
   },
  "data": {
    "url": "internetUrl",
    "name": "stringShort",
    "text": "stringLong",
    "_repeat": 8
  }
}
```

Listing 1.13: Payload of the request sent to the fakeJSON service to generate fake search results.

of 237 workers access the task deployed to try to complete the HIT assigned. The workers issue 3520 different queries. The minimum number of queries issued by a worker is 2. This worker abandons the task after a short while. The maximum number is 58. Workers with high amounts of queries issued perform multiple tries or write multiple queries for one or more elements of the HIT assigned. The workers issue on average roughly 15 queries each. The last query is issued after 2 days and 8 hours, 2 minutes, and 17 seconds from the first one. The queries can be split into 2498 unique transactions-per-second (TPS) blocks. A TPS block contains a set of queries sent within a single second. It is thus possible to detect 2498 seconds between the first and the last query. During such seconds, an arbitrary number of workers issued one or more queries. These 2498 seconds (roughly 42 minutes) happened between the timestamps of the first and last query. The smallest TPS size is 1. A block of such a size contains a single query issued by a single worker in a single second. The biggest TPS block size is 5. A block of such size contains five queries issued by one or more workers in a single second. The average TPS size is 1.40. The definition does not imply that the TPS blocks are consecutive. Splitting the queries into such blocks allows understanding the query throughput towards the platforms that provide the search APIs. The variant of task considered uses the `BingWebSearch` provider, thus relying on the Microsoft Bing Search API. The pricing plans[24] of the API differs along three parameters. These parameters are the amount of TPS allowed, the type of search offered, and the cost of 1000 transactions. The last parameter differs when using or not the Japanese geographical market. Crowd_Frame relies on the Bing Web Search subset of the API for the United States market within the S3 pricing plan. The plan allows a maximum TPS size of 100. In other words, a maximum number of 100 queries can be received each second by the API. The usage of the service is throttled to be within the threshold if it is exceeded. This translates into a slower search experience for the user. The plan applies a fee of $4 for a set of 1000 transactions. A transaction is a successful request received by the API. Usually, a single request is sent for a single query. The usage of advanced features such as auto-completion

[24] https://www.microsoft.com/en-us/bing/apis/pricing

leads to more transactions to process a single query. However, the provided implemented by Crowd_Frame does not use such a feature. The fee is thus applied four times since there 3520 queries have been issued. The maximum TPS size obtained during the task is 5, far below the threshold of 100 transactions per second. This means that not any query saw its processing throttled. Equation 1.14 shows the computation of the cost of the custom search engine used for the task considered.

$$\begin{aligned} \text{Bing Web Search} &= \$4 * \lceil \text{Query Number / Billing Threshold} \rceil \\ &= \$4 * \lceil 3520/1000 \rceil = \$4 * \lceil 3.52 \rceil = \$16 \\ &\quad \text{threshold: 1000 transactions} \end{aligned} \tag{1.14}$$

The usage of the `PubmedSearch` provider does require any form of payment. The Entrez system and its E-utilities are publicly available for free. The only requirement is complying with the guidelines suggested by the NCBI. Similarly, the `FakerWebSearch` provided does not enforce any payment. The pricing model[25] of the fakeJSON service offers a free plan constrained to 1000 requests per day, which are usually enough to test the task's interface during its design. The service stops providing search results if the threshold is reached. Eventually, it is possible to upgrade the account to plans with a higher number of maximum requests per day. The cheapest plan is the Developer one, which costs $12 per month and offers up to 50000 requests per day, along with other features.

1.5.5 Logger

Crowd_Frame implements a logging component that allows capturing worker behavior during the task, as shown by Figure 1.11.

1.5.5.1 Architecture

The log messages produced when capturing user behavior events are stored using a cloud-based logging server which relies on the infrastructure provided by Amazon Web Services. Figure 1.20 shows an overview of the whole logging pipeline.

API Gateway[26] is a service used to implement APIs for web applications or other AWS services. It addresses traffic management, cross-origin resource sharing (CORS) support, authorization and access control, request throttling, and monitoring of an API layer. In more detail, Crowd_Frame uses an HTTP API layer. Each user action is captured by the Logger component and sent to the layer using an HTTP message. The CORS is an HTTP-header based mechanism that allows a server to indicate any origins (in terms of port, domain or scheme) other than its own from which a browser should permit loading resources. The API layer's CORS is configured to allow receiving POST messages only. The layer receives the messages through a single endpoint provided by the application. The body of each message is redirected to a queue upon reception. Simple Queue Service[27] (SQS) is a

[25] `https://fakejson.com/pricing`
[26] `https://aws.amazon.com/api-gateway/`
[27] `https://aws.amazon.com/sqs/`

service used to create and manage queues of messages. It allows the creation of two types of queues, namely Standard or FIFO. The first type aims to ensure the best delivery sequence and each message may be delivered more than once if its processing fails or does not complete on time. The second type, on the other hand, ensures a first-in-first-out message delivery sequence where each message is processed exactly once. The main drawback is that the cost is higher. The usage of a message queue allows decoupling of the reception of the log requests from their processing. The payload of each log request has a sequence number. Each request is independent and the whole stream of messages can be reordered at a later time. Crowd_Frame thus uses a standard queue. The attached access policy allows only the gateway to send messages to the queue itself.

Lambda[28] is a service that provides serverless computing service that allows running source code without explicitly provisioning and maintaining servers of any kind. One of the core features of the service is the auto scalability feature of the instances depending on the size of the workload that the code must handle. In other words, Lambda allows running an algorithm only when needed, using a self-activating approach. A serverless function deployed using Lambda polls the queue in search of new messages and collects batches of up to 100 messages. The function parses each log message and the user action is stored as a record in the DynamoDB log table shown in Figure 1.11. The whole logging system back-end can be deployed also on a private server. The requester can set a custom endpoint to where each log message will be sent. There is a trigger between the function and the queue. The trigger is activated when the queue receives a new message. In more detail, the lambda function provides a JavaScript algorithm. Initially, it performs a polling operation of a batch of log requests. The JSON payload of each message is parsed and the server time is added. Then, the payload is stored in the table. The function is configured to repeat the polling of new log requests every 20 seconds. It uses by default five parallel long polling connections. The service can reduce the interval between each polling operation and increase the number of instances assigned to the functions. Such a feature helps empty the queue when it is filling up, thus avoiding overload. In such a case, the messages would need to be enqueued again. There is a DynamoDB table for each batch deployed for each task. The Logger component targets each table accordingly.

1.5.5.2 Event Handling

Angular uses custom markup elements. Such elements are defined using an extension of HTML that allows defining tags whose content is created and controlled by client-side scripting implemented using JavaScript or Typescript. In other words, these markup elements are useful in a dynamic environment, where flexibility is needed to create constantly changing web experiences. Angular developers were thus forced to provide a mechanism to bind event listeners to the custom elements used. Thus, in each custom element tag, additional syntax can be used to define the listener and specify the behavior when the corresponding event is triggered. This technique is adequate when there are only a few elements. In a real-world scenario where tens if not hundreds of tags are generated, another approach is needed. Angular implements so-called `Directives`.[29] A `Directive` is a class that can

[28] `https://aws.amazon.com/lambda/`

[29] `https://angular.io/guide/attribute-directives`

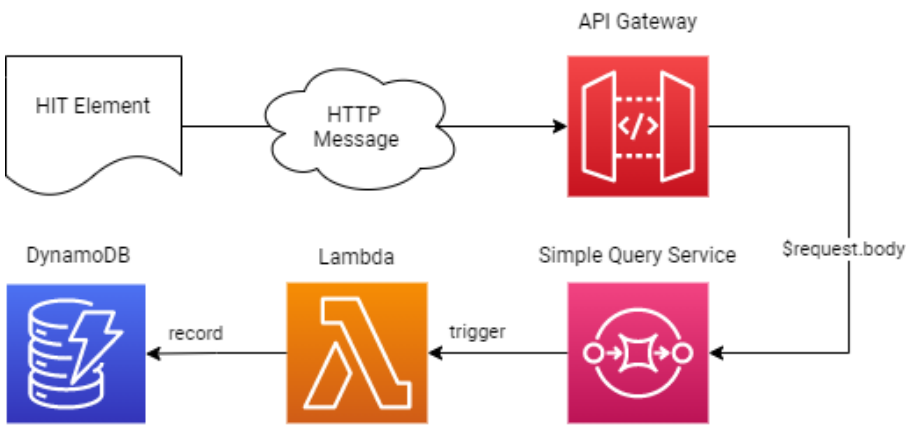

Figure 1.20: Infrastructure of the Logger component of Crowd_Frame.

dynamically add behavior to custom elements. A decorator is used to specify a CSS selector to target the custom elements. A constructor is used to pass a function that provides the implementation of the custom behavior. Furthermore, it attaches an event listener. The overall idea, thus, is that the set of markup elements to target depends on the type of event to log.

There are several different events to monitor. Listing 1.14 shows the generic JSON payload of each log request sent after having listened to a given event. The `detail` object contains the data logged depending on the type of the event. Each event detected is thus encapsulated by the Logger component into an instance of the base payload which is then enriched using the custom properties of the current event. Then, it is sent to the AWS infrastructure using an HTTP POST request to be processed. The server time is added when the infrastructure receives the message. Section 1.5.5.6 describes the payload of each event monitored by the Logger component inserted into the `detail` object of the base payload. The list of events is obtained from the analysis of the HTML structure of Crowd_Frame.

There is a directive available for each event. Every directive shares the same base template. Each event is monitored relative to a custom element selected using its CSS selector. A custom function to handle each event exists within the Logger component. Both the element and the logging function are thus provided to the directive constructor and the directive adds the custom behavior to the element targeted. Such an event listening behavior is attached to the corresponding markup element if and only if the Logger component is enabled by the requester and configured to log the corresponding event. The requester can enable or disable each event using the Generator component. The event listeners debounce and optimize function calls to reduce the processing load triggered by spammable events such as mouse clicks, movements and scrolls. The `RxJS`[30] library is used to further enhance the logging capabilities. It allows the addition of the event listeners while piping a set of operations to extract data and perform various manipulations. The usage of such a library allows the composition of events. For instance, a text selection event starts only when the

[30] `https://rxjs.dev/`

```
{
    "bucket": "string",
    "worker": "string",
    "task": "string",
    "batch": "string",
    "unitId": "string",
    "try_current": "integer",
    "type": "string",
    "sequence": "integer",
    "client_time": "string",
    "server_time": "string",
    "details": {
        ...
    }
}
```

Listing 1.14: Payload of each log request sent by the Logger component of Crowd_Frame.

mouse button is pressed and held and terminates when the mouse button is released.

1.5.5.3 Performance Evaluation

Locust[31] is a Python-based performance testing tool. It allows the definition of custom user behavior and swarms a system with millions of simultaneous requests. The main script allows modelling a worker node. Each worker node, in this case, sends POST requests to the provided host address (i.e., the endpoint exposed by the API gateway). The cluster mode of Locust leads to having a master node that coordinates the worker nodes by declaring how many of them are required, the request spawn rate, and the duration of the test. The tool provides a file containing various statistics when the test ends. The master node is spawned on a local machine. Each instance of the cluster runs two processes that spawn log requests, one for each instance's core, for a total starting amount of 200 processes. Each process sends a message every 10 ms. The test involves five rounds, where each round lasts for five minutes. The number of worker nodes doubles after each round, up to a total of 1600 nodes.

The pipeline has been initially tested on a dedicated server using an Amazon Elastic Compute Cloud[32] instance based on a 2nd generation AMD EPYC processor with 4 cores/8 threads, running at frequencies up to 3.3 GHz, and having 16 GB of RAM. Figure 1.21 shows that the server manages to process, without losing messages, up to 5000 requests/s. Over this limit, the server overloads thus failing progressively to accept new requests. The mean (median) time estimated between each log message to reach such a limit is 2.76 (1) seconds. Thus, at least 5000 workers working at the same time are required to overload the server. Such behavior is confirmed by looking at the requests managed per second, shown in Figure 1.22. CPU and memory usages drop with more than 800 worker nodes, as shown in

[31] https://locust.io/
[32] https://aws.amazon.com/ec2/

Figure 1.23 and Figure 1.24. The causes of such an overload cannot be determined precisely, since EC2 does not fully disclose the details of the underlying architecture. However, a hypothesis can be made in light of the data concerning the requests managed per second and CPU load. In more detail, it can be seen that the CPU is put under at nearly 100% of usage for a whole minute. Then, the performance drops abruptly. This could happen due to thermal throttling. The machine thus reaches the maximum number of manageable concurrent connections after one minute and then stops accepting new requests. The performances of the AWS-based pipeline have been tested using a cluster of 100 EC2 instances. The chosen type of instance is a t3.micro, which features two cores from an Intel Xeon Platinum 8000 series processor with a clock speed of up to 3.1 GHz, 1 GB of RAM, and up to 10 Gb/s of network burst bandwidth. Each instance deployed the content of an image available on the Docker Hub.[33] The image is based on Ubuntu and contains a Locust script that allows interaction with the other nodes of the cluster. The management of the cluster of instances has been performed using Elastic Container Service.[34]

The first round of the tests (i.e., the one with 200 worker nodes only) lead to 16538 Lambda invocations to manage the whole set of 1653801 requests sent by the cluster, in contrast with the 575000 requests processed by the dedicated server solution. The estimated mean number of requests processed each second by the pipeline is 5512. A negligible amount of messages is lost, due to network communication. This confirms that the API layer of the infrastructure can manage each request rapidly and forward it to the message queue, preventing network congestion. The maximum default limit of concurrent Lambda instances is 1000. Such a limit has not been reached during the test. The maximum number of concurrent invocations is 206. In other words, the pipeline can scale in size.

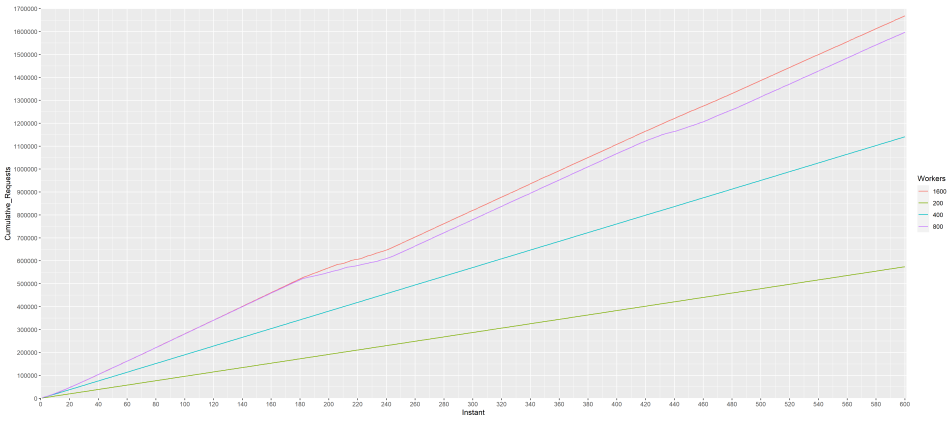

Figure 1.21: Requests sent to evaluate the performances of the infrastructure during each round of the test.

[33] hhttps://hub.docker.com/
[34] https://aws.amazon.com/ecs/

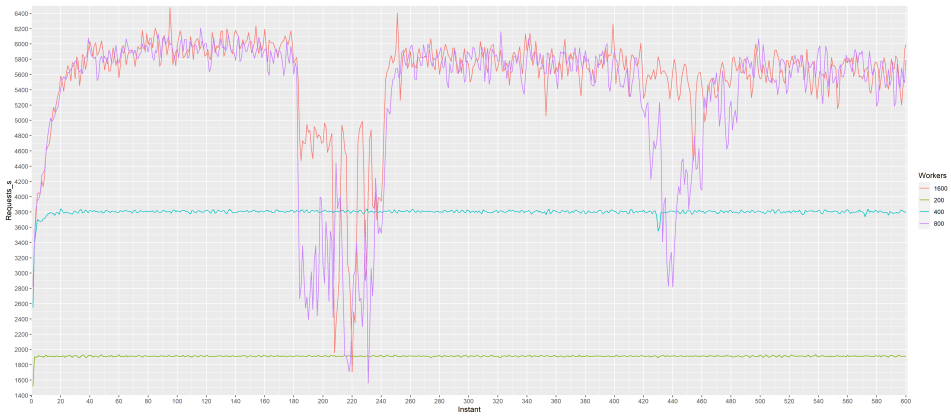

Figure 1.22: Requests managed per second by the dedicated server solution during each round of the test.

Figure 1.23: CPU usage of the dedicated server solution during each round of the test.

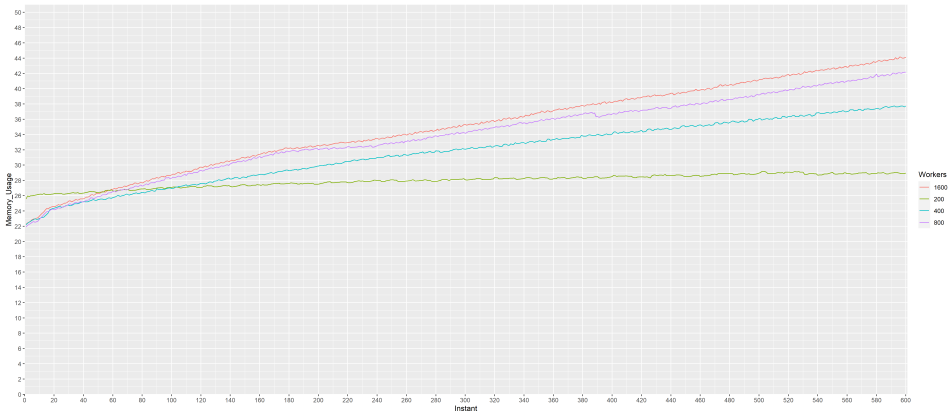

Figure 1.24: Memory usage of the dedicated server solution during each round of the test.

1.5.5.4 Pilot Experiment

A variant of the task proposed by Roitero et al. [58] where workers are asked to evaluate the truthfulness of information items has been published on Amazon Mechanical Turk to gather real log data. The data is used to estimate the impact of the Logger component on the overall cost of a crowdsourcing task deployed using Crowd_Frame. The component has been configured to monitor every log event described in Appendix 1.5.5.6.

The task involves 46 workers, who produce 12051 log requests. Each worker produces a mean value of 262 requests. The median value is 155. The minimum number of requests sent by a worker is 11, while the maximum is 1192. The mean task duration is 447 seconds and the median value is 230 seconds. The shortest try performed by a worker lasts for 2 seconds, while the longest lasts for 4588 seconds. The most frequent log event types collected are mouse movements, scrolls and clicks. Figure 1.25 shows the distribution of event types detected by the component during the task. As one may expect, the most frequent event detected is mouse movement. Figure 1.27 reconstructs the behavior of a worker that answers one of the questionnaires of the task deployed. The numbered points represent clicks, the blue line represents the movement trace, and the orange rectangles define a section of text selected. Figure 1.26 reconstructs the behavior of a worker evaluating the first element of the HIT assigned.

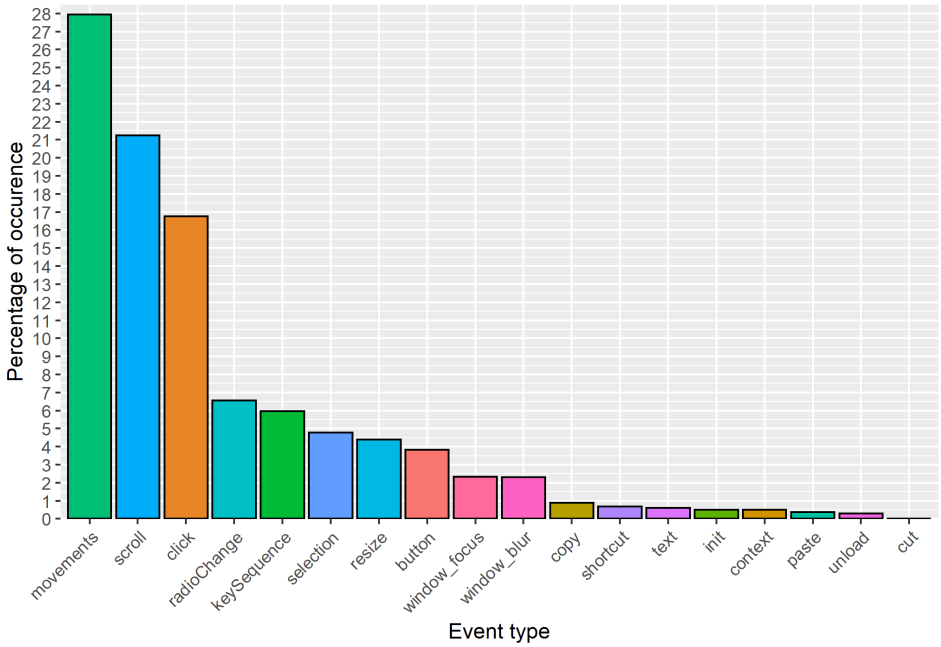

Figure 1.25: Event type distribution of the pilot test deployed to gather real data using the Logger component.

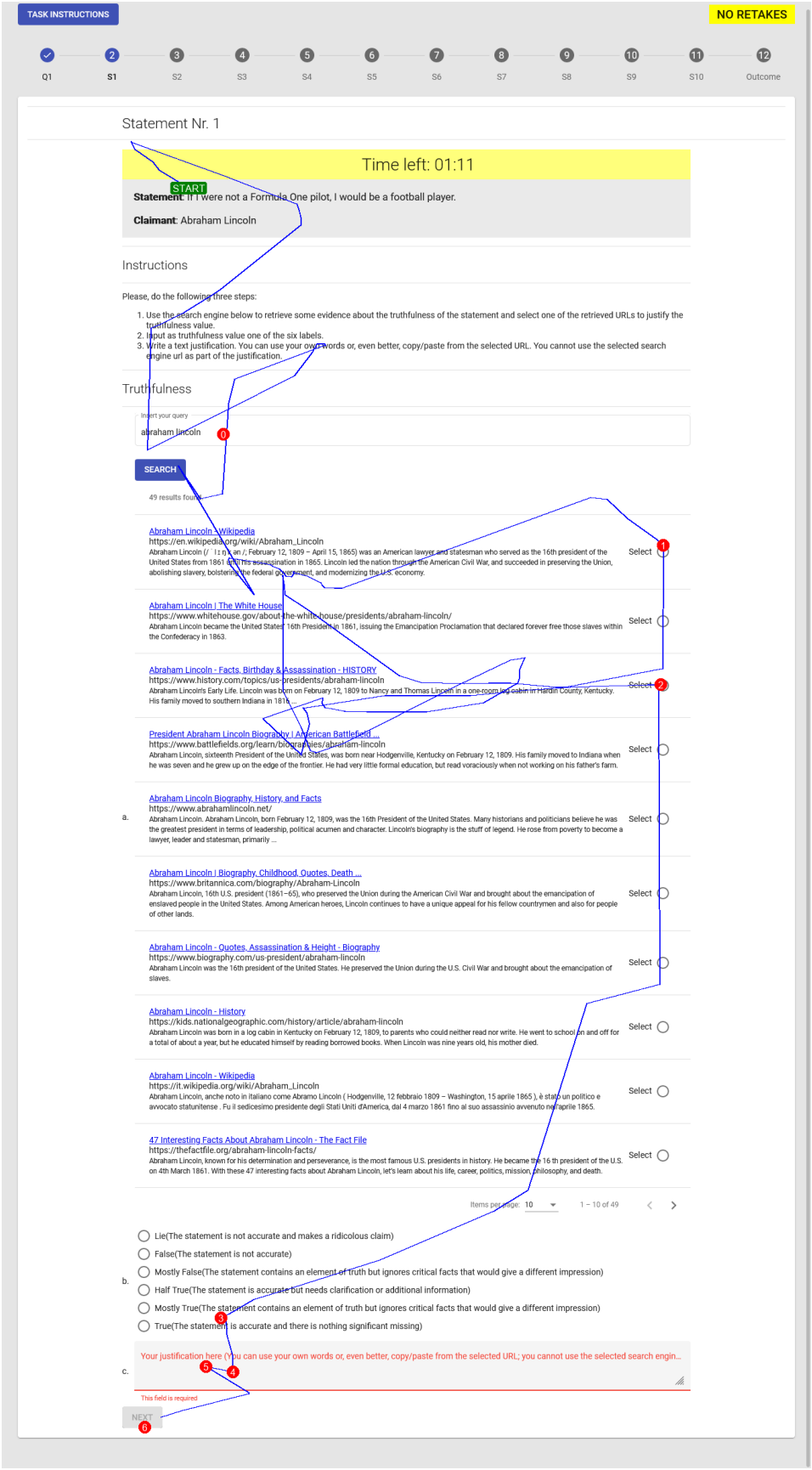

Figure 1.26: Worker behavior while evaluating a HIT's element reconstructed using the log data produced by the Logger component.

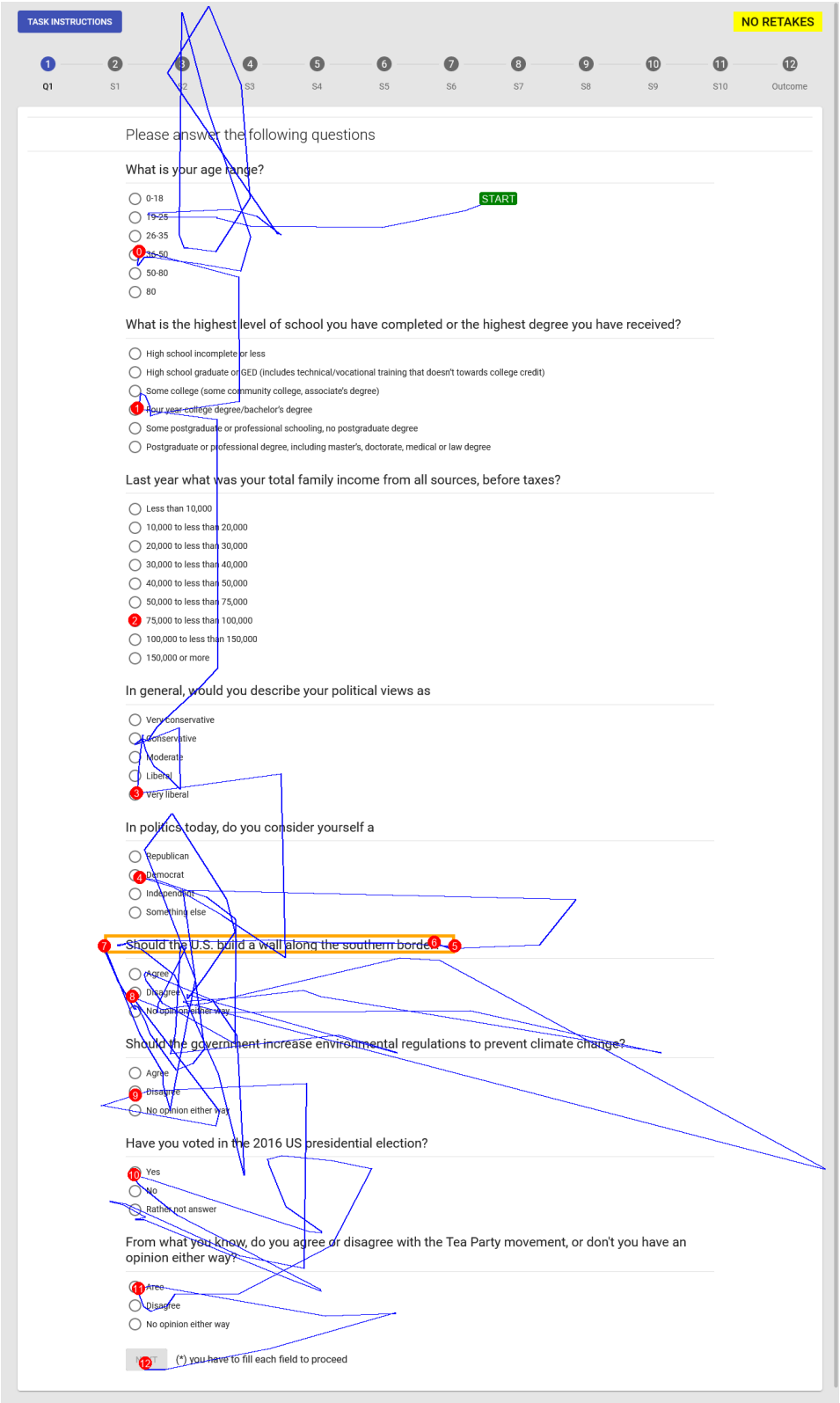

Figure 1.27: Worker behavior while answering a questionnaire reconstructed using the log data produced by the Logger component.

1.5.5.5 Cost Estimation

Let us hypothesize a monthly stream of one million log messages where the average size of each message is the pilot test described in Section 1.5.5.4. Each message thus contains, on average, a header of 550 bytes and a body of 529 bytes, for a total size of 1079 bytes. Let us recall the logging pipeline described in Figure 1.20.

The first AWS service to consider is API Gateway. For an HTTP API, the service applies a fee[35] of $1 for one million of request units up to 300 million a month, $0.90 otherwise. The first million requests per month are free. Requests units are defined as having 512 KB of data. A request with 513 KB of data is counted as two separate units. The cost of one million requests is thus excepted to be $1, given the average weight of the requests considered. Equation 1.15 shows the detailed computation of each cost component.

$$\begin{aligned}
\text{API Gateway} &= \$1 * (\text{Message Number} * \lceil \text{Average Message Size (KB) / Billing Factor} \rceil) \\
&= \$1 * (1000000 * \lceil (1079/1024) \text{ / } 512 \rceil \\
&= \$1 * 1 \text{ Billable Requests} = \$1 \quad (\$0 \iff \text{free tier}) \\
&\quad \text{threshold: 300 million requests} \\
&\quad \text{free tier: first million requests}
\end{aligned} \tag{1.15}$$

The second service to consider is Simple Queue Service. The service applies a fee[36] of $0.40 for one million of requests units a month, up to 100 billions. A message with a payload of size up to 64 KB is considered as a single request unit. The first million of requests per month is free. The service applies a fee also for data transfer, however inbound transfers are free of charge. The same holds also for outbound data towards a Lambda function in the same region. In light of this, the cost expected for data transfer is $0. Equation 1.16 shows the detailed computation of each cost component.

$$\begin{aligned}
\text{Simple Queue Service} &= \$0.0000004000 * (\text{Multiplier} * \text{Requests Number (M)}) \\
&= \$0.0000004000 * (\text{Multiplier} * (\text{Message Number} * \\
&\quad \text{(Average Message Size (KB) / Max Payload Size)})) \\
&= \$0.0000004000 * (1000000 * ((1079/1024)/64)) \\
&= \$0.0000004000 * (1000000 * (0.016464)) \\
&= \$0.0000004000 * 16464 \text{ Requests} \\
&= \$0.0000004000 * 0 \text{ Billable Requests} = \$0 \\
&\quad \text{threshold: 1 million requests} \\
&\quad \text{free tier: first million requests}
\end{aligned} \tag{1.16}$$

[35] `https://aws.amazon.com/api-gateway/pricing/`
[36] `https://aws.amazon.com/sqs/pricing/`

The third service to consider is Lambda. The pricing of a function depends on several factors such as the number of executions requested, the duration of each request and the amount of memory allocated for processing. The performance estimation described in Section 1.5.5.3 allows understanding that in an ideal setting, each invocation of a Lambda function retrieves 100 requests and evaluates them in at most 5 seconds. On the other hand, in a not optimal setting, each function invocation retrieves up to 10 requests and executes them in around 500 milliseconds. The cost component[37] of the service to consider are thus three. The service applies a fee of \$0.20 for every million requests. Then, it applies a fee of \$0.0000166667 for every GB-second of execution of an x86-based architecture, for the first 6 billion GB-seconds. Lastly, it applies a fee of \$0.0000021 for every second of usage of 128 MB of primary memory. In the ideal setting, 10000 function invocations lasting 5 milliseconds each are needed. In the not optimal setting, 10000000 invocations lasting 50 milliseconds each are needed. A final average estimation can thus be 5000000 function invocations lasting 250 milliseconds each for a total amount of 0.54 GB to process. Equation 1.17 shows the detailed computation of each cost component.

$$
\begin{aligned}
\text{Lambda} &= \$0.0000166667 * \text{Max}(\text{Invocations Number} * \text{Avg. Proc. Time (s)}* \\
&\quad (\text{Mem. Allocation (GB)}), 0) + \$0.0000002 * \text{Invocations Number} \\
&= \$0.0000166667 * \text{Max}(500000 * (250/1000 * 0.125), 0) + \$0.0000002 * 500000 \\
&= \$0.0000166667 * \text{Max}(125000 * 0.125), 0) + \$0.10 \\
&= \$0.0000166667 * \text{Max}(15625 \text{ GB/s}, 0) + \$0.10 \\
&= \$0.0000166667 * 15625 \text{ Billable GB/s} + \$0.10 = \$0.3603 \quad (\$0 \iff \text{free tier}) \\
&\quad \text{thresholds: 1 million invocations, first 6 billions GB-Sec/Month} \\
&\quad \text{free tier: subtract 40000 GB/s from Max() left hand side}
\end{aligned}
\tag{1.17}
$$

The last service to consider is DynamoDB. Section 1.5.3.5 describes the role of each cost component of DynamoDB. Equation 1.18 shows the detailed computation of each cost component for the example considered.

[37] `https://aws.amazon.com/lambda/pricing/`

$$
\begin{aligned}
\text{DynamoDB} &= \text{WRUs} + \text{RRUs} + \text{Data Storage} \\
&= \$1.25 * (\text{Message Number} * \lceil \text{Avg. Payload Size (KB) / Unit Amount} \rceil) + \\
&\quad \$0.25 * (0.5 * (\text{Message Number} * \lceil \text{Avg. Payload Size (KB) / Unit Amount} \rceil)) + \\
&\quad \$0.25 * (\text{Message Number * Avg. Payload Size (GB)}) \\
&= \$0.00000125 * (1000000 * \lceil (1079/1024)/1 \rceil) + \\
&\quad \$0.00000025 * (0.5 * (1000000 * \lceil (1079/1024)/4 \rceil)) + \\
&\quad \$0.25 * (1000000 * ((1079/1024)/1024)/1024) \\
&= \$0.00000125 * (1000000 * 2 \text{ Billable WRUs}) + \\
&\quad \$0.00000025 * (1000000 * 0.5 \text{ Billable RRUs}) + \\
&\quad \$0.25 * (1 \text{ GB}) \\
&= \$0.00000125 * 2000000 + \$0.00000025 * 500000 + \$0.25 * 1 \\
&= \$2.50 + \$0.125 + \$0.25 = \$2.875 \quad (\$2.67 \iff \text{free tier}) \\
&\quad \text{threshold: 1 million WRUs, 1 million RRUs} \\
&\quad \text{note: on-demand capacity mode, standard table class} \\
&\quad \text{free tier: 25 GB/Month data storage}
\end{aligned}
\tag{1.18}
$$

Equation 1.19 estimates the cost required to maintain the infrastructure of the Logger component of Crowd_Frame hypothesizing that one million of log requests is processed each month. After a year of usage, the total cost of the Logger component would thus be roughly around $51.

$$
\begin{aligned}
\text{Logger} &= \text{API Gateway} + \text{SQS} + \text{Lambda} + \text{DynamoDB} \\
&= \$1 + \$0 + \$2.875 + \$0.3603 \\
&= \$4.2353
\end{aligned}
\tag{1.19}
$$

1.5.5.6 Log Events

List of events monitored by the Logger component of Crowd_Frame described in Section 1.5.5. The payloads shown in the following are encapsulated in the `detail` object of the base payload shown in Listing 1.14.

Context The log message of `"type": "context"` contains information about the user agent and the IP address of the worker. It is the only not containing the `section` field.

```
1 {
2   "ua": "string",
```

```
  "ip": "string"
}
```

Mouse Movements The log message of `"type": "movements"` describes a mouse movement performed by the worker. The event is detected every 100 ms. The timestamp and (x, y) coordinates are mapped to a dictionary and buffered. The dictionaries contained in the buffer are pushed in an array of the `details` after a dwell time of 500 ms. Each dictionary is pushed in the `points` array.

```
{
  "section": "string",
  "points": [
    {
      "timeStamp": "string",
      "x": "integer",
      "y": "integer"
    },
    {
      "timeStamp": "string",
      "x": "integer",
      "y": "integer"
    },
    ...
  ]
}
```

Mouse Clicks The log message of `"type": "click"` describes a mouse click performed by the worker. The event is detected when the left or right mouse button is pressed. The event is detected after a fixed debounce time to prevent spamming too many log requests. The timestamps of the first and last clicks of each click sequence are stored. Furthermore, the `(x, y)` coordinates, the DOM element targeted and the number of clicks in the sequence are logged.

```
{
  "section": "string",
  "mouseButton": "right || left",
  "startTime": "string",
  "endTime": "string",
  "x": "integer",
  "y": "integer",
  "target": "string",
  "clicks": "integer"
}
```

Button Click The log message of `"type": "button"` describes a mouse click performed by the worker on a `button` DOM element. This event has a debounce time like the base click event, but different information is extracted. The information include the DOM button targeted, the timestamp of the click, and the `(x, y)` coordinates.

```
{
  "section": "string",
  "timestamp": "string",
  "button": "string",
  "x": "integer",
  "y": "integer"
}
```

Shortcuts The log message of `"type": "shortcut"` a keystroke combination used by the worker. Only combinations that involve the keys `CTRL` or `ALT` are monitored.

Key combinations corresponding to shortcuts are monitored. From the event key pressed for the shortcut are extracted:

- `"ctrl": "boolean"` set to `true` if the `CTRl` or `CMD` key is pressed.
- `"alt": "boolean"` set to `true` if the `ALT` key is pressed.
- `"key": "string"` contains the value of the key pressed within the shortcut detected.

```
{
  "section": "string",
  "timestamp": "string",
  "ctrl": "boolean",
  "alt": "boolean",
  "key": "string"
}
```

Keypress The log message of `"type": "keySequence"` describes a sequence of keystrokes performed by the worker. Every keypress is stored as a dictionary in a buffer array. Each dictionary contains the timestamp and the key pressed. The full sentence is reconstructed. The event handling completes after a dwell time of 1 second.

```
{
  "section": "string",
  "keySequence": [
    {
      "timeStamp": "string",
      "key": "string"
    },
```

```
    {
      ...
    }
    ...
  ],
  "sentence": "string"
}
```

Selection The log message of `"type": "selection"` describes a selection operation performed by the worker. The information logged includes the start and end timestamps and the content of the selection.

```
{
  "section": "string",
  "startTime": "string",
  "endTime": "string",
  "selected": "string"
}
```

Before Unload, Focus, and Blur The log message of `"type": "unload || window_focus || window_blur"` describes the last log requests produced when the worker closes the page of the task or performs or the window loses the focus or blurs. The information includes only the corresponding timestamp.

```
{
  "section": "string",
  "timestamp": "string"
}
```

Scroll The log message of `"type": "scroll"` describes a scroll operation performed by the worker. Similarly to mouse clicks and movements, a debounce time of 300ms is used to prevent spamming too many log requests. The information includes the start and end timestamps and the coordinates of the top left corner from which the scroll started. Scroll has a specific event listener and, like move movements, it needs a debouncing factor to prevent "spamming". For this event, the debounce time is set to 300 ms and the start timestamp, end timestamp, and (x, y) coordinates of the top left corner, are saved for logging.

```
{
  "section": "string",
  "startTimestamp": "string",
  "endTimestamp": "string",
  "x": "integer",
```

```
  "y": "integer"
}
```

Resize The log message of `"type": "resize"` is detected when the worker resizes the window of the task page. The information includes the start and end timestamp and the updated window size.

```
{
  "section": "string",
  "width": "integer",
  "height": "integer",
  "scrollWidth": "integer",
  "scrollHeight": "integer",
  "timestamp": "string"
}
```

Copy, Cut, and Paste The log message of `"type": "copy || cut || paste"` is detected when the worker cuts/copies and pastes some content during the task. The information includes the timestamp of the event. The attribute `target` refers to the DOM element targeted by a `copy` or `cut` event. The attribute is substituted with the `text` one when a `paste` event is detected and contains the text pasted.

```
{
  "section": "string",
  "timestamp": "string",
  "target": "string"
}
```

Text Input Backspace and Blur The log message of `"type": "text"` is detected when the backspace key is pressed inside a text input. The event is detected also then the input is blurred. The information includes the timestamp and the text contained in the input.

```
{
  "section": "string",
  "timestamp": "string",
  "text": "string"
}
```

Radio Group Input The event of `"type": "radioChange"` is detected when the workers chooses a new value within a radio button control. The information includes the timestamp and the new value of the radio button.

```
{
  "section": "string",
  "timestamp": "string",
  "group": "string",
  "value": "string"
}
```

Search Engine Queries and Results The event of `"type": "query || queryResults"` is detected when the worker queries the search engine and when results are retrieved. The information includes the text of the query in the former case, and the list of URL retrieved in the latter.

```
{
  "section": "string",
  "query": "string"
}
```

```
{
  "section": "string",
  "urlArray": [
    ...
  ]
}
```

1.6 System Usage

Section 1.6.1 describes the prerequisites and the initial setup needed to start using Crowd_Frame, while Section 1.6.2 provides the whole set of environment variables that can be used to customize the overall behavior. Then, Section 1.6.3 describes the output of a successful execution of the initialization script. Section 1.6.4 shows how to access the task configuration interface and Section 1.6.5 details the format required to provide the HITs of a task, together with a procedure to allocate them. Section 1.6.6 describes how to implement a custom quality check on the answers provided by the workers. Finally, Section 1.6.7 shows how to enable the local development modality.

1.6.1 Getting Started

The GitHub repository contains detailed and updated instructions.[38] Four (4) main prerequisites are required to start using Crowd_Frame. In more detail, these prerequisites are

[38] https://github.com/Miccighel/Crowd_Frame#readme

the AWS Command Line Interface[39] and distributions of Node.js[40] and Python.[41] Docker[42] may be optionally needed. Yarn[43] is used to manage the software dependencies. There are 15 steps to follow to successfully initialize Crowd_Frame and the overall infrastructure. The first 13 steps must be performed only once. Then, the task requester can vary the configuration and repeat the steps #14 and #15.

1. Create a new Amazon AWS account.
2. Create a new IAM User[44] using a custom name such as `your_iam_user`.
3. Attach the `AdministratorAccess` policy (cfr. Listing 1.15).
4. Generate a new access key pair.
5. Store the access key in the `credentials.json` file (cfr. Listing 1.16).
6. Clone the repository in the local filesystem.
7. Enable the Yarn global library using the command: `corepack enable`.
8. Move to the repository's folder: `cd ~/path/to/project`.
9. Install the dependencies using the command: `yarn install`.
10. Move to the data folder using the command: `cd data`.
11. Create the environment file `.env` at path `your_repo_folder/data/.env`.
12. Add to the environment file the subset of required variables (cfr. Listing 1.17).
13. Install the Python packages required (cfr. Listing 1.18)
14. Run the Python script `init.py`. The script will:
    - read the environment variables;
    - setup the whole AWS infrastructure;
    - generate a sample task configuration;
    - deploy the sample source files on the public bucket.
15. Visit the task deployed using the link in the format:
    - `https://your_deploy_bucket.s3.your_aws_region.amazonaws.com/your_task_name/your_batch_name/index.html`

[39] https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
[40] https://nodejs.org/it/download/
[41] https://www.python.org/downloads/
[42] https://docs.docker.com/get-docker/
[43] https://yarnpkg.com/
[44] https://docs.aws.amazon.com/IAM/latest/UserGuide/id_users.html

Crowd_Frame interacts with diverse Amazon Web Services to deploy crowdsourcing tasks, store the data produced and so on, as described in Section 1.5. Each service used falls within the AWS Free Tier[45] program. The task requester can set the budget limit using the `budget_limit` environment variable. Thus, the usage of the services will be blocked if/when such a limit is surpassed.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "*",
      "Resource": "*"
    }
  ]
}
```

Listing 1.15: Input data specification for a project on Toloka.

```
[your_iam_user]
region = your_region
aws_access_key_id=your_key
aws_secret_access_key=your_secret
```

Listing 1.16: Sample `credentials.json` file to store IAM user access key.

```
mail_contact=your_email_address
budget_limit=your_usd_budget_limit
task_name=your_task_name
batch_name=your_batch_name
admin_user=your_admin_username
admin_password=your_admin_password
server_config=none
aws_region=your_aws_region
aws_private_bucket=your_private_bucket_name
aws_deploy_bucket=your_deploy_bucket_name
```

Listing 1.17: Subset of environment variables required by Crowd_Frame.

[45] https://aws.amazon.com/free/

```
boto3==1.26.37
ipapi==1.0.4
ipinfo==4.4.2
mako==1.2.4
chardet==5.1.0
docker==6.0.1
python-dotenv==0.21.0
rich==12.6.0
tqdm==4.64.1
scipy==1.9.3
pycountry==22.3.5
numpy==1.24.1
pandas==1.5.2
toloka-kit==1.0.2
python-on-whales==0.55.0
beautifulsoup4==4.11.1
aiohttp==3.8.3
datefinder==0.7.3
```

Listing 1.18: Python packages required to initialize Crowd_Frame.

1.6.2 Environment Variables

Table 1.9 describes each environment variable that can be set in the environment file to customize Crowd_Frame behavior.

Table 1.9: Environment variables used to customize Crowd_Frame.

Variable	Description	Required	Value
`profile_name`	Name of the IAM profile created during step 2 (cfr. Section 1.6.1). If unspecified, the variable uses the value: `default`.	No	`your_iam_user`
`mail_contact`	Contact mail to receive AWS budgeting related comunications	Yes	Valid email address
`platform`	Platform used to publish the crowdsourcing task. Set it to `none` to recruit the workers manually.	Yes	`none`, `mturk`, `prolific` or `toloka`
`budget_limit`	Maximum monthly amount of money allowed to operate in USD$; e.g., 5.0.	Yes	Positive float number
`task_name`	Identifier of the crowdsourcing task.	Yes	Any string
`batch_name`	Identifier of the current batch.	Yes	Any string

Continues in the next page

Table 1.9: Environment variables used to customize (cont.) Crowd_Frame.

`batch_prefix`	Prefix of the identifiers of one or more task's batches. The variable can be used to filter the final result set.	No	Any string
`admin_user`	Username of the admin user. Allows unlocking the Generator.	Yes	Any string
`admin_password`	Password of the admin user. Allows unlocking the Generator.	Yes	Any string
`aws_region`	Region of the AWS account.	Yes	Valid AWS region identifier; e.g., `us-east-1`[46]
`aws_private_bucket`	Name of the private S3 bucket in which store task configuration and data	Yes	String unique across AWS account
`aws_deploy_bucket`	Name of the public S3 bucket in which deploy the source code	Yes	String unique across AWS account
`server_config`	Used to specify where the worker behavior logging interface is. Set it to `aws` to deploy the AWS-based infrastructure. Set it to `custom` to provide a custom logging endpoint. Set it to `none` if logging worker behavior is not needed.	Yes	`aws`, `custom` or `none`
`enable_crawling`	Enables the crawling of the results retrieved by the search engine.	No	`true` or `false`
`enable_solver`	Allows to deploy the HITs solver locally. Allows to provide a set of elements to automatically allocated into a set of HITs. Requires the usage of Docker. Warning: the feature is *experimental*.	No	`true` or `false`
`prolific_completion_code`	Prolific study completion code. Provide here the code if you recruit crowd workers via Prolific . Required if the platform chosen is `prolific`.	No	Valid Prolific completion code

Continues in the next page

[46] `https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.RegionsAndAvailabilityZones.html`

Table 1.9: Environment variables used to customize (cont.) Crowd_Frame.

`toloka_oauth_token`	Token to access Toloka 's API. Required if the platform chosen is `toloka`.	No	Valid Toloka OAuth token
`ip_info_token`	API Key to use `ipinfo.com` tracking functionalities.	No	Valid IP Info API key
`ip_geolocation_api_key`	API Key to use `ipinfo.com` tracking functionalities.	No	Valid IP Info API key
`ipapi_api_key`	API Key to use `ipgeolocation.info` tracking functionalities.	No	Valid IP Geolocation API key
`user_stack_token`	API Key to use `userstack.com` tracking functionalities.	No	Valid Userstack API key
`bing_api_key`	API Key to use `BingWebSearch` search provider.	No	Valid Bing API Web Search API key
`fake_json_token`	API Key to use `FakerWebSearch` search provider. Returns dummy responses useful to test the search engine.	No	Valid `fakeJSON.com` API key

1.6.3 Build Output

Each execution of the initialization script of Crowd_Frame described in Section 1.6.1 populates a build folder on the local filesystem, at path: `cd ~/path/to/project/build/`. The folder may contain up to 6 subfolders, depending on the crowdsourcing platforms used. Table 1.10 provides a general description of the content of each subfolder.

Table 1.10: Folder structure of the output of a Crowd_Frame build.

Sub Folder	**Description**
`build/task/`	Contains the configuration of the task to deploy.
`build/config/`	Contains the encrypted credentials used to unlock the `Generator` component.
`build/environments/`	Contains the development and production environments.
`build/mturk/`	Contains three files needed to publish the task using Amazon Mechanical Turk.
`build/toloka/`	Contains six files needed to publish the task using Toloka.
`build/skeleton/`	Contains a interface between the HITs and the application and a file used to implement quality checks.
`build/deploy/`	Contains the source files of the task to deploy.

1.6.3.1 `build/task/`

The folder contains the 7 configuration files of the task deployed. The Generator component described in Section 1.5.2 populates one of these files along each step. In other words, each task deployed using Crowd_Frame is configured by using 8 special JSON files. Table 1.11 provides a general description of the content of each configuration file.

Table 1.11: Configuration files of a task deployed using Crowd_Frame.

File	Description
`hits.json`	Contains the whole set of HITs of the task.
`questionnaires.json`	Contains the definition of each questionnare of the task.
`dimensions.json`	Contains the definitions of each evaluation dimension of the task.
`instructions_general.json`	Contains the general instructions of the task.
`instructions_evaluation.json`	Contains the evaluation instructions of the task.
`search_engine.json`	Contains the configuration of the custom search engine.
`task.json`	Contains several general settings of the task.
`workers.json`	Contains settings concerning worker access to the task.

1.6.3.2 `build/environments/`

The folder contains the development and production environments of Crowd_Frame. Each environment contains the values of the variables shown in Table 1.9 along with additional data. The environments are overwritten during each execution of the initialization script to reflect changes in the environment variables.

1.6.3.3 `build/config/`

The folder contains the encrypted credentials used to unlock access to the Generator component. There is a single file named `admin.json` that contains a hash generated using the HMAC [4] scheme. The hash is built using the values of the `admin_user` and `admin_password` variables shown in Table 1.9.

1.6.3.4 `build/skeleton/`

The folder contains an interface called `Document` in a file named `document.ts`. Such an interface is a bridge between the Angular application and the HITs configured. The interface is generated during the execution of the initialization script if the HITs' attributes change. The folder contains also a file named `goldChecker.ts`. It provides a static method that a developer can implement to perform a custom quality check when enabled for one or more evaluation dimensions.

1.6.3.5 `build/deploy/`

The folder contains the three source files built by Angular that the initialization script deploys on the public S3 bucket. Table 1.12 describes each of these source files. The role of the public bucket is simply making them available publicly on the Internet. The whole client-side code base is included. This means that a developer can deploy the files on a private server. The task will be initialized without issues.

Table 1.12: Source files of a task deployed using Crowd_Frame.

File	Description
`index.html`	Markup of the task deployed.
`styles.css`	Styling of the task deployed.
`scripts.js`	Client side code of the task deployed.

1.6.3.6 `build/mturk/`

The folder contains the wrapper initialized to be deployed on Amazon Mechanical Turk in a file named `index.html`. The wrapper is initialized for the current task starting from a general model contained in the file named `model.html`. The initialization script uses a template engine called `Mako`[47] to initialize the wrapper. The folder contains also a file named `tokens.csv`. It contains the input and output tokens to be provided to the platform. Such tokens are those present in the file containing the definition of the whole set of HITs.

1.6.3.7 `build/toloka/`

The folder contains the wrapper initialized to be deployed on Toloka, the input and output data specification and the tokens to be provided. Table 1.13 describes each file. It is generated using Mako from a model as for Amazon Mechanical Turk. However, the final result is split into three different files, due to Toloka's requester interface. The files `interface.html`, `interface.css`, and `interface.js` contain respectively the markup, the styling and the client-side code of the wrapper. The two JSON files provide the input and output data specification to be used for the task to be deployed. The TSV file contains the input and output tokens to be provided to the platform.

1.6.4 Task Configuration

The Generator component must be accessed to configure the crowdsourcing task deployed. This involves 4 steps:

1. Open the administrator panel by appending the suffix `?admin=true` to the task's URL (cfr. step #15, Section 1.6.1).
2. Click the `Generate` button to open the login form.

[47] `https://www.makotemplates.org/`

Table 1.13: Content of the build folder to deploy a task on Toloka .

File	Description
`index.html`	Markup of the wrapper.
`styles.css`	Styling of the wrapper.
`scripts.js`	Client side code of the wrapper.
`input_specification.json`	Input data specification.
`output_specification.json`	Output data specification.
`tokens.tsv`	Tokens to be provided to the platform.

3. Input the admin credentials set in the corresponding environment variables (cfr. Table 1.9).
4. Proceed through each configuration step and upload the final configuration.

The final configuration can be uploaded using the `Upload` button. Table 1.1 provides a summary of each task configuration step. The initialization script synchronizes the local configuration and the remote one bidirectionally by downloading (uploading) the most recent one.

1.6.5 HITs Allocation

The HITs for a crowdsourcing task designed and deployed using Crowd_Frame must be stored in a special JSON file. Such a file can be manually uploaded when configuring the crowdsourcing task itself, as described in Section 1.5.2. The file must comply with a special format that satisfies 5 requirements:

1. There must be an array of HITs (also called *units*).
2. Each HIT must have a unique input/output token attribute pair.
3. The number of elements to assess must be specified for each HIT.
4. Each element must have an attribute named `id`.
5. Each element can have an arbitrary number of attributes.

Listing 1.19 shows a valid set composed of a single HIT to be assessed within a crowdsourcing task configured using Crowd_Frame.

1.6.5.1 Manual Approach

The requester can build manually the set of HITs compliant with the format required by Crowd_Frame. Initially, the requester chooses an attribute whose values split the dataset into different classes. The core idea is to build pools of elements to allocate, one for each class. Four parameters are thus established. These parameters are the total number of elements to allocate in the whole set of HITs, the number of elements that each HIT must

```
[
  {
    "unit_id": "unit_0",
    "token_input": "ABCDEFGHILM",
    "token_output": "MNOPQRSTUVZ",
    "documents_number": 1,
    "documents": [
      {
        "id": "identifier_1",
        "text": "Lorem ipsum dolor sit amet"
      }
    ]
  }
]
```

Listing 1.19: Valid set of one HIT for a task designed and deployed using Crowd_Frame.

contain, the number of elements to allocate for each class, and the number of repetitions for each element. The pools of elements must thus be updated to include all the repetitions required. Each HIT is then built using a loop. It is useful to define a support function. The core idea is to sample the required number of elements for each class until a sample without duplicates is obtained. The elements are then removed from the pool of those still available if the condition is satisfied. The total number of HITs required depends on the parameters previously established. The lists of elements allocated in HITs can be serialized for later reference. Using such an allocation matrix, the requester can finally build the set of HITs in the format required.

Algorithm 1.1 provides a pseudocode that further details the allocation procedure. Let us hypothesize a requester that wants to determine the number m of HITs that they will publish on the crowdsourcing platform. Algorithm 1.2 details the SINGLEHIT(...) subprocedure used by the main algorithm to sample a set of elements without duplicates. The sample obtained is used to build a single HIT of the whole set. Let us hypothesize a requester that wants to allocate n elements in HITs made of k positions. Each element is repeated in p different HITs. The final number of HITs required is $m = (n * p)/k$.

1.6.5.2 Automatic Approach

Crowd_Frame provides a solution to allocate automatically the elements to evaluate in a set of HITs. It is experimental and works only when using Crowd_Frame within the local filesystem. Future versions of the software will consolidate and generalize such a feature. Ceschia et al. [11] optimize HITs construction by providing its formal definition and by applying a local search method to solve the formalized problem. Crowd_Frame allows deploying an implementation of their solver and provides a way to communicate with it. Docker needs to be installed in the local system since the usage of a container is required to allow software and solver to communicate. The container contains deployed using Docker contains two services. One of these services provides the implementation of the solver it-

Algorithm 1.1 Procedure to allocate a dataset into HITs using the format required

```
elementsFiltered ← filterElements(attribute, valuesChosen)
classes ← valuesChosen
pools ← List()
forEach class ∈ classes do
    elementsClass ← findElements(elementsFiltered, class)
    pool ← unique(elementsClass)
    pools.Append(pool)
end for
totalElements ← len(elementsFiltered)
classElementsNumber ← len(classes)
hitElementsNumber ← k
repetitionsElement ← p
forEach pool ∈ pools do
    pool ← extendPool(repetitionsElement)
end for
poolsDict ← mergePools(pools, classes)
hits ← List()
forEach index ∈ range((totalElements * repetitionsElement)/hitElementsNumber) do
    hitSample ← singleHit(poolsMerged)
    hitSample ← shuffle(hitSample)
    hits.append(hitSample)
end for
hits.serialize(pathAssignments)
hitsFinal ← List()
forEach hit ∈ hits do
    index ← index(hit)
    unitId ← concat("unit_", index)
    tokenInput ← randomString(11)
    tokenOutput ← randomString(11)
    hitObject ← BuildJSON(unitId, tokenInput, tokenOutput, hitElementsNumber)
    forEach indexElem ∈ range(hitElementsNumber) do
        hitObject["documents"] ← hits[indexElem]
    end for
    hitsFinal.append(hitObject)
end for
hitsFinal.serialize(pathHits)
```

Algorithm 1.2 Procedure to sample elements without duplicates for a single HIT

```
1: containsDuplicates ← True
2: while containsDuplicates do
3:     sample ← LIST()
4:     forEach class ∈ classes do
5:         forEach indexClass ∈ RANGE(classElementsNumber) do
6:             element ← RANDOM(poolsDict[class])
7:             sample.APPEND(element)
8:         end for
9:     end for
10:    if CHECKDUPLICATES(sample)==False then
11:        containsDuplicates ← False
12:    end if
13: end while
14: forEach s ∈ sample do
15:    forEach c ∈ classes do
16:        if s ∈ pool[c] then
17:            pool[c].REMOVE(s)
18:        end if
19:    end for
20: end for
21: return sample
```

self, while the other provides a reverse proxy based on the Nginx[48] web server. The reverse proxy forwards HTTP messages to the solver. The solver processes the messages and responds. Figure 1.28 shows a deployment diagram which describes the interaction between Crowd_Frame, solver, and reverse proxy.

The requester can enable the feature using the `enable_solver` environment variable shown in Table 1.9. They can take advantage of the solver while configuring the task using the Generator component, during the sixth step of the configuration (cfr. Table 1.1). The first step required to create the input data required by the solver involves uploading the elements to be allocated into a set of HITs. Each element must share the same set of attributes and the overall set must be provided in the form of a JSON array. In other words, the requester can upload the value of the `documents` object shown in Listing 1.19, without writing any token or `unit_id`. Then, the requester can configure three parameters concerning the allocation. They thus configures the number of workers that evaluate each element and the overall number of workers among which the elements must be allocated. Lastly, the requester chooses the subset of attributes used to categorize the elements across different HITs. The requester must also indicate how many elements must be assigned to each worker for every possible value of the category chosen. For each category/element number pair the system verifies whether the two values are compatible. The minimum number of workers needed to evaluate the whole set of HITs is thus computed if the verification is successful. The requester can increase such a number as they prefers.

To provide an example of when such a verification can fail, let us hypothesize a requester

[48] `https://www.nginx.com/`

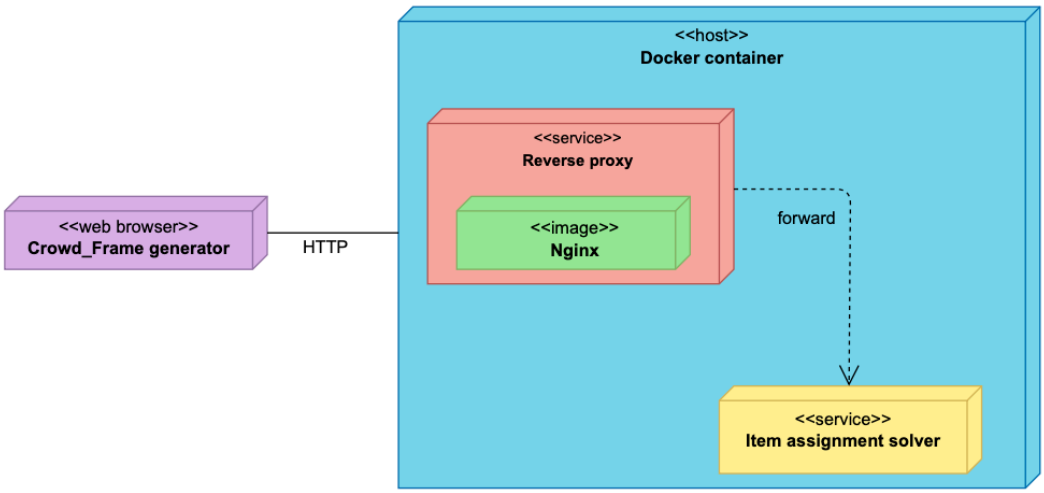

Figure 1.28: Deploy diagram of the infrastructure used to allow Crowd_Frame and the solver communicating.

who chooses as category an attribute named `A1` which has 2 different values among the elements to be evaluated. The requester requires that each worker evaluates 2 elements for each attribute's value. Then, a second attribute named `A2` is also chosen, which has 3 different values. The requester requires that each worker evaluates 1 element for each attribute's value. This means that according to the attribute `A1` each worker evaluates 4 elements, while according to `A2` each worker evaluates 3 elements. Such a selection of values is not allowed. Figure 1.29 shows a sample configuration for the subset of 120 statements sampled from PolitiFact [67] by Roitero et al. [57] and evaluated in their task. Listing 1.20 shows a fragment of such elements. The requester, hence, uploads a JSON file containing 120 elements to allocate. They chooses that each element must be assigned to 10 different workers. The attribute `party` is selected as a category. Each worker must evaluate 6 elements for each of the 2 values of the category. In other words, each worker must evaluate 12 different elements. The verification steps thus enforce a minimum number of 100 workers to recruit. The `Generator` allows selecting as categories only the attributes which are balanced with respect to the number of documents. In other words, those attributes are repeated across the same number of elements. Such a design choice is needed to provide input data to the solver compliant with the formalization of Ceschia et al. [11]. The request is now ready to send the request to the solver, which computes the allocation and returns a solution. Listing 1.21 shows an example of the final allocation. The solution returned is then used by Crowd_Frame to build the corresponding set of HITs compliant with the format shown in Listing 1.19.

1.6.6 Quality Checks

Crowd_Frame provides a way to manually define custom quality checks triggered for each evaluation dimension when the corresponding setting is enabled in the configuration. A custom quality check is obtained by providing an implementation for the static method

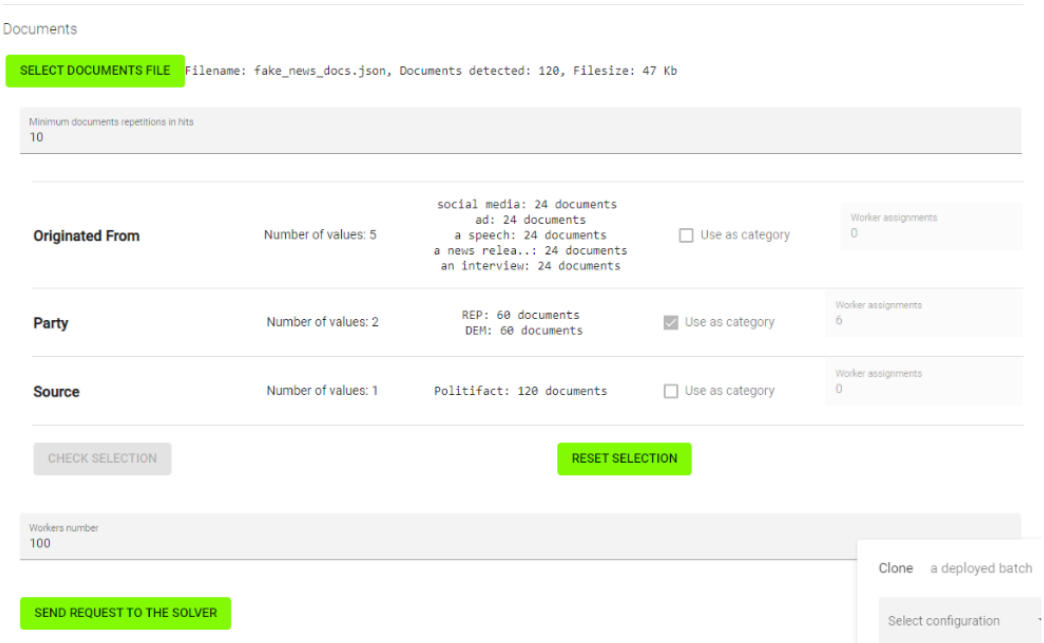

Figure 1.29: Sample configuration for the solver used to automatically allocate elements to be evaluated in a set of HITs.

```
[
  {
    "name": "REP_HALFTRUE_doc5",
    "statement": "Houston now has more debt per capita than California.",
    "claimant": "Rick Perry",
    "date": "2010",
    "originated_from": "ad",
    "id_par": "1796.json",
    "job": "Governor",
    "party": "REP",
    "source": "Politifact"
  },
  ...
]
```

Listing 1.20: Fragment of elements to evaluated in the task published by Roitero et al. [57].

```
{
  "finished": true,
  "runner": "BSA",
  "solution": {
    "Instance_id": "1658421484781",
    "Used_workers": 100,
    "Workers": [
      {
        "Assignments": [
          "REP_FALSE_doc8", "DEM_TRUE_doc5", "REP_BARELYTRUE_doc6",
          "DEM_MOSTLYTRUE_doc5", "REP_TRUE_doc7", "DEM_LIE_doc9"
        ], "Id": "W0"
      },
      {
        "Assignments": [
          "REP_TRUE_doc5", "DEM_TRUE_doc7", "REP_LIE_doc1",
          "DEM_TRUE_doc1", "REP_HALFTRUE_doc2", "DEM_FALSE_doc6"
        ], "Id": "W1"
      },
      ...
    ]
  },
  "completed": "2022-07-21 16:37:56",
  "started": "2022-07-21 16:37:56",
  "submitted": "2022-07-21 16:37:56",
  "task_id": "5447375273499815497"
}
```


Listing 1.21: Fragment of the allocation built automatically using the solver integrated with Crowd_Frame.

`performGoldCheck`, as described in Section 1.6.3. A custom quality check is triggered only for certain elements of a HIT, with respect to a subset of the evaluation dimensions. An element can be marked for the quality check by prepending the string `GOLD` to its `id` attribute.

Listing 1.22 shows a single HIT where its second element is marked for the quality check. Listing 1.23 shows the default implementation of the method generated by the initialization script. The `document` array provides the set of elements marked for the quality check. The `answers` array contains the answers provided by the worker for the evaluation dimensions that require the quality check. The check must be implemented among the two comments shown.

```
[
  {
    "unit_id": "unit_0",
    "token_input": "ABCDEFGHILM",
    "token_output": "MNOPQRSTUVZ",
    "documents_number": 1,
    "documents": [
      { "id": "identifier_1", "text": "Lorem ipsum dolor sit amet" },
      { "id": "GOLD-identifier", "text": "Lorem ipsum dolor sit amet" }
    ]
  }
]
```

Listing 1.22: Valid set of one HIT with two elements, where one is used within a custom quality check.

1.6.7 Local Development

Crowd_Frame provides a way to manually edit and test the configuration locally, without deploying the overall infrastructure. Enabling the local development capability involves 4 steps:

1. Move to the environments folder (Section 1.6.3):
   - `cd your_repo_folder/data/build/environments/`.
2. Open the development environment file:
   - `environment.ts`
3. Edit the variable `configuration_local` by setting the value `true`.
4. Run the command `ng serve`.

The requester may also skip the first three steps and simply run the `ng serve` command. In such a case, the source code can be modified locally while using the remote configuration to initialize the application. Listing 1.24 shows a valid development environment file that

```
export class GoldChecker {
  static performGoldCheck(goldConfiguration : Array<Object>) {
    let goldChecks = new Array<boolean>()
    /* If there are no gold elements there is nothing to be checked */
    if(goldConfiguration.length<=0) {
      goldChecks.push(true)
      return goldChecks
    }
    for (let goldElement of goldConfiguration) {
      /* Element attributes */
      let document = goldElement["document"]
      /* Worker's answers for each gold dimensions */
      let answers = goldElement["answers"]
      /* Worker's notes*/
      let notes = goldElement["notes"]
      let goldCheck = true
      /* CONTROL IMPLEMENTATION STARTS HERE */
      /* The check for the current element holds if goldCheck remains
       ↪  true */
      ...
      /* CONTROL IMPLEMENTATION ENDS HERE */
      /* Push goldCheck inside goldChecks array for the current gold
       ↪  element */
      goldChecks.push(goldCheck)
   }
   return goldChecks
  }
}

```

Listing 1.23: Default implementation of the static method that performs custom quality checks in Crowd_Frame.

allows testing the task configuration locally. It must be noted that every execution of the `init.py` script overwrites the environment files. Thus, the local testing capability must be enabled again if the requester deploys the task by running the script itself.

```
export const environment = {
    production: false,
    configuration_local: true,
    platform: "mturk",
    taskName: "your_task_name",
    batchName: "your_batch_name",
    region: "your_aws_region",
    bucket: "your_private_bucket",
    aws_id_key: "your_aws_key_id",
    aws_secret_key: "your_aws_key_secret",
    prolific_completion_code: false,
    bing_api_key: "your_bing_api_key",
    fake_json_key: "your_fake_json_key",
    log_on_console: false,
    log_server_config: "none",
    table_acl_name: "Crowd_Frame-your_task_name_your_batch_name_ACL",
    table_data_name: "Crowd_Frame-your_task_name_your_batch_name_Data",
    table_log_name: "Crowd_Frame-your_task_name_your_batch_name_Logger",
    hit_solver_endpoint: "None",
};
```

Listing 1.24: Sample development environment of Crowd_Frame.

1.7 Task Performing

Section 1.7.1 provides some initial considerations about worker recruitment to perform a crowdsourcing task deployed using Crowd_Frame. Section 1.7.2 how to recruit each worker manually. Then, Section 1.7.3 explains how to recruit the workers from Amazon Mechanical Turk, while Section 1.7.4 addresses Toloka. Finally, Section 1.7.5 focuses on Prolific.

1.7.1 Getting Started

Publishing a crowdsourcing task configured using Crowd_Frame involves choosing the platform to recruit the human workforce, even though the requester can also manually recruit each worker needed. The process to publish and start the task deployed is slightly different depending on such a choice.

1.7.2 Manual Recruitment

A task requester that aims to manually recruit each worker to perform the task deployed must:

1. Set the environment variable `platform` (cfr. Table 1.9) using the value `none`.
2. Generate ad assign each worker an alphanumeric identifier, such as `randomWorkerId`.
3. Append the identifier generated as a GET parameter to the task deploy link:
   - `?workerId=randomWorkerId`
4. Provide each worker with the link to the task deployed.
   - `https://your_deploy_bucket.s3.your_aws_region.amazonaws.com/your_task_name/your_batch_name/index.html?workerID=randomWorkerId`
5. Wait for task completion.

Steps #2 and #3 can be skipped because the task URL can be provided to a worker also without manually adding an identifier. In such a case, Crowd_Frame will automatically generate it.

1.7.3 Amazon Mechanical Turk

A task requester that aims to recruit each worker using Amazon Mechanical Turk must:

1. Create the task and set its general parameters and criterion (cfr. Figure 1.1).
2. Move to the build output folder for the platform:
   - `data/build/mturk/`
3. Copy the code of the wrapper:
   - `data/build/mturk/index.html`
4. Paste everything into the `Design Layout` box.
5. Preview and save the task project.
6. Publish the task and recruit a batch of workers by uploading the file containing the input/output tokens:
   - `data/build/mturk/tokens.csv`
7. Review the status of each submission by using the `Manage` tab.

1.7.4 Toloka

A task requester that aims to recruit each worker using Toloka must:

1. Create the project and set its general parameters (cfr. Figure 1.3).
2. Move to the build output folder for the platform:
   - `data/build/toloka/`

3. Copy the markup, JavaScript code, and CSS styles of the wrapper:
   - `data/build/toloka/interface.html`
   - `data/build/toloka/interface.js`
   - `data/build/toloka/interface.css`
4. Paste each source code into the `Task Interface` box, using the corresponding section of the `HTML/JS/ CSS` (cfr. Figure 1.4).
5. Copy the input and output data specification (cfr. Listing 1.3 and Listing 1.4):
   - `data/build/toloka/input_specification.json`
   - `data/build/toloka/output_specification.json`
6. Paste each data specification into the `Data Specification` box, using the corresponding sections (cfr. Figure 1.5).
7. Copy the text of the task general instructions:
   - `data/build/task/instructions_general.json`
8. Paste the texts into the `Instructions for Tolokers` box, using the source code edit modality.
9. Create a new pool of workers by defining the parameters of the audience and the reward (cfr. Figure 1.6).
10. Publish the task and recruit the audience of workers for each pool by uploading the file containing the input/output tokens (cfr. Listing 1.5).
    - `data/build/mturk/tokens.tsv`
11. Review the status of each submission by using the each pool's page.

1.7.5 Prolific

A task requester that aims to recruit each worker using Prolific must:

1. Create the study and set its general parameters (cfr. Figure 1.7).
2. Set the data collection modality required (cfr. Figure 1.8):
   - Choose `External study link` as the modality to collect data.
   - Provide the URL of the task deployed.
   - Choose using the URL parameters to record Prolific IDs.
   - Rename the `PROLIFIC_PID` parameter to `workerId`.
   - Choose to redirect the participants to confirm completion using a URL.
   - Copy the completion code from the URL shown (i.e., the `cc` parameter).
   - Set the `prolific_completion_code` environment variable using the completion code found as value.

3. Configure the parameters and criterion of the audience of workers to recruit (cfr. Figure 1.9).
4. Set the overall study cost (cfr. Figure 1.10).
5. Review the status of each submission by using the study's page.

1.8 Task Results

Section 1.8.1 describes the prerequisites and the initial setup needed to download the results of a task deployed using Crowd_Frame. Section 1.8.2 describes the content of the `Task` folder, while Section 1.8.3 describes the `Data` one. Then, Section 1.8.4 focuses on the `Resources` folder and Section 1.8.5 on the `Crawling` one. Finally, Section 1.8.6 describes the `Dataframe` folder.

1.8.1 Getting Started

The requester can download the final results of a crowdsourcing task deployed using Crowd_Frame by using the download script. This involves four steps:

1. Access the main folder: `cd ~/path/to/project/`.
2. Access the data folder: `cd data`.
3. Run the `download.py` script. The script will:
   - Download and store snapshots of the raw data produced by each worker.
   - Refine the raw data using a tabular format.
   - Download and store the configuration of the task deployed.
   - Build and store support files containing worker and user agent attributes.

The whole set of output data is stored in the results folder: `data/result/task_name/`, where `task_name` is the value of the corresponding environment variable shown in Table 1.9. The folder is created by the download script if it does not exist. It contains 5 subfolders, one for each type of output data. Table 1.14 describes each of these subfolders.

Table 1.14: Structure of the results folder of Crowd_Frame.

Folder	Description
`Data`	Contains snapshots of the raw data produced by each worker.
`Dataframe`	Contains tabular based refined versions of the raw data.
`Resources`	Contains two support files for each worker with attribute about him/herself and the user agent.
`Task`	Contains a backup of the task's configuration.
`Crawling`	Contains a crawl of the pages retrieved while using the search engine.

1.8.2 result/Task/

The Task folder contains the backup of the task configuration. Table 1.11 (Section 1.6.3) describes its content.

1.8.3 result/Data/

The Data folder contains a snapshot of the data produced by each worker. A snapshot is a JSON dictionary. The top-level object is an array. The download script creates an object for each batch of workers recruited within a crowdsourcing task. Listing 1.25 shows the snapshot for a worker with identifier ABEFLAGYVQ7IN4 who participates in the batch Your_Batch of the task Your_Task. This means that the snapshot contains an array with a single object. The source_* attributes represent the DynamoDB tables and the path on the local filesystem.

```
[
  {
    "source_data": "Crowd_Frame-Your-Task_Your-Batch_Data",
    "source_acl": "Crowd_Frame-Your-Task_Your-Batch_ACL",
    "source_log": "Crowd_Frame-Your-Task_Your-Batch_Logger",
    "source_path": "result/Your_Task/Data/ABEFLAGYVQ7IN4.json",
    "data_items": 1,
    "task": {...},
    "worker": {...},
    "ip": {...},
    "uag": {...},
    "checks": [...],
    "questionnaires_answers": [...],
    "documents_answers": [...]
    "comments": [...],
    "logs": [],
    "questionnaires": {...},
    "documents": {...},
    "dimensions": {...}
  }
]
```

Listing 1.25: Snapshot of a worker who participates in a task with a single batch deployed using Crowd_Frame.

1.8.4 result/Resources/

The Resources folder contains two JSON files for each worker. Let us hypothesize a worker recruited using the identifier ABEFLAGYVQ7IN4. The two support files are named ABEFLAGYVQ7IN4_ip.json and ABEFLAGYVQ7IN4_uag.json. The former contains attributes

obtained by performing the reverse lookup of the worker's IP addresses. The latter contains attributes obtained by analyzing the user agent strings. Since a worker could access a task from multiple locations and using multiple devices, the two files can contain multiple IP addresses and user agent strings. Listing 1.26 and Listing 1.27 show a subset of the information provided by the two support files.

1.8.5 `result/Crawling/`

The `Crawling` folder contains a crawl of the web pages retrieved by the search engine when queried by a worker. A task requester who deploys a crowdsourcing task which uses the search engine within one or more evaluation dimensions can choose to enable the crawling by using the `enable_crawling` shown in Table 1.9. The download script thus tries to crawl each web page if the variable is enabled. Initially, the download script creates two subfolders, namely `Metadata/` and `Source/`. Each web page is then assigned an UUID (Universally Unique IDentifier). Let us hypothesize a page assigned with the UUID `59c0f70f-c5a6-45ec-ac90-b609e2cc66d7`, The script tries to download its source code. It is stored in the `Source` folder if the operation succeeds, in a file named `59c0f70f-c5a6-45ec-ac90-b609e2cc66d7_s` The extension depends on the page's source code. Then, the script stores some metadata about the crawling operation of the page in the `Metadata` folder, in a file named `59c0f70f-c5a6-45ec-ac90-b609e2c` It is possible to understand whether the operation succeeded or not and why (i.e., by acknowledging the HTTP response code) and to read the value of each HTTP header. Listing 1.28 show an example of metadata produced by the download script while trying to crawl one of the pages retrieved.

1.8.6 `result/Dataframe/`

The `Dataframe` folder contains a refined version of the data stored within each worker snapshot. The download script inserts the raw data into structures called "DataFrame". A `DataFrame`[49] is a two-dimensional data structure with labelled axes that contains heterogeneous data. Such structures may thus be implemented as two-dimensional arrays or tables with rows and columns. The download script refines the raw data into up to 10 tabular dataframe serialized into CSV files. The final amount of dataframes serialized in the `Dataframe` folder depends on the environment variables configured by the task requester.

Each `DataFrame` has a variable number of rows and columns. Part of them provides general information, thus being composed of one row for each worker, such as `workers_info` and `workers_acl`. The remaining ones have higher granularity. For instance, a row of the `workers_url` dataframe contains a row for each result retrieved for each query submitted to the search engine while analyzing a single HIT's element during a given try by a single worker. A row of the `workers_answers` dataframe contains the answers for the evaluation dimensions provided for a single HIT's element during a given try by a single worker, and so on. The requester must thus be careful while exploring each `DataFrame` and properly understand what kind of data they are exploring and analyzing. Listing 1.29 provides an example of the access control list for a task with a single worker recruited. Listing 1.30 pro-

[49] `https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html`

```
{
  "<ip_address_1>": {
      "continent_code": "NA",
      "continent_name": "North America",
      "country_capital": "Washington D.C.",
      "country_code_iso2": "US",
      "country_code_iso3": "USA",
      "country_currency_code_iso3": "USD",
      "country_currency_name": "US Dollar",
      "country_currency_numeric": "840",
      "country_currency_symbol": "\$",
      "country_flag_emoji": "...",
      "country_flag_emoji_unicode": "...",
      "country_flag_url": "...",
      "country_is_eu": false,
      "country_name": "United States",
      "country_name_official": "United States of America",
      "country_numeric": "840",
      "hostname": "...",
      "ip": "...",
      "ip_address_type": "...",
      "latitude": "...",
      "location_calling_code": "...",
      "location_coordinates": "...",
      "location_geoname_id": "...",
      "location_is_eu": false,
      "location_languages": [...],
      "location_name": "...",
      "location_postal_code": "...",
      "longitude": "...",
      "provider_name": "...",
      "region_code": "...",
      "region_code_full": "...",
      "region_name": "Louisiana",
      "region_type": "State",
      "timezone_name": "America/Chicago",
      ...
  },
  ...
```

Listing 1.26: Subset of the information obtained by perform the reverse lookup of the IP address of a worker.

```
{
  "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
  ↪ like Gecko) Chrome/105.0.5195.102 Safari/537.36": {
      "browser_engine": "WebKit/Blink",
      "browser_name": "Chrome",
      "browser_version": "105.0.5195.102",
      "browser_version_major": "105",
      "device_is_crawler": false,
      "device_is_mobile_device": false,
      "device_type": "desktop",
      "os_code": "windows_10",
      "os_family": "Windows",
      "os_family_code": "windows",
      "os_family_vendor": "Microsoft Corporation.",
      "os_icon": "https://assets.userstack.com/icon/os/windows10.png",
      "os_icon_large": "...",
      "os_name": "Windows 10",
      "os_url": "https://en.wikipedia.org/wiki/Windows_10",
      "ua": "...",
      "ua_type": "browser",
      "ua_url": "https://about.google/",
      ...
  },
  ...
}
```


Listing 1.27: Subset of the information obtained by analyzing the user agent string of a worker.

Table 1.15: DataFrame produced when downloading the final results of a task.

DataFrame	Description
`workers_acl.csv`	Access control list of the workers.
`workers_ip_addresses.csv`	Data concerning the IP addresses of the workers.
`workers_user_agents.csv`	Data concerning the User Agent of the workers.
`workers_answers.csv`	Answers provided for each evaluation dimension by workers.
`workers_questionnaire.csv`	Answers provided for each questionnaire by workers.
`workers_dimensions_selection.csv`	Temporal order along with each worker chooses a value for each evaluation dimension.
`workers_notes.csv`	Textual annotations provided by workers.
`workers_urls.csv`	Queries to the search engine provided by workers along with results retrieved.
`workers_crawling.csv`	Data concerning the crawling of web pages retrieved by the search engine.
`workers_logs.csv`	Log data produced by the `Logger` component while the workers perform the task.
`workers_comments.csv`	Final comments provided by workers to the requester at the end of the task.
`workers_mturk_data.csv`	Data concerning workers produced by Amazon Mechanical Turk.
`workers_toloka_data.csv`	Data concerning workers produced by Toloka.
`workers_prolific_study_data.csv`	Data concerning the study deployed on Prolific and its submissions.
`workers_prolific_demographic_data.csv`	Data concerning the demographics of the workers who participate in a study published on Prolific.

```
{
  "attributes": {
    "response_uuid": "59c0f70f-c5a6-45ec-ac90-b609e2cc66d7",
    "response_url": "...",
    "response_timestamp": "...",
    "response_error_code": null,
    "response_source_path": "...",
    "response_metadata_path": "...",
    "response_status_code": 200,
    "response_encoding": "utf-8",
    "response_content_length": 125965,
    "response_content_type": "text/html; charset=utf-8"
  },
  "data": {
    "date": "Wed, 08 Jun 2022 22:33:12 GMT",
    "content_type": "text/html; charset=utf-8",
    "content_length": "125965",
    "..."
  }
}
```


Listing 1.28: Metadata produced by the download script while trying to crawl a webpage retrieved by the search engine of Crowd_Frame.

vide an example composed of the answers provided by a single worker for two elements of the HIT assigned.

Each dataframe has its own characteristics and peculiarities. However, there are several rules of thumb that a requester should remember and eventually consider while they perform some kind of analysis:

- The attribute `paid` is present in the whole set of dataframes. It can be used to split the data among the workers who completed or did not the task. The requester may want to explore the data of who failed the task.
- The attribute `batch_name` is present in a subset of dataframes. It can be used to split the data among the different batches of workers recruited. The requester may want to analyze separately each subset of data.
- The attributes `try_current` and `try_last` are present in a subset of dataframes. They can be used to split the data among each try performed by each worker. The latter attribute indicates the most recent try. The requester should not forget the possible presence of multiple tries for each worker while analyzing the data.
- The attribute `access_counter` is present in a subset of dataframes. It can be used to split the data among subsequent accesses performed by each worker. The requester should not forget that a worker is allowed to close the browser's tab and come back later. Moreover, a given access registered in the dataframe can span across multiple tries, if the workers perform an additional try after failing the previous one without

```
worker_id,generated,in_progress,paid,platform,task_name,batch_name,unit_id,token_input,token_output,try_current,try_last,try_left,tries_amount,status_code,access_counter,time_arrival,time_arrival_parsed,time_submit,time_submit_parsed,time_completion,time_completion_parsed,time_expiration_nearest,time_expiration_nearest_parsed,time_expiration,time_expiration_parsed,time_expired,time_removal,time_removal_parsed,questionnaire_amount,questionnaire_amount_start,questionnaire_amount_end,dimensions_amount,documents_amount,ip_address,ip_source,user_agent,user_agent_source,folder,source_path,source_acl,source_data,source_log,study_id,session_id,n_submissions
ABEFLAGYVQ7IN4,False,False,False,mturk,Task-Sample,Batch-Sample,unit_0,KDSCKUOHINM,VQULVJHRTOZ,1,1,10,10,,1,"Thu, 14 Apr 2022 12:37:47 GMT",2022-04-14 12:37:47 00:00,"Thu, 14 Apr 2022 12:38:44 GMT",2022-04-14 12:38:44 00:00,,,,,,,True,"Thu, 14 Apr 2022 15:32:39 GMT",2022-04-14 15:32:39 00:00,3,3,0,0,0,75.41.166.226,cf,"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.75 Safari/537.36",cf,Task-Sample/Batch-Sample/Data/A3CGQOJC28OVGN/,result/Task-Sample/Data/A3CGQOJC28OVGN.json,Crowd_Frame-Task-Sample_Batch-Sample_ACL,Crowd_Frame-Task-Sample_Batch-Sample_Data,...,,,
```

Listing 1.29: Example of the `workers_acl` dataframe produced by Crowd_Frame.

coming back later.

- The attribute `action` is present in a subset of dataframe dataframe. It can be used to understand whether the worker proceeded to the previous/following HIT's element. The possible values are `Back`, `Next` and `Finish`. The `Finish` value indicates the last element evaluated before completing a given try. The requester should remember that only the rows with the latter two values describe the most recent answers for each element.
- The attribute `index_selected` is present in the `workers_urls` dataframe. It can be used to filter the results retrieved by the search engine. The results with a value different from -1 for the attribute have been selected by the worker on the user interface. If its value is equal to 4, three other results have been previously selected. If its value is equal to 7 six other results have been previously selected, and so on. The requester may want to simply analyze the results with whom the worker interacted.
- The attribute `type` is present in the `workers_logs` dataframe. It specifies the type of log record described by each row. The log records are generally sorted using the global timestamp. The requester can use the attribute to split the whole set of log records into subsets of the same type.
- The dataframe `workers_acl` contains several useful information about each worker. The requester may want to merge it with the rows of the other dataframe using the `worker_id` attribute as the key.
- The dataframe `workers_urls` contains the whole set of results retrieved by the search

```
worker_id,paid,task_id,batch_name,unit_id,try_last,try_current,action,time_submit,time_submit_parsed,doc_index,doc_id,doc_fact_check_ground_truth_label,doc_fact_check_ground_truth_value,doc_fact_check_source,doc_speaker_name,doc_speaker_party,doc_statement_date,doc_statement_description,doc_statement_text,doc_truthfulness_value,doc_accesses,doc_time_elapsed,doc_time_start,doc_time_end,global_outcome,global_form_validity,gold_checks,time_spent_check,time_check_amount
ABEFLAGYVQ7IN4,False,Task-Sample,Batch-Sample,unit_1,1,1,Next,"Wed, 09 Nov 2022 10:19:16 GMT",2022-11-09 10:19:16 00:00,0.0,conservative-activist-steve-lonegan-claims-social-,false,1,Politifact,Steve Lonegan,REP,2022-07-12,"stated on October 1, 2011 in an interview on News 12 New Jersey's Power & Politics show:","Today, the Social Security system is broke.",10,1,2.1,1667989144,1667989146.1,False,False,False,False
ABEFLAGYVQ7IN4,False,Task-Sample,Batch-Sample,unit_1,1,1,Next,"Wed, 09 Nov 2022 10:19:25 GMT",2022-11-09 10:19:25 00:00,1,yes-tax-break-ron-johnson-pushed-2017-has-benefite,true,5,Politifact,Democratic Party of Wisconsin,DEM,2022-04-29,"stated on April 29, 2022 in News release:","The tax carve out (Ron) Johnson spearheaded overwhelmingly benefited the wealthiest, over small businesses.",100,1,10.27,1667989146.1,1667989156.37,False,False,False,False,False
```

Listing 1.30: Example of the `workers_answers` dataframe produced by Crowd_Frame.

engine. The dataframe `workers_crawling` contains information about the crawling of each result. The requester may want to merge the rows of the two dataframe `response_uuid` attribute as the key.

- The dataframe `workers_dimensions_selection` shows the temporal ordering along with the workers' chosen answers for the evaluation dimensions. It is ordered using the global timestamp along with each worker making a choice. This means that the rows belonging to a worker may occur in different positions in the dataframe. This may happen if multiple workers perform the task at the same time. The requester should consider this aspect while exploring the dataframe.
- The dataframe `worker_comments` provides the final comments of the worker. The requester should remember that providing a final comment is not mandatory for the worker, thus the dataframe may be empty.

1.9 Conclusions

Crowd_Frame is a relatively young software which still has a long road ahead before becoming truly effective and usable by the whole population of task requesters. There is much that can be done to implement new features and strengthen the overall implementation. Despite everything, the software has been already used to deploy several tasks.

Roitero et al. [57] study whether crowdsourcing be reliably used to assess the truthfulness of information items and to create large-scale labelled collections for information

credibility systems. They deploy a crowdsourcing task to collect thousands of truthfulness judgments over two sets of statements using judgment scales with various granularity levels. Roitero et al. [58] study whether crowdsourcing is an effective and reliable method to assess truthfulness during a pandemic, targeting statements related to COVID-19, thus addressing (mis)information that is both related to a sensitive and personal issue and very recent as compared to when the judgment is done. They deploy a crowdsourcing task where workers are asked to judge the truthfulness of a set of COVID-19 related statements and provide evidence for their judgments. Soprano et al. [62] hypothesize that a unidimensional truthfulness scale is not enough to account for the subtle differences that exist among publicly available statements to fact check. They thus propose a multidimensional notion of truthfulness. They deploy a crowdsourcing task where workers are asked to judge a set of statements with respect to seven different dimensions of truthfulness. Soprano et al. [61] design a questionnaire that aims to understand how longitudinal studies are performed by crowd workers and which factors influence participation across different crowdsourcing platforms. They include the questionnaire within a crowdsourcing task deployed across Amazon Mechanical Turk, Toloka and Prolific to build a large-scale set of answers.

Draws et al. [21] perform a systematic exploratory analysis of publicly available crowdsourced data to identify a set of potential systematic biases that may occur when crowd workers perform crowdsourcing tasks. Then, they deploy a crowdsourcing task to collect a novel set of truthfulness judgments to validate their hypotheses. Brand et al. [8] generate truthfulness predictions for claims using machine learning models and jointly provide a human-readable explanation. Then, they deploy a crowdsourcing task to gather human evaluations on the impact of explanations generated. Ceolin et al. [10] introduce a rating system to identify review arguments and define weighted semantics using formal argumentation theory. Then, they identify and build the argumentation graph, thus deploying crowdsourcing tasks to evaluate their contribution by comparing the results of the argumentation dataset with the upvotes received by the reviews.

The aforementioned crowdsourcing tasks deployed by several researchers and practitioners demonstrate the effectiveness of Crowd_Frame to gather crowd-powered data across multiple platforms and marketplaces.

1.10 Future Work

Crowd_Frame is still characterized by some limitations that need to be addressed in future versions. The requester needs to build manually the overall set to HITs to allocate the element that must be evaluated. The result must be compliant with the format described in Section 1.6.5. The feature provided to allocate automatically the elements is still experimental. The first development direction is stabilizing such a feature. The requester must upload the set of elements manually anyway. The user interface of the Generator component may thus be enriched to facilitate such operation. The interface between Crowd_Frame and the solver used may be extended to specify attributes to characterize and differentiate each worker, such as their level of expertise.

The Generator component is bundled and shipped together with the Skeleton component. The use case diagrams shown in Figure 1.12 and Figure 1.17 show that different actors in-

teract with the two components. Such components, in other words, allow performing two different procedures. On top of that, the user interface of the generator needs to be improved to become truly effective. The two components should have a separate implementation in the future. The generator could be published as a standalone tool and should be able to interact with buckets of the requester. In the past, such a decision has been made to cope with some development difficulties. Initially, Crowd_Frame was meant to be completely client-side. Still today, there is not any kind of backend that gets initialized. The client-side codebase of the application communicates directly with Amazon Web Services. The various building blocks of Crowd_Frame can be put together to build complex tasks. However, it may be hard for inexperienced requesters to understand how to design effective crowdsourcing tasks. The system should offer a set of templates ready to be used to design and eventually customize the most common types of crowdsourcing tasks, as allowed by Amazon Mechanical Turk (Figure 1.1).

Furthermore, Crowd_Frame does not have an interface to monitor the progress of a task deployed. A requester must use the user interface of the crowdsourcing platform chosen to understand and review the status of each HIT. Such an interface will be developed in the future. It will interact with the crowdsourcing platform chosen using the corresponding software development kit and will provide feedback within the system. The software development kits of each crowdsourcing platform allow, among everything, also to automatically publish the tasks and approve each worker submission. The usage of such features will prevent the requester from performing the phases described in Section 1.7 and will speed up the whole workflow. In the future, such a possibility will be provided. However, Crowd_Frame needs a more stable implementation to provide such a feature to task requesters in a safe way.

Crowd_Frame relies on Amazon Web Services to deploy the whole backend infrastructure needed. However, task requesters may want to use different cloud providers for various reasons. In the future, the software will provide the possibility to rely on other providers such as Microsoft Azure[50] or the Google Cloud Platform.[51] Furthermore, the deployment of the whole infrastructure on private servers will be facilitated.

[50] `https://azure.microsoft.com/`
[51] `https://cloud.google.com/`

Bibliography

[1] Lewis R. Aiken. *Rating scales and checklists: Evaluating behavior, personality, and attitudes.* Rating scales and checklists: Evaluating behavior, personality, and attitudes. Oxford, England: John Wiley & Sons, 1996. ISBN: 0-471-12787-6.

[2] Corin R. Anderson, Pedro Domingos, and Daniel S. Weld. Relational Markov Models and Their Application to Adaptive Web Navigation. In: *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '02. Edmonton, Alberta, Canada: Association for Computing Machinery, 2002, pp. 143–152. ISBN: 158113567X. DOI: 10.1145/775047.775068.

[3] Mamoun A. Awad and Issa Khalil. Prediction of User's Web-Browsing Behavior: Application of Markov Model. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 42.4 (2012), pp. 1131–1142. DOI: 10.1109/TSMCB.2012.2187441.

[4] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying Hash Functions for Message Authentication. In: *Advances in Cryptology — CRYPTO '96*. Ed. by Neal Koblitz. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 1–15. ISBN: 978-3-540-68697-2. DOI: 10.1007/3-540-68697-5_1.

[5] Fabrício Benevenuto, Tiago Rodrigues, Meeyoung Cha, and Virgílio Almeida. Characterizing User Behavior in Online Social Networks. In: *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*. IMC '09. Chicago, Illinois, USA: Association for Computing Machinery, 2009, pp. 49–62. ISBN: 9781605587714. DOI: 10.1145/1644893.1644900.

[6] Jose Borges and Mark Levene. Evaluating Variable-Length Markov Chain Models for Analysis of User Web Navigation Sessions. In: *IEEE Transactions on Knowledge and Data Engineering* 19.4 (2007), pp. 441–452. DOI: 10.1109/TKDE.2007.1012.

[7] Erik Brand, Kevin Roitero, Michael Soprano, and Gianluca Demartini. E-BART: Jointly Predicting and Explaining Truthfulness. In: *Proceedings of the 2021 Truth and Trust Online Conference*. Ed. by Isabelle Augenstein, Paolo Papotti, and Dustin Wright. Virtual Event, Oct. 2021, pp. 18–27. URL: https://truthandtrustonline.com/wp-content/uploads/2021/10/TTO2021_paper_16-1.pdf.

[8] Erik Brand, Kevin Roitero, Michael Soprano, Afshin Rahimi, and Gianluca Demartini. A Neural Model to Jointly Predict and Explain Truthfulness of Statements. In: *Journal of Data and Information Quality. Journal Rank: Scimago Q2 (2021)* (July 2022). ISSN: 1936-1955. DOI: 10.1145/3546917.

[9] Jailson Brito, Vaninha Vieira, and Adolfo Duran. Towards a Framework for Gamification Design on Crowdsourcing Systems: The G.A.M.E. Approach. In: *2015 12th International Conference on Information Technology - New Generations*. 2015, pp. 445–450. DOI: 10.1109/ITNG.2015.78.

[10] Davide Ceolin, Giuseppe Primiero, Michael Soprano, and Jan Wielemaker. Transparent assessment of information quality of online reviews using formal argumentation theory. In: *Information Systems* 110 (July 2022), p. 102107. ISSN: 0306-4379. DOI: 10.1016/j.is.2022.102107.

[11] Sara Ceschia, Kevin Roitero, Gianluca Demartini, Stefano Mizzaro, Luca Di Gaspero, and Andrea Schaerf. Task design in complex crowdsourcing experiments: Item assignment optimization. In: *Computers & Operations Research* 148 (2022), p. 105995. ISSN: 0305-0548. DOI: 0.1016/j.cor.2022.105995.

[12] Jesse J. Chandler and Gabriele Paolacci. Lie for a Dime: When Most Prescreening Responses Are Honest but Most Study Participants Are Impostors. In: *Social Psychological and Personality Science* 8.5 (2017), pp. 500–508. DOI: 10.1177/1948550617698203.

[13] Charles Chen, Sungchul Kim, Hung Bui, Ryan Rossi, Eunyee Koh, Branislav Kveton, and Razvan Bunescu. Predictive Analysis by Leveraging Temporal User Behavior and User Embeddings. In: *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. CIKM '18. Torino, Italy: Association for Computing Machinery, 2018, pp. 2175–2182. ISBN: 9781450360142. DOI: 10.1145/3269206.3272032.

[14] Wai Ki Ching, Eric S. Fung, and Michael K. Ng. Higher-order Markov chain models for categorical data sequences*. In: *Naval Research Logistics* 51.4 (2004), pp. 557–574. DOI: 0.1002/nav.20017.

[15] Michael Chmielewski and Sarah C. Kucker. An MTurk Crisis? Shifts in Data Quality and the Impact on Study Results. In: *Social Psychological and Personality Science* 11.4 (2020), pp. 464–473. DOI: 10.1177/1948550619875149.

[16] Benjamin Y Clark, Nicholas Zingale, Joseph Logan, and Jeffrey Brudney. "A framework for using crowdsourcing in government". In: *Social Entrepreneurship: Concepts, Methodologies, Tools, and Applications*. 2019, pp. 405–425. DOI: 10.4018/978-1-5225-8182-6.ch020.

[17] Crowd_Frame: A Simple and Complete Framework to Deploy Complex Crowdsourcing Tasks Off-the-Shelf. In.

[18] Mukund Deshpande and George Karypis. Selective Markov Models for Predicting Web Page Accesses. In: *ACM Transactions on Internet Technology* 4.2 (May 2004), pp. 163–184. ISSN: 1533-5399. DOI: 10.1145/990301.990304.

[19] Djellel Difallah, Elena Filatova, and Panos Ipeirotis. Demographics and Dynamics of Mechanical Turk Workers. In: *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*. WSDM '18. Marina Del Rey, CA, USA: Association for Computing Machinery, 2018, pp. 135–143. ISBN: 9781450355810. DOI: 10.1145/3159652.3159661.

[20] Xing Dongshan and Shen Junyi. A new Markov model for Web access prediction. In: *Computing in Science & Engineering* 4.6 (2002), pp. 34–39. DOI: 10.1109/MCISE.2002.1046594.

[21] Tim Draws, David La Barbera, Michael Soprano, Kevin Roitero, Davide Ceolin, Alessandro Checco, and Stefano Mizzaro. The Effects of Crowd Worker Biases in Fact-Checking Tasks. In: *2022 ACM Conference on Fairness, Accountability, and Transparency*. FAccT '22. Seoul, Republic of Korea: Association for Computing Machinery, 2022, pp. 2114–2124. ISBN: 9781450393522. DOI: 10.1145/3531146.3534629.

[22] Anca Dumitrache, Oana Inel, Lora Aroyo, Benjamin Timmermans, and Chris Welty. CrowdTruth 2.0: Quality Metrics for Crowdsourcing with Disagreement. In: *Proceedings of the 1st Workshop on Subjectivity, Ambiguity and Disagreement in Crowdsourcing, and Short Paper Proceedings of the 1st Workshop on Disentangling the Relation Between Crowdsourcing and Bias Management co-located the 6th AAAI Conference on Human Computation and Crowdsourcing*. Ed. by Lora Aroya, Anca Dumitrache, Praveen K. Paritosh, Alexander J. Quinn, Chris Welty, Alessandro Checco, Gianluca Demartini, Ujwal Gadiraju, and Cristina Sarasua. Vol. 2276. CEUR Workshop Proceedings. Zürich, Switzerland: CEUR-WS.org, 2018, pp. 11–18. URL: http://ceur-ws.org/Vol-2276/paper2.pdf.

[23] Thomas Erickson. Some Thoughts on a Framework for Crowdsourcing. In: *CHI 2011 Workshop on Crowdsourcing and Human Computation*. 2011, pp. 1–4.

[24] Ju Fan, Guoliang Li, Beng Chin Ooi, Kian-lee Tan, and Jianhua Feng. ICrowd: An Adaptive Crowdsourcing Framework. In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. SIGMOD '15. Melbourne, Victoria, Australia: Association for Computing Machinery, 2015, pp. 1015–1030. ISBN: 9781450327589. DOI: 10.1145/2723372.2750550.

[25] Shane Frederick. Cognitive Reflection and Decision Making. In: *Journal of Economic Perspectives* 19.4 (Dec. 2005), pp. 25–42. DOI: 10.1257/089533005775196732.

[26] Ujwal Gadiraju, Sebastian Möller, Martin Nöllenburg, Dietmar Saupe, Sebastian Egger-Lampl, Daniel Archambault, and Brian Fisher. "Crowdsourcing Versus the Laboratory: Towards Human-Centered Experiments Using the Crowd". In: *Evaluation in the Crowd. Crowdsourcing and Human-Centered Experiments*. Ed. by Daniel Archambault, Helen Purchase, and Tobias Hoßfeld. Cham: Springer International Publishing, 2017, pp. 6–26. ISBN: 978-3-319-66435-4. DOI: 10.1007/978-3-319-66435-4_2.

[27] Mihajlo Grbovic, Nemanja Djuric, Vladan Radosavljevic, Fabrizio Silvestri, and Narayan Bhamidipati. Context- and Content-Aware Embeddings for Query Rewriting in Sponsored Search. In: *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '15. Santiago, Chile: Association

for Computing Machinery, 2015, pp. 383–392. ISBN: 9781450336215. DOI: 10.1145/2766462.2767709.

[28] Mihajlo Grbovic, Vladan Radosavljevic, Nemanja Djuric, Narayan Bhamidipati, Jaikit Savla, Varun Bhagwan, and Doug Sharp. E-Commerce in Your Inbox: Product Recommendations at Scale. In: *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '15. Sydney, NSW, Australia: Association for Computing Machinery, 2015, pp. 1809–1818. ISBN: 9781450336642. DOI: 10.1145/2783258.2788627.

[29] Aymen Hamrouni, Hakim Ghazzai, Mounir Frikha, and Yehia Massoud. A Spatial Mobile Crowdsourcing Framework for Event Reporting. In: *IEEE Transactions on Computational Social Systems* 7.2 (2020), pp. 477–491. DOI: 10.1109/TCSS.2020.2967585.

[30] Lei Han, Alessandro Checco, Djellel Difallah, Gianluca Demartini, and Shazia Sadiq. Modelling User Behavior Dynamics with Embeddings. In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. CIKM '20. Virtual Event, Ireland: Association for Computing Machinery, 2020, pp. 445–454. ISBN: 9781450368599. DOI: 10.1145/3340531.3411985.

[31] Lei Han, Kevin Roitero, Ujwal Gadiraju, Cristina Sarasua, Alessandro Checco, Eddy Maddalena, and Gianluca Demartini. All Those Wasted Hours: On Task Abandonment in Crowdsourcing. In: *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. WSDM '19. Melbourne VIC, Australia: Association for Computing Machinery, 2019, pp. 321–329. ISBN: 9781450359405. DOI: 10.1145/3289600.3291035.

[32] Tom L. Han, Kevin Roitero, Ujwal Gadiraju, C. Sarasua, A. Checco, Eddy Maddalena, and Gianluca Demartini. The Impact of Task Abandonment in Crowdsourcing. In: *IEEE Transactions on Knowledge & Data Engineering* 1.1 (Oct. 2019), pp. 1–1. ISSN: 1558-2191. DOI: 10.1109/TKDE.2019.2948168.

[33] Wen-Hua Ju and Yehuda Vardi. A Hybrid High-Order Markov Chain Model for Computer Intrusion Detection. In: *Journal of Computational and Graphical Statistics* 10.2 (2001), pp. 277–295. ISSN: 10618600. URL: http://www.jstor.org/stable/1391012.

[34] Melissa G. Keith, Louis Tay, and Peter D. Harms. Systems Perspective of Amazon Mechanical Turk for Organizational Research: Review and Recommendations. In: *Frontiers in Psychology* 8 (2017). Ed. by Darren C. Treadway, p. 1359. DOI: 10.3389/fpsyg.2017.01359.

[35] Ryan Kennedy, Scott Clifford, Tyler Burleigh, Philip D. Waggoner, Ryan Jewell, and Nicholas J. G. Winter. The shape of and solutions to the MTurk quality crisis. In: *Political Science Research and Methods* 8.4 (2020), pp. 614–629. DOI: 10.1017/psrm.2020.6.

[36] Aniket Kittur, Boris Smus, Susheel Khamkar, and Robert E. Kraut. CrowdForge: Crowdsourcing Complex Work. In: *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology*. UIST '11. Santa Barbara, California, USA: Association for Computing Machinery, 2011, pp. 43–52. ISBN: 9781450307161. DOI: 10.1145/2047196.2047202.

[37] David La Barbera, Kevin Roitero, Gianluca Demartini, Stefano Mizzaro, and Damiano Spina. Crowdsourcing Truthfulness: The Impact of Judgment Scale and Assessor Bias. In: *Advances in Information Retrieval*. Ed. by Joemon M. Jose, Emine Yilmaz, João Magalhães, Pablo Castells, Nicola Ferro, Mário J. Silva, and Flávio Martins. Cham: Springer International Publishing, 2020, pp. 207–214. ISBN: 978-3-030-45442-5. DOI: 10.1007/978-3-030-45442-5_26.

[38] Quoc Le and Tomas Mikolov. Distributed Representations of Sentences and Documents. In: *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32*. ICML'14. Beijing, China: JMLR.org, 2014, pp. II–1188–II–1196. DOI: 10.5555/3044805.3045025.

[39] Ming Li, Jian Weng, Anjia Yang, Wei Lu, Yue Zhang, Lin Hou, Jia-Nan Liu, Yang Xiang, and Robert H. Deng. CrowdBC: A Blockchain-Based Decentralized Framework for Crowdsourcing. In: *IEEE Transactions on Parallel and Distributed Systems* 30.6 (2019), pp. 1251–1266. DOI: 10.1109/TPDS.2018.2881735.

[40] Ting Li, Ning Liu, Jun Yan, Gang Wang, Fengshan Bai, and Zheng Chen. A Markov Chain Model for Integrating Behavioral Targeting into Contextual Advertising. In: *Proceedings of the Third International Workshop on Data Mining and Audience Intelligence for Advertising*. ADKDD '09. Paris, France: Association for Computing Machinery, 2009, pp. 1–9. ISBN: 9781605586717. DOI: 10.1145/1592748.1592750.

[41] Rensis Likert. A technique for the measurement of attitudes. In: *Archives of Psychology* 22.140 (1932), pp. 55–55.

[42] Leib Litman, Aaron Moss, Cheskie Rosenzweig, and Jonathan Robinson. Reply to MTurk, Prolific or panels? Choosing the right audience for online research. In: *Social Science Research Network* (Feb. 2021). DOI: 10.2139/ssrn.3775075.

[43] Leib Litman and Jonathan Robinson. *Conducting Online Research on Amazon Mechanical Turk and Beyond*. 1st ed. Thousand Oaks, California, Dec. 2021. DOI: 10.4135/9781506391151.

[44] Leib Litman, Jonathan Robinson, and Tzvi Abberbock. TurkPrime.com: A versatile crowdsourcing data acquisition platform for the behavioral sciences. In: *Behavior Research Methods* 49.2 (Apr. 2017), pp. 433–442. ISSN: 1554-3528. DOI: 10.3758/s13428-016-0727-z.

[45] Sophia B. Liu. Crisis Crowdsourcing Framework: Designing Strategic Configurations of Crowdsourcing for the Emergency Management Domain. In: *Computer Supported Cooperative Work* 23.4 (Dec. 2014), pp. 389–443. ISSN: 1573-7551. DOI: 10.1007/s10606-014-9204-3.

[46] Eddy Maddalena, Marco Basaldella, Dario De Nart, Dante Degl'Innocenti, Stefano Mizzaro, and Gianluca Demartini. Crowdsourcing Relevance Assessments: The Unexpected Benefits of Limiting the Time to Judge. In: *Proceedings of the Fourth AAAI Conference on Human Computation and Crowdsourcing*. Texas, USA: AAAI Press, 2016, pp. 129–138. URL: http://aaai.org/ocs/index.php/HCOMP/HCOMP16/paper/view/14040.

[47] E. Manavoglu, D. Pavlov, and C.L. Giles. Probabilistic user behavior models. In: *Third IEEE International Conference on Data Mining*. 2003, pp. 203–210. DOI: 10.1109/ICDM.2003.1250921.

[48] Alexandra M. Mellis and Warren K. Bickel. Mechanical Turk data collection in addiction research: utility, concerns and best practices. In: *Addiction* 115.10 (2020), pp. 1960–1968. DOI: 10.1111/add.15032.

[49] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed Representations of Words and Phrases and Their Compositionality. In: *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2*. NIPS'13. Lake Tahoe, Nevada: Curran Associates Inc., 2013, pp. 3111–3119. DOI: 10.5555/2999792.2999959.

[50] Howard R Moskowitz. Magnitude Estimation: Notes on What, How, When, and Why to Use It. In: *Journal of Food Quality* 1.3 (1977), pp. 195–227. DOI: 10.1111/j.1745-4557.1977.tb00942.x.

[51] Stefan Palan and Christian Schitter. Prolific.ac—A subject pool for online experiments. In: *Journal of Behavioral and Experimental Finance* 17 (2018), pp. 22–27. ISSN: 2214-6350. DOI: https://doi.org/10.1016/j.jbef.2017.12.004.

[52] Gabriele Paolacci, Jesse Chandler, and Panagiotis G Ipeirotis. Running experiments on Amazon Mechanical Turk. In: *Judgment and Decision Making* 5.5 (2010), pp. 411–419. URL: https://journal.sjdm.org/10/10630a/jdm10630a.pdf.

[53] Eyal Peer, Laura Brandimarte, Sonam Samat, and Alessandro Acquisti. Beyond the Turk: Alternative platforms for crowdsourcing behavioral research. In: *Journal of Experimental Social Psychology* 70 (2017), pp. 153–163. ISSN: 0022-1031. DOI: 10.1016/j.jesp.2017.01.006.

[54] Eyal Peer, David Rothschild, Andrew Gordon, Zak Evernden, and Ekaterina Damer. Data quality of platforms and panels for online behavioral research. In: *Behavior Research Methods* 54.4 (Aug. 2022), pp. 1643–1662. ISSN: 1554-3528. DOI: 10.3758/s13428-021-01694-3.

[55] Huorong Ren, Zhixing Ye, and Zhiwu Li. Anomaly detection based on a dynamic Markov model. In: *Information Sciences* 411 (2017), pp. 52–65. ISSN: 0020-0255. DOI: 10.1016/j.ins.2017.05.021.

[56] Jonathan Robinson, Cheskie Rosenzweig, Aaron J. Moss, and Leib Litman. Tapped out or barely tapped? Recommendations for how to harness the vast and largely unused potential of the Mechanical Turk participant pool. In: *PLOS ONE* 14.12 (Dec. 2019), pp. 1–29. DOI: 10.1371/journal.pone.0226394.

[57] Kevin Roitero, Michael Soprano, Shaoyang Fan, Damiano Spina, Stefano Mizzaro, and Gianluca Demartini. Can The Crowd Identify Misinformation Objectively? The Effects of Judgment Scale and Assessor's Background. In: *Proceedings of the 43st International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '20. Xi'an, China (Virtual): Association for Computing Machinery, July 2020, pp. 439–448. ISBN: 9781450380164. DOI: 10.1145/3397271.3401112.

[58] Kevin Roitero, Michael Soprano, Beatrice Portelli, Massimiliano De Luise, Damiano Spina, Vincenzo Della Mea, Giuseppe Serra, Stefano Mizzaro, and Gianluca Demartini. Can The Crowd Judge Truthfulness? A Longitudinal Study On Recent Misinformation About COVID-19. In: *Personal and Ubiquitous Computing* (Sept. 2021). ISSN: 1617-4917. DOI: 10.1007/s00779-021-01604-6.

[59] Hongzhou Shen, Junpeng Shi, and Yihan Zhang. CrowdEIM: Crowdsourcing emergency information management tasks to mobile social media users. In: *International Journal of Disaster Risk Reduction* 54 (2021), p. 102024. ISSN: 2212-4209. DOI: 10.1016/j.ijdrr.2020.102024.

[60] Diwakar Shukla and Rahul Singhai. Analysis of Users Web Browsing Behavior Using Markov chain Model. In: *International Journal of Advanced Networking and Applications* 2 (Mar. 2011), pp. 824–830. DOI: 10.35444/IJANA.2021.

[61] Michael Soprano, Kevin Roitero, Ujwal Gadiraju, Eddy Maddalena, and Gianluca Demartini. Longitudinal Loyalty: Understanding the Barriers to Running Longitudinal Studies on Crowdsourcing Platforms. In: *Proceedings of the 2023 The Web Conference*. Under Review. Austin, Texas, USA, May 2023.

[62] Michael Soprano, Kevin Roitero, David La Barbera, Davide Ceolin, Damiano Spina, Stefano Mizzaro, and Gianluca Demartini. The many dimensions of truthfulness: Crowdsourcing misinformation assessments on a multidimensional scale. In: *Information Processing & Management* 58.6 (2021), p. 102710. ISSN: 0306-4573. DOI: 10.1016/j.ipm.2021.102710.

[63] Neil Stewart, Christoph Ungemach, Adam J. L. Harris, Daniel M. Bartels, Ben R. Newell, Gabriele Paolacci, and Jesse Chandler. The average laboratory samples a population of 7,300 Amazon Mechanical Turk workers. In: *Judgment and Decision Making* 10.5 (2015), pp. 479–491. URL: https://journal.sjdm.org/14/14725/jdm14725.pdf.

[64] Jenny Tang, Eleanor Birrell, and Ada Lerner. How Well Do My Results Generalize Now? The External Validity of Online Privacy and Security Surveys. In: (2022). DOI: 10.48550/ARXIV.2202.14036.

[65] Thanh Tran, Kyumin Lee, Yiming Liao, and Dongwon Lee. Regularizing Matrix Factorization with User and Item Embeddings for Recommendation. In: *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. CIKM '18. Torino, Italy: Association for Computing Machinery, 2018, pp. 687–696. ISBN: 9781450360142. DOI: 10.1145/3269206.3271730.

[66] Maja Vukovic. Crowdsourcing for Enterprises. In: *2009 Congress on Services - I*. 2009, pp. 686–692. DOI: 10.1109/SERVICES-I.2009.56.

[67] William Yang Wang. "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. Ed. by Regina Barzilay and Min-Yen Kan. Vol. 4. Association for Computational Linguistics, July 2017, pp. 422–426. DOI: 10.18653/v1/P17-2067.

[68] Margaret A. Webb and June P. Tangney. Too Good to Be True: Bots and Bad Data From Mechanical Turk. In: *Perspectives on Psychological Science* (Nov. 2022), p. 17456916221120027. DOI: 10.1177/17456916221120027.

[69] Cheng Ye, Joseph Coco, Anna Epishova, Chen Hajaj, Henry Bogardus, Laurie Novak, Joshua Denny, Yevgeniy Vorobeychik, Thomas Lasko, Bradley Malin, et al. A Crowdsourcing Framework for Medical Data Sets. In: *AMIA Summits on Translational Science Proceedings* 2018 (2018), p. 273. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5961774/pdf/2840819.pdf.

[70] Arkaitz Zubiaga, Maria Liakata, Rob Procter, Kalina Bontcheva, and Peter Tolmie. Crowdsourcing the Annotation of Rumourous Conversations in Social Media. In: *Proceedings of the 24th International Conference on World Wide Web*. WWW '15 Companion. Florence, Italy: Association for Computing Machinery, 2015, pp. 347–353. ISBN: 9781450334730. DOI: 10.1145/2740908.2743052.

Analytical Index

Symbols

p, 68

A

Amazon
- Mechanical Turk, 1–7, 11–13, 16, 24, 49, 64, 66, 76, 77, 84, 88, 89
- Web Services
  - API Gateway, 43, 52
  - Command Line Interface, 60
  - DynamoDB, 17, 23, 25, 28, 32, 53
  - Elastic Compute Cloud, 46
  - Free Tier, 61
  - Identity Access Management, 60
  - Lambda, 44, 47, 53
  - S3, 17, 26, 32
  - Simple Queue Service, 43, 52

AMD EPYC, 46
Angular, 44, 66

B

Bing
- Web Search, 35, 42
  - API, 35

C

Cognitive
- Reflection Test, 19, 20

COPY, 26
COVID-19, 88
Crowd Elements, 5, 7
Crowd_Frame, 1, 4, 16–30, 32–35, 37, 40–46, 49, 54, 59–68, 70, 71, 73–77, 79, 80, 85–89
CSS, 8, 13, 78
CSV, 7, 11, 25, 81
cURL, 36, 41

D

DataFrame, 81
DELETE, 40
Directive, 44
Docker, 60
- Hub, 47

E

Entrez Programming Utilities, 37, 43
- CitMatch, 37
- GQuery, 37
- Info, 37
- Link, 37
- Post, 37
- Search, 37, 39

Spell, 37
Summary, 37, 39

F

fakeJSON, 40, 41, 43
FakerWebSearch, 40, 43

G

Generator, 16, 17, 19, 20, 63, 65, 66, 70, 71, 88
GitHub, 59
Google
Cloud Platform, 89

H

HIT, 2, 5, 7, 8, 11, 13, 16, 17, 19, 20, 22–27, 41, 42, 49, 50, 59, 63–68, 70, 71, 74, 81, 85, 86, 88, 89
HMAC, 65

I

Intel Xeon Platinum 8000, 47

J

JavaScript, 8, 13, 44, 78
JSON, 8, 11, 25, 36, 65–67, 70, 80

L

Likert, 19
LIST, 26
Locust, 46, 47
Logger, 16, 17, 29, 45, 46, 49–51, 54

M

Mako, 66
MEDLINE, 37
Microsoft Azure, 89

N

Nginx, 70
Node.js, 60
NoSQL, 17, 25

O

Ocp-Apim-Subscription-Key, 35

P

PATCH, 40
PolitiFact, 71
POST, 26, 40
Prolific, 1, 3, 4, 12–16, 24, 26, 63, 76, 78, 84, 88
PubMed, 37
Search, 39, 43
PUT, 26, 40
Python, 60

Q

Qualification Type, 5

R

RRU, 29
RxJS, 45

S

Search Engine, 16, 17, 19, 33–35, 37, 40, 41
Skeleton, 16, 17, 22, 23, 25, 26, 30, 32, 41, 88
Standard, 19, 26

T

Toloka, 1, 4, 7–12, 16, 24, 61, 64, 66, 67, 76, 77, 84, 88

TPS, 42
TSV, 66

U

Ubuntu, 47
UUID, 81

W

WRU, 29

X

XLSX, 11

Y

Yarn, 60